



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ ПО ПРОЕКТУ

по дисциплине «Системная и программная инженерия»

Студент группы ИКБО-50-23

Павлов Н.С.

(подпись студента)

Руководитель проектной работы

Мельников Д.А.

(подпись руководителя)

Работа представлена

« ____ » _____ 2026 г.

Допущен к работе

« ____ » _____ 2026 г.

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	2
1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА	4
1.1. Целевая аудитория	4
1.1.1. Основной портрет	4
1.1.2. Второстепенный портрет	4
1.2. Требования к системе	4
1.2.1. User Story	4
1.2.2. Функциональные требования	5
1.2.3. Нефункциональные требования	6
1.2.4. Матрица требований	8
1.3. Техническое задание.....	14
1.4. Программа и методика испытаний	15
1.5. План реализации проекта	15
1.6. Анализ и оценка рисков при разработке программного проекта	16
1.6.1. Описание рисков проекта	16
1.6.2. Оценка вероятности и силы последствий	18
1.6.3. Матрица рисков	20
1.6.4. Планы реагирования на риски	21
1.7. Анализ эксплуатации проекта	23
1.7.1. Анализ технологических угроз и их влияния на функционал	23
1.7.2. Преобразование технологических угроз в риски	24
1.7.3. Методы обработки рисков эксплуатации	25
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ	27
2.1. Поведенческие диаграммы	27
2.1.1. Диаграмма вариантов использования	27
2.1.1.1. Обоснование сценариев использования	27
2.1.1.2. Схематическое представление (Use Case Diagram)	27
2.1.2. Диаграмма последовательности	28
2.2. Процесс работы в системе	29
2.3. Структурные диаграммы	30
2.3.1. Диаграмма классов	30
2.3.2. Диаграмма объектов	31
2.4. Информационное взаимодействие компонентов системы	31
2.4.1. Текстовое описание	31
2.4.2. Диаграмма DFD	32
2.5. Модель Базы данных	36

2.6.	Архитектура приложения	37
2.6.1.	Обоснование выбора	38
2.6.2.	Диаграмма архитектуры	40
2.7.	Проектирование интерфейса	41
2.7.1.	Наполнение экранов.....	41
2.7.2.	Обоснование дизайн-решений.....	41
2.7.3.	Макеты экранов и прототип	43
3.	РАЗРАБОТКА ПРИЛОЖЕНИЯ	47
3.1.	Методология управления процессом разработки	47
3.2.	Инструменты разработки ПО	48
3.3.	Создание удаленного git репозитория	49
3.4.	Настройка системы сборки Gradle	50
3.5.	Реализация ключевых компонентов	52
3.5.1.	Frontend.....	52
3.5.1.1.	Структура слоя.....	52
3.5.1.2.	Листинги компонентов интерфейса:	54
3.5.1.2.1.	Структура и навигация:	54
3.5.1.2.2.	Основные экраны:.....	58
3.5.1.2.3.	Компоненты интерфейса:	100
3.5.1.2.4.	Состояние и управление:.....	118
3.5.1.2.5.	Ресурсы и визуальные элементы:.....	123
3.5.2.	Backend	149
3.5.2.1.	Структура слоя.....	149
3.5.2.2.	Листинги компонентов интерфейса:	151
3.5.2.2.1.	Модель данных:	151
3.5.2.2.2.	Репозиторий и вспомогательные утилиты:	151
3.5.2.2.3.	Конфигурация и инициализация БД:	156
3.5.2.2.4.	Операции с категориями (CRUD):	161
3.5.2.2.5.	Операции с транзакциями доходов (CRUD):	165
3.5.2.2.6.	Операции с транзакциями расходов (CRUD):.....	169
3.6.	Документирование кода	173
3.6.1.	Документация разработчика.....	173
3.6.2.	Документация пользователя	173
4.	Тестирование приложения	175
4.1.	Тестовые сценарии (Test Cases)	175
4.2.	Инструменты для проведения тестирования.....	179
4.3.	Тестирование пользовательского интерфейса	181
4.4.	Чек-лист тестирования (Check-list).....	186
	ЗАКЛЮЧЕНИЕ	189

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	191
--	-----

ВВЕДЕНИЕ

Современная жизнь требует от человека, особенно от студента, грамотного управления личными финансами. Проект "Finlytics: Офлайн-менеджер финансов" на языке Kotlin направлен на разработку десктопного приложения, которое предоставит пользователю удобный инструмент для учета доходов и расходов, категоризации трат и визуализации финансовой статистики без необходимости подключения к интернету. Основной целью является создание простого, интуитивно понятного и производительного приложения, которое помогает пользователям контролировать свой бюджет.

Краткое описание проекта:

Проект представляет собой десктопное приложение "Finlytics" для управления персональными финансами. Ключевые функции: внесение операций "доход" и "расход" с возможностью их категоризации, фильтрация данных по различным временным промежуткам, а также наглядное представление финансовой статистики в виде диаграмм и списков.

Проблематика:

Актуальность проекта обусловлена необходимостью простого и доступного инструмента для планирования и контроля личного бюджета. Многие существующие решения либо требуют постоянного подключения к интернету, либо обладают избыточной сложностью, либо неудобны в повседневном использовании. Студенты, как социальная группа с часто ограниченным бюджетом, особенно нуждаются в таком инструменте для формирования навыков финансовой грамотности.

Для достижения цели необходимо выполнить следующие задачи:

- Задача 1: Разработать архитектуру данных и пользовательский интерфейс для удобного внесения и редактирования финансовых операций.
- Задача 2: реализовать механизм категоризации доходов и расходов с возможностью добавления пользовательских категорий.

- Задача 3: внедрить систему фильтрации и анализа данных по временным промежуткам (день, неделя, месяц, год, все время) с визуализацией результатов в виде диаграмм и отчетов.

Состав команды и распределение ролей:

- Враженко Д.О. – Team Lead, менеджер проекта, разработчик, тестировщик, технический писатель. Ответственный за постановку задач, общую архитектуру проекта, разработку ключевых модулей и составление отчетности.
- Хохряков А.Ю. – Аналитик, разработчик, тестировщик, технический писатель. Ответственный за реализацию модуля работы с данными и бизнес-логики.
- Павлов Н.С. – UI/UX дизайнер, разработчик, тестировщик, технический писатель. Ответственный за проектирование и реализацию пользовательского интерфейса, включая визуализацию данных (диаграммы).

1. ПЛАНИРОВАНИЕ И ПОДГОТОВКА

1.1. Целевая аудитория

1.1.1. Основной портрет

Студент, начинающий контролировать личный бюджет.

Это основная целевая аудитория. Данный пользователь — молодой человек или девушка, часто впервые сталкивающийся с необходимостью самостоятельного управления финансами (стипендия, подработки, деньги от родителей). Его бюджет ограничен, и он ищет простой и понятный инструмент, чтобы отслеживать, на что уходят деньги, и учиться планировать траты.

1.1.2. Второстепенный портрет

Занятый профессионал, ценящий приватность и офлайн-доступ.

Этот пользователь — работающий человек, который уже имеет некоторый финансовый опыт, но предпочитает держать свои данные под полным контролем. Он ценит скорость и эффективность, у него нет времени на изучение сложного софта. Ему нужен надежный инструмент для быстрой фиксации основных операций и получения сводки без подключения к интернету.

1.2. Требования к системе

1.2.1. User Story

Таблица 1.1 – User Story

Кто?	Что хочет?	С какой целью?
Пользователь	Чтобы при первом запуске в приложении уже был предустановленный набор категорий	Чтобы не приходилось тратить время на их создание с нуля, и пользователь мог сразу начать вносить свои траты

Пользователь	Быстро добавить информацию о том, что потратил деньги, указав сумму, категорию и дату	Чтобы точно фиксировать все траты
Пользователь	Внести запись о получении денег	Чтобы видеть полную картину своих финансов
Пользователь	Иметь возможность создавать свои собственные категории для доходов и расходов	Чтобы не ограничиваться предустановленными категориями
Пользователь	Когда открывается главный экран, отображать, какой баланс, а также общую сумму доходов и расходов за выбранный период	Чтобы быстро оценить свое финансовое состояние
Пользователь	Видеть наглядную диаграмму, которая показывает, на что именно тратятся деньги больше всего	Чтобы понимать структуру своих расходов
Пользователь	Иметь возможность посмотреть свою финансовую статистику и историю операций не только за всё время, но и за конкретный период (день, неделю, месяц или год)	Чтобы отслеживать динамику трат и доходов
Пользователь	Иметь возможность открыть запись и редактировать её	Чтобы устранить ошибку при записи данных
Пользователь	Иметь возможность удалить запись из истории	Чтобы она не учитывалась в общей статистике
Пользователь	Иметь возможность удалять созданные мной категории	Чтобы убрать неиспользуемые категории
Пользователь	Иметь возможность сравнивать свои текущие показатели трат с предыдущим месяцем	Чтобы понимать тенденцию и частоту своих трат и корректировать свои планы для большей экономии.

1.2.2. Функциональные требования

На основе описанных User Story были сформулированы следующие функциональные требования проекта:

- Система должна автоматически создавать предустановленный список категорий доходов и расходов при первом запуске приложения (инициализация БД).
- Пользователь должен иметь возможность создавать новые категории доходов и расходов.
- Пользователь должен иметь возможность удалять существующие категории.

- Система должна блокировать удаление категории, если она используется хотя бы в одной операции.
- Пользователь должен иметь возможность добавлять операции типа "Доход" и "Расход".
- Операция должна содержать: сумму, категорию, дату и описание (необязательно).
- Система должна валидировать вводимые данные: сумма должна быть положительным числом, дата — корректной, категория — выбрана из списка.
- Пользователь должен иметь возможность редактировать любую существующую операцию.
- Пользователь должен иметь возможность удалять любую существующую операцию.
- На главном экране отображать сводку: баланс, общие доходы и расходы за выбранный период.
- На главном экране отображать круговую диаграмму распределения операций по категориям (с переключением "Доходы/Расходы").
- На экране истории отображать полный список операций.
- Реализовать фильтрацию данных на главном экране и в истории по временным промежуткам (день, неделя, месяц, год, все время).
- Реализовать фильтрацию в истории по типу операции (Все/Доходы/Расходы).
- Реализовать timeline с моделью «Призрака» с показателями трат за текущий месяц в сравнении с прошлым месяцем.

1.2.3. Нефункциональные требования

Учитывая, что проект представляет собой десктопное офлайн-приложение, были определены следующие нефункциональные требования:

Таблица 1.2 – Нефункциональные требования

Тип требования	Содержание требования
Технические ограничения	Десктопное приложение должно функционировать на операционных системах семейства Windows (10 и выше), macOS (11 и выше) и Linux (дистрибутивы с поддержкой JRE).
	Приложение должно распространяться в виде исполняемого JAR-файла, для работы которого требуется Java Runtime Environment (JRE) версии 8 или выше.
	Для хранения данных должно использоваться исключительно локальное хранилище (SQLite) без необходимости подключения к интернету.
Производительность	Время отклика интерфейса на действия пользователя (открытие экрана, добавление операции, применение фильтра) не должно превышать 500 миллисекунд.
	Запуск приложения и загрузка начальных данных (история операций и категории) должны занимать не более 3 секунд.
	Операции фильтрации списка транзакций и пересчета статистики по временным периодам должны выполняться мгновенно, без заметных задержек.
Надежность	Приложение должно корректно завершать работу и сохранять целостность данных при закрытии окна или выключении компьютера.
	Сбои в работе отдельных компонентов (например, при попытке удалить используемую категорию) не должны приводить к полной остановке или некорректному состоянию приложения.
	Так как приложение является однопользовательским и однопоточным (в рамках одного окна), но для надежности, при любых операциях с БД должен использоваться корректный механизм блокировок, предоставляемый SQLite, чтобы предотвратить повреждение данных при гипотетических попытках записи из нескольких потоков.
Безопасность	Локальная база данных (SQLite) не должна быть доступна для чтения или изменения другими приложениями напрямую (защита на уровне файловой системы).
	Приложение должно блокировать SQL-инъекции при формировании запросов к базе данных (за счет использования параметризованных запросов в коде).
	Приложение не должно требовать прав администратора (root) для своей работы, чтобы минимизировать риски, связанные с повышением привилегий. Все операции с файлами должны выполняться в контексте прав текущего пользователя.
Удобство использования	Интерфейс приложения должен быть интуитивно понятным и не требовать от пользователя изучения дополнительной документации для выполнения основных сценариев (добавление, редактирование, удаление).
	Все ключевые элементы управления (кнопки, поля ввода) должны быть снабжены всплывающими подсказками (ToolTips), поясняющими их назначение.
	Система должна предоставлять пользователю понятные сообщения об ошибках ввода данных (например, при вводе некорректной суммы или даты).

1.2.4. Матрица требований

Матрица объединяет функциональные требования, выявленные на этапе планирования, и нефункциональные требования, сформулированные выше. Она структурирована по основным компонентам приложения и источникам.

Таблица 1.3 – Матрица требований

№	Требование	Суть	Автор	Ссылки	Критерии проверки	Компоненты архитектуры	Результат тестирования
1	Клиентское приложение (Desktop)						
1.1	Управление операциями						
1.1.1	Добавление операции	Пользователь должен иметь возможность добавить новую финансовую операцию с указанием типа (доход/расход), суммы, категории, даты и описания.	Павлов Н.С.	User Story (Таблица 1.1 – User Story)	При нажатии на кнопку "Добавить" открывается диалоговое окно, в котором можно ввести все параметры и сохранить операцию.	UI-компоненты (диалог добавления операции) ViewModel (обработка события сохранения) Репозиторий (координация записи) Слой данных (запись в таблицы транзакций)	Выполнено. Операция успешно добавляется и отображается.
1.1.2	Редактирование операции	Пользователь должен иметь возможность открыть существующую запись и изменить любые её параметры.	Павлов Н.С.	User Story (Таблица 1.1 – User Story)	При нажатии на кнопку "Редактировать" у элемента списка открывается диалог с пред заполненными полями, после изменения и сохранения данные	UI-компоненты (список операций, диалог редактирования) ViewModel (хранение редактируемой операции) Репозиторий (координация	Выполнено. Данные операции корректно обновляются.

					обновляются.	обновления) Слой данных (обновление записей)	
1.1. 3	Удаление операции	Пользователь должен иметь возможность удалить запись из истории, при этом она перестает учитываться в статистике.	Павлов Н.С.	User Story (Таблица 1.1 – User Story)	При нажатии на кнопку "Удалить" появляется диалог подтверждения. После подтверждения операция исчезает из списка и не влияет на баланс и диаграммы.	UI-компоненты (диалог подтверждения) ViewModel (обработка события удаления) Репозиторий (координация удаления) Слой данных (удаление записей)	Выполнено. Операция удаляется, статистика пересчитываетс я.
1.2	Управление категориями						
1.2. 1	Предустановленные категории	При первом запуске в приложении должен быть предустановленн ый набор категорий для доходов и расходов.	Хохряко в А.Ю.	User Story (Таблица 1.1 – User Story)	После чистой установки и первого запуска в настройках отображаются категории: Зарплата, Стипендия, Еда, Транспорт и др.	Слой данных (инициализатор БД) SQLite (таблицы категорий)	Выполнено. Все стандартные категории созданы.
1.2. 2	Создание пользовательских категорий	Пользователь должен иметь возможность создавать свои собственные категории для доходов и расходов.	Хохряко в А.Ю.	User Story (Таблица 1.1 – User Story)	В экране "Настройки" при нажатии на кнопку "+" появляется диалог, позволяющий ввести имя новой категории и добавить её в соответствующий	UI-компоненты (диалог добавления категории) ViewModel (проверка уникальности) Репозиторий (координация добавления) Слой данных	Выполнено. Новая категория добавляется и доступна для выбора.

					список.	(запись в таблицы категорий)	
1.2. 3	Удаление категорий	Пользователь должен иметь возможность удалять созданные им категории. Система должна блокировать удаление категории, если она используется в операциях.	Хохряко в А.Ю.	User Story (Таблица 1.1 – User Story)	При попытке удалить категорию, которая привязана к какой-либо операции, должно появляться предупреждение, и удаление не должно выполняться.	UI-компоненты (кнопка удаления) ViewModel (проверка наличия связанных операций) Репозиторий (координация удаления) Слой данных (удаление с проверкой внешних ключей)	Выполнено. Удаление заблокировано, предупреждение отображено.
1.3	Аналитика и просмотр						
1.3. 1	Финансовая сводка	На главном экране должна отображаться сводка: баланс, общая сумма доходов и расходов за выбранный период.	Враженко Д.О.	User Story (Таблица 1.1 – User Story)	На экране "Обзор" при выборе периода (День/Неделя/Месяц) цифры в блоках "Доходы", "Расходы" и "Баланс" пересчитываются.	UI-компоненты (блоки статистики) ViewModel (расчет сводных показателей) Репозиторий (предоставление данных) Слой данных (извлечение транзакций)	Выполнено. Показатели пересчитываются корректно.
1.3. 2	Визуализация данных	На главном экране должна отображаться круговая диаграмма, показывающая структуру доходов или	Враженко Д.О.	User Story (Таблица 1.1 – User Story)	При переключении между режимами "Доходы" и "Расходы" на главном экране меняется диаграмма и список категорий,	UI-компоненты (кастомный компонент диаграммы) ViewModel (группировка данных по категориям)	Выполнено. Диаграмма и список категорий обновляются.

		расходов по категориям за выбранный период.			отображая соответствующие данные.	Репозиторий (предоставление данных) Слой данных (извлечение транзакций)	
1.3.3	Фильтрация истории	На экране истории пользователь должен иметь возможность фильтровать список операций по временному промежутку (день, неделя, месяц, год, все время) и по типу операции (все, доходы, расходы).	Враженк о Д.О.	User Story (Таблица 1.1 – User Story)	При изменении параметров фильтрации на экране "История" список операций должен обновляться в соответствии с выбранными критериями.	UI-компоненты (элементы фильтрации) ViewModel (применение фильтров) Репозиторий (предоставление данных) Слой данных (извлечение транзакций)	Выполнено. Список операций обновляется по критериям.
2	Правовые нормы и стандарты						
2.1	Защита персональных данных	Локальная база данных с персональными финансовыми данными пользователя не должна быть доступна для чтения сторонними приложениями.	Враженк о Д.О.	ГОСТ 34.602-2020, Федеральный закон № 152-ФЗ "О персональных данных"	При проверке файловой системы, файл базы данных (Finlytics.db) не должен открываться стандартными средствами ОС или другими программами без специального доступа.	SQLite (файл базы данных) ОС (права доступа к файлам)	Выполнено. Файл защищен на уровне файловой системы.
2.2	Соответствие	Исходный код	Враженк	ГОСТ 34.201-	При проверке	Все компоненты	Выполнено.

	стандартам документирования	должен быть документирован в соответствии со стандартом KDoc для языка Kotlin.	о Д.О.	2020, официальная документация Kotlin	исходного кода, все публичные классы и функции должны содержать KDoc-комментарии с описанием.	системы (документирование кода)	KDoc присутствует, сгенерирована документация Dokka.
3	Технические и нефункциональные ограничения						
3.1	Кроссплатформенность	Приложение должно работать в ОС Windows, macOS и Linux.	Враженко Д.О.	Анализ рынка аналоговых приложений	Собранный JAR-файл успешно запускается и корректно отображает интерфейс, а также выполняет все функции на каждой из трех целевых ОС.	Kotlin/JVM (кроссплатформенная среда) Compose for Desktop (кроссплатформенный UI) JRE (среда выполнения)	Выполнено. Проверено на Windows 11 и Ubuntu 22.04.
3.2	Производительность фильтрации	Операции фильтрации данных по временным промежуткам должны выполняться мгновенно, без заметных задержек для пользователя.	Павлов Н.С.	Нефункциональные требования (1.2.3)	При переключении между вкладками "День/Неделя/Месяц" на экране "Обзор" или "История" интерфейс обновляется быстрее, чем за 0.5 секунды.	ViewModel (оптимизированная обработка) Репозиторий (эффективное предоставление данных) SQLite (оптимизированные запросы)	Выполнено. Время отклика менее 300 мс.
3.3	Безопасность данных	Приложение должно использовать параметризованные SQL-запросы для исключения	Хохряков А.Ю.	Нефункциональные требования (1.2.3)	Проверка кода: все обращения к БД в объектах AddCategory, AddIncomeTransaction и др. должны	Слой данных (параметризованные запросы через JDBC)	Выполнено. Инъекция не прошла, везде используется PreparedStatement.

		возможности повреждения БД некорректным вводом.			использовать PreparedStatement с плейсхолдерами (?).		
--	--	---	--	--	--	--	--

1.3. Техническое задание

При разработке технического задания (ТЗ) на создание программного продукта «Finlytics: Офлайн-менеджер финансов» в качестве основного руководящего документа был выбран межгосударственный стандарт ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Выбор данного стандарта обусловлен следующими факторами:

1. Целевая направленность. ГОСТ 34.602-2020 входит в комплекс стандартов на автоматизированные системы (АС) и специально разработан для регламентации процесса создания технического задания. Он устанавливает четкий состав, содержание и правила оформления документа, что позволяет структурировать требования к разрабатываемому продукту на самом раннем этапе его жизненного цикла.

2. Универсальность и полнота. Несмотря на то, что стандарт разработан достаточно давно, он носит универсальный характер и не привязан к конкретным технологиям или архитектурам. Структура ТЗ, предлагаемая стандартом, в полной мере подходит для описания широкого класса систем, включая современные десктопные приложения, такие как Finlytics. Это позволило всесторонне описать проект, не упустив ключевых аспектов: от целевой аудитории до технических требований.

3. Соответствие специфике проекта (Анализ применимости разделов). Все ключевые разделы стандарта были успешно адаптированы под особенности разрабатываемого офлайн-приложения.

4. Актуальность редакции. Версия стандарта 2020 года является действующей и актуальной. Использование устаревших редакций могло бы привести к несоответствию современным требованиям к оформлению и содержанию проектной документации.

Составленное ТЗ на разработку проекта представлено в файле ниже:



1.4. Программа и методика испытаний

По завершению разработки все модули программного продукта и документации должны пройти полное тестирование на соответствие требованиям ТЗ и настоящего документа.

Составленная программа и методика испытаний приведена в файле ниже:



1.5. План реализации проекта

После детального анализа и выявления требований к системе был составлен план работ и распределены задачи и обязанности между участниками проекта в сервисе Miro (Рисунок 1.1).

	Title	Description	Status	Person	Start Date	End Date	
1	Выбор темы	Выбор темы проекта, определение основных идей. Распределение ролей внутри проекта	Complete	Даниил n1k_pavlov Andrey...	Feb 10, 2026	Feb 10, 2026	
2	Выявление требований	Выявление функциональных и нефункциональных требований, Построение диаграммы вариантов использования и диаграммы последовательности. Создание матрицы требований	Complete	Andrey... Даниил	Feb 10, 2026	Feb 24, 2026	
3	Проектирование сиситемы	Проектирование логики работы системы в различных нотациях (IDEF0, DFD, UML...)	Not Started	Andrey... n1k_pavlov			
4	Разработка ТЗ	Оформление итогового ТЗ, включающего все требования и модели	In Progress	Даниил	Feb 17, 2026		
5	Создание прототипа интерфейса	Создание макетов экранов в Figma, Разработка дизайн-системы (цветовая палитра, типографика, отступы). Подготовка кликабельного прототипа для демонстрации пользовательского пути	In Progress	n1k_pavlov	Feb 15, 2026		
6	Разработка модуля работы с БД	Разработка модулей для подключения к SQLite и инициализации базы данных. Реализация операций CRUD для категорий и транзакций. Формирование единого репозитория для доступа к данным	Not Started	Andrey...			
7	Разработка пользовательского интерфейса	Верстка главного экрана (обзорная статистика, фильтры, диаграмма). Верстка экрана истории операций (список, фильтрация). Верстка экрана настроек (управление категориями). Реализация диалогов для добавления/редактирования операций и категорий. Разработка настомной диаграммы для визуализации	Not Started	n1k_pavlov			
8	Связь модулей согласно MVVM	Написание базовой логики ViewModel (управление состоянием, обработка действий пользователя). Интеграция слоя данных с ViewModel. Подключение интерфейса к ViewModel (реактивное обновление данных). Реализация навигации между экранами. Интеграция диалоговых окон	Not Started	Даниил			
9	Тестирование	Тестирования всех сценариев использования. Тестирование корректности работы с базой данных (целостность, запросы). Тестирование пользовательского интерфейса на соответствие макетам	Not Started	Даниил n1k_pavlov Andrey...			
10	Подготовка отчетной документации	Документирование ключевых компонентов кода. Разработка руководства пользователя и прочей отчетной документации	Not Started	Даниил n1k_pavlov Andrey...			
11	Подготовка к презентации продукта	Демонстрация дизайн-решений и реализованного интерфейса. Проверка работоспособности приложения на целевых платформах. Презентация готового продукта	Not Started	Даниил n1k_pavlov Andrey...			

Рисунок 1.1 — Kanban-доска проекта

1.6. Анализ и оценка рисков при разработке программного проекта

1.6.1. Описание рисков проекта

На основе анализа проектной документации и специфики разработки приложения «Finlytics» были выявлены следующие риски, представленные в формате «причина-риск-эффект».

Таблица 1.4 — Риски проекта «Finlytics»

№	Риск (причина-риск-эффект)
1	– Члены команды имеют разный уровень навыков работы с технологией Compose для Desktop. – Некоторые компоненты UI (например, кастомная круговая диаграмма) могут потребовать больше времени на реализацию, чем ожидалось. – Задержка в разработке пользовательского интерфейса и его интеграции с ViewModel.
2	– В проекте используется несколько взаимосвязанных модулей (UI, ViewModel, Repository, DB) и система сборки Gradle. – При слиянии изменений из разных веток (Павлов, Хохряков, Враженко) могут возникать конфликты, нарушающие сборку проекта. – Потеря времени на разрешение конфликтов и восстановление работоспособности приложения.
3	– Тестирование интеграции с базой данных SQLite может быть пропущено на ранних этапах. – SQL-запросы могут оказаться неоптимальными, или структура БД может потребовать изменений при росте объема данных. – Снижение производительности при фильтрации и отображении большого количества операций (нефункциональное требование <500 мс может быть нарушено).
4	– Студенческий график членов команды (сессия, другие проекты) может меняться непредсказуемо. – Участник команды (например, Team Lead Враженко Д.О.) может временно выпасть из процесса разработки. – Отсутствие единого центра принятия решений, задержки в решении критических задач и демотивация команды.
5	– Заказчик (преподаватель) или пользователи могут предоставить обратную связь на этапе демонстрации промежуточного результата. – Может потребоваться доработка интерфейса или добавление новой функциональности, не предусмотренной изначальным ТЗ. – Увеличение трудозатрат и сдвиг сроков сдачи проекта.
6	– В проекте используется Kotlin Coroutines и Flow для асинхронных операций. – Неправильная обработка исключений в корутинах или обновление состояния (StateFlow) из неправильного потока может привести к нестабильной работе или крашу приложения. – Нарушение надежности приложения (нефункциональное требование), трудноуловимые ошибки.
7	– Приложение использует локальную базу данных SQLite, и все данные хранятся в одном файле. – При некорректном завершении работы приложения или сбое в системе во время записи данных файл БД может быть поврежден. – Полная или частичная потеря всех финансовых данных пользователя, невозможность запуска приложения.
8	– Приложение распространяется как исполняемый JAR-файл и требует наличия JRE. – На компьютере пользователя может отсутствовать или быть устаревшая версия Java Runtime Environment. – Пользователь не сможет запустить приложение, получив ошибку о невозможности найти или загрузить главный класс.
9	– Для управления проектом используется Kanban-доска в Miro и Git для кода.

	– Участники могут забывать обновлять статусы задач на доске или оставлять неинформативные сообщения в коммитах. – Потеря актуального представления о прогрессе по проекту, сложности в планировании и выявлении отставания.
10	– В проекте реализована сложная логика фильтрации на главном экране и в истории. – При определенном сочетании фильтров (период «Неделя» + тип «Расходы») могут возникать ошибки округления или неверно рассчитываться проценты для категорий. – Пользователь видит некорректные финансовые данные, что подрывает доверие ко всему приложению.

1.6.2. Оценка вероятности и силы последствий

Произведем оценку каждого риска по шкале от 1 до 10, где 1 — очень низкая вероятность/несущественный ущерб, а 10 — очень высокая вероятность/катастрофический ущерб. На их основе рассчитаем важность (Вероятность * Последствия).

Таблица 1.5 — Оценка важности рисков

№	Риск (причина-риск-эффект)	Вероятность (1-10)	Последствия (1-10)	Важность
1	– Члены команды имеют разный уровень навыков работы с технологией Compose для Desktop. – Некоторые компоненты UI (например, кастомная круговая диаграмма) могут потребовать больше времени на реализацию, чем ожидалось. – Задержка в разработке пользовательского интерфейса и его интеграции с ViewModel.	8	6	48
2	– В проекте используется несколько взаимосвязанных модулей (UI, ViewModel, Repository, DB) и система сборки Gradle. – При слиянии изменений из разных веток (Павлов, Хохряков, Враженко) могут возникать конфликты, нарушающие сборку проекта. – Потеря времени на разрешение конфликтов и восстановление работоспособности приложения.	8	2	16
3	– Тестирование интеграции с базой данных SQLite может быть пропущено на ранних этапах. – SQL-запросы могут оказаться неоптимальными, или структура БД	6	8	48

	может потребовать изменений при росте объема данных. – Снижение производительности при фильтрации и отображении большого количества операций (нефункциональное требование <500 мс может быть нарушено).			
4	– Студенческий график членов команды (сессия, другие проекты) может меняться непредсказуемо. – Участник команды (например, Team Lead Враженко Д.О.) может временно выпасть из процесса разработки. – Отсутствие единого центра принятия решений, задержки в решении критических задач и демотивация команды.	9	4	36
5	– Заказчик (преподаватель) или пользователи могут предоставить обратную связь на этапе демонстрации промежуточного результата. – Может потребоваться доработка интерфейса или добавление новой функциональности, не предусмотренной изначальным ТЗ. – Увеличение трудозатрат и сдвиг сроков сдачи проекта.	6	8	48
6	– В проекте используется Kotlin Coroutines и Flow для асинхронных операций. – Неправильная обработка исключений в корутинах или обновление состояния (StateFlow) из неправильного потока может привести к нестабильной работе или крашу приложения. – Нарушение надежности приложения (нефункциональное требование), трудноуловимые ошибки.	4	7	28
7	– Приложение использует локальную базу данных SQLite, и все данные хранятся в одном файле. – При некорректном завершении работы приложения или сбое в системе во время записи данных файл БД может быть поврежден. – Полная или частичная потеря всех финансовых данных пользователя, невозможность запуска приложения.	3	10	30
8	– Приложение распространяется как исполняемый JAR-файл и требует наличия JRE.	5	9	45

	– На компьютере пользователя может отсутствовать или быть устаревшая версия Java Runtime Environment. – Пользователь не сможет запустить приложение, получив ошибку о невозможности найти или загрузить главный класс.			
9	– Для управления проектом используется Kanban-доска в Miro и Git для кода. – Участники могут забывать обновлять статусы задач на доске или оставлять неинформативные сообщения в коммитах. – Потеря актуального представления о прогрессе по проекту, сложности в планировании и выявлении отставания.	7	2	14
10	– В проекте реализована сложная логика фильтрации на главном экране и в истории. – При определенном сочетании фильтров (период «Неделя» + тип «Расходы») могут возникать ошибки округления или неверно рассчитываться проценты для категорий. – Пользователь видит некорректные финансовые данные, что подрывает доверие ко всему приложению.	4	9	36

1.6.3. Матрица рисков

Распределим риски по матрице в зависимости от их вероятности и уровня ущерба.

Таблица 1.6 — Матрица рисков

Вероятность	Уровень ущерба				
	Несущественные (1-2)	Низкие (3-4)	Средние (5-6)	Существенные (7-8)	Катастрофические (9-10)
Весьма вероятно (9-10)		4			
Вероятно (7-8)	2, 9		1		
Возможно (5-6)				3, 5	8
Маловероятно (3-4)				6	7, 10

Крайне маловероятно (1-2)					
---------------------------	--	--	--	--	--

1.6.4. Планы реагирования на риски

Разработаем планы реагирования для самых значимых рисков (важность > 40 и/или попавших в красные зоны матрицы).

Таблица 1.7 — Планы реагирования

№	Риск (причина-риск-эффект)	Стратегия	Основной план	Отходной план (или Дополнительный)
1	– Члены команды имеют разный уровень навыков работы с технологией Compose для Desktop. – Некоторые компоненты UI (например, кастомная круговая диаграмма) могут потребовать больше времени на реализацию, чем ожидалось. – Задержка в разработке пользовательского интерфейса и его интеграции с ViewModel.	Снижение	Провести внутренний воркшоп по Compose для Desktop. Павлов Н.С. (UI/UX дизайнер) готовит примеры ключевых компонентов. Взять на реализацию сложных частей (PieChart) с запасом времени.	Создать упрощенный макет диаграммы с использованием стандартных компонентов, если кастомная реализация займет слишком много времени.
3	– Тестирование интеграции с базой данных SQLite может быть пропущено на ранних этапах. – SQL-запросы могут оказаться неоптимальными, или структура БД может потребовать изменений при росте объема данных. – Снижение производительности при фильтрации и отображении большого количества операций (нефункциональное требование <500 мс)	Снижение	Еще на этапе проектирования БД создать индексы для полей date в таблицах транзакций. При разработке использовать EXPLAIN QUERY PLAN для анализа сложных запросов.	Если производительность упадет, реализовать кэширование результатов фильтрации в ViewModel на уровне приложения, чтобы не обращаться к БД при каждом переключении фильтра.

	может быть нарушено).			
5	– Заказчик (преподаватель) или пользователи могут предоставить обратную связь на этапе демонстрации промежуточного результата. – Может потребоваться доработка интерфейса или добавление новой функциональности, не предусмотренной изначальным ТЗ. – Увеличение трудозатрат и сдвиг сроков сдачи проекта.	Принятие	Демонстрировать заказчику (преподавателю) работающие инкременты продукта на каждом этапе (после реализации ключевого функционала). Сразу согласовывать все изменения.	Заложить в план проекта отдельную задачу «Доработки по результатам демонстрации» с резервом в 1-2 дня.
7	– Приложение использует локальную базу данных SQLite, и все данные хранятся в одном файле. – При некорректном завершении работы приложения или сбоя в системе во время записи данных файл БД может быть поврежден. – Полная или частичная потеря всех финансовых данных пользователя, невозможность запуска приложения.	Страхование / Снижение	Реализовать механизм резервного копирования БД: при каждом успешном запуске приложения создавать копию файла Finlytics.db.bak. Использовать транзакции SQLite для атомарных операций.	Предоставить пользователю в настройках функцию «Восстановить из резервной копии». В случае неустранимого повреждения БД, предложить создать новую БД со стандартными категориями (без потери возможности работы).
8	– Приложение распространяется как исполняемый JAR-файл и требует наличия JRE. – На компьютере пользователя может отсутствовать или быть устаревшая версия Java Runtime Environment. – Пользователь не сможет запустить	Уклонение	Использовать инструменты сборки Gradle (например, jpackage или badass-runtime plugin) для создания нативных установщиков (.msi, .dmg, .deb), которые включают в себя собственную JRE.	В README.md на GitHub и в пользовательской документации явно указать требование к версии JRE (например, JRE 21) и предоставить прямую ссылку для скачивания.

	приложение, получив ошибку о невозможности найти или загрузить главный класс.			
10	– В проекте реализована сложная логика фильтрации на главном экране и в истории. – При определенном сочетании фильтров (период «Неделя» + тип «Расходы») могут возникать ошибки округления или неверно рассчитываться проценты для категорий. – Пользователь видит некорректные финансовые данные, что подрывает доверие ко всему приложению.	Снижение	Покрыть функции фильтрации модульными тестами. Проверить граничные случаи (пустые списки, високосные года).	Добавить ручной чек-лист тестирования всех фильтров перед релизом.

1.7. Анализ эксплуатации проекта

1.7.1. Анализ технологических угроз и их влияния на функционал

На основе анализа архитектуры и функциональных требований проекта «Finlytics» были выбраны три технологические угрозы, которые могут возникнуть в процессе эксплуатации приложения. Оценка влияния каждой угрозы на ключевые функциональные требования выполнена по 5-балльной шкале.

Выбранные технологические угрозы:

1. Угроза А: Повреждение или сбой в работе локального файла базы данных SQLite (например, из-за некорректного завершения работы приложения, сбоя ОС или повреждения сектора на диске).

2. Угроза Б: Отсутствие или несовместимость версии Java Runtime Environment (JRE) на компьютере пользователя.

3. Угроза В: Снижение производительности системы при накоплении большого объема данных (например, при нескольких тысячах операций).

Таблица 1.8 — Оценка влияния технологических угроз на функциональные требования

№ требования	Функциональное требование	Угроза А	Угроза Б	Угроза В
1.1.1	Добавление новой финансовой операции	5	5	2
1.1.2	Редактирование существующей операции	5	5	2
1.1.3	Удаление операции	5	5	2
1.2.1	Автоматическое создание предустановленных категорий при первом запуске	5	5	0
1.2.2	Создание новых категорий доходов и расходов	5	5	1
1.2.3	Блокировка удаления категории, используемой в операциях	5	5	1
1.3.1	Отображение финансовой сводки (баланс, доходы, расходы) на главном экране	5	5	2
1.3.2	Отображение круговой диаграммы распределения по категориям	5	5	2
1.3.3	Фильтрация истории операций по временному промежутку и типу	5	5	2

1.7.2. Преобразование технологических угроз в риски

На основе выбранных угроз, уязвимостей и активов проекта «Finlytics» сформулируем риски в формате «Угроза + Уязвимость + Актив = Риск».

Таблица 1.9 — Преобразование технологических угроз в риски эксплуатации

№	Угроза	Уязвимость	Актив	Риск	P _i
РА	Повреждение или потеря данных.	Данные хранятся в одном локальном файле Finlytics.db, нет встроенной репликации.	База данных SQLite со всеми финансовыми операциями и категориями пользователя.	Полная или частичная потеря всех финансовых данных пользователя из-за повреждения единственного файла БД.	3
РБ	Невозможность запустить исполняемый файл.	Приложение распространяется как JAR-файл и требует наличия JRE на целевой системе.	Исполняемый JAR-файл приложения Finlytics.jar.	Невозможность запуска приложения пользователем из-за отсутствия или несовместимости Java Runtime Environment.	5
РВ	Замедление или "зависание" при выполнении запросов.	В проекте используется прямое выполнение SQL-	Процессор и оперативная память компьютера	Снижение производительности и нарушение времени отклика (<500 мс) при	6

		запросов без дополнительного кэширования на уровне приложения.	пользователя, SQLite БД.	фильтрации и отображении большого количества операций.	
--	--	--	--------------------------	--	--

1.7.3. Методы обработки рисков эксплуатации

Для каждого выявленного риска предложим метод обработки и обоснуем его с помощью подхода «Треугольник управления проектом» (Сроки, Бюджет, Содержание работ).

Таблица 1.10 — Методы обработки рисков эксплуатации

№ Риска	Риск (формулировка)	Метод обработки	Обоснование метода (через изменения)
РА	Полная или частичная потеря всех финансовых данных пользователя из-за повреждения единственного файла БД.	Снижение риска	Технические изменения: Реализовать механизм автоматического резервного копирования файла БД (Finlytics.db.bak) при каждом успешном запуске приложения. Также использовать транзакции SQLite для атомарных операций записи. Влияние на треугольник: Потребуется дополнительное время на разработку и тестирование (Сроки и Бюджет), но не меняет содержание работ.
РБ	Невозможность запуска приложения пользователем из-за отсутствия или несовместимости JRE.	Уклонение от риска	Архитектурные и технические изменения: Использовать инструменты сборки Gradle (например, jpackage или плагин badass-runtime) для создания нативных установщиков (.msi, .dmg, .deb), которые включают в себя собственную «связанную» JRE. Влияние на треугольник: Увеличение времени на настройку сборки (Сроки), но полностью убирает зависимость от окружения пользователя. Содержание работ остается прежним.
РВ	Снижение производительности при фильтрации большого количества операций.	Снижение риска	Технические изменения: 1. Создать индексы в БД SQLite для полей income_transaction_date и expenses_transaction_date. 2. Реализовать простое кэширование результатов фильтрации в ViewModel на уровне приложения для часто повторяющихся запросов. Влияние на треугольник: Потребуется дополнительное время на

			разработку и оптимизацию (Сроки), но сохраняет полный функционал и улучшает пользовательский опыт.
--	--	--	--

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Поведенческие диаграммы

2.1.1. Диаграмма вариантов использования

2.1.1.1. Обоснование сценариев использования

- Пользователь добавляет новую операцию (доход/расход).
- Пользователь редактирует существующую операцию.
- Пользователь удаляет операцию.
- Пользователь просматривает историю операций.
- Пользователь фильтрует операции по дате и категории.
- Пользователь просматривает финансовую сводку.
- Пользователь управляет категориями.

2.1.1.2. Схематическое представление (Use Case Diagram)

Диаграмма включает одного основного актора – Пользователь. Сценарии использования: "Добавить операцию", "Редактировать операцию", "Удалить операцию", "Просмотреть историю операций", "Отфильтровать операции", "Просмотреть финансовую сводку", "Добавить категорию".

Результат создания диаграммы представлен на Рисунок 2.1.

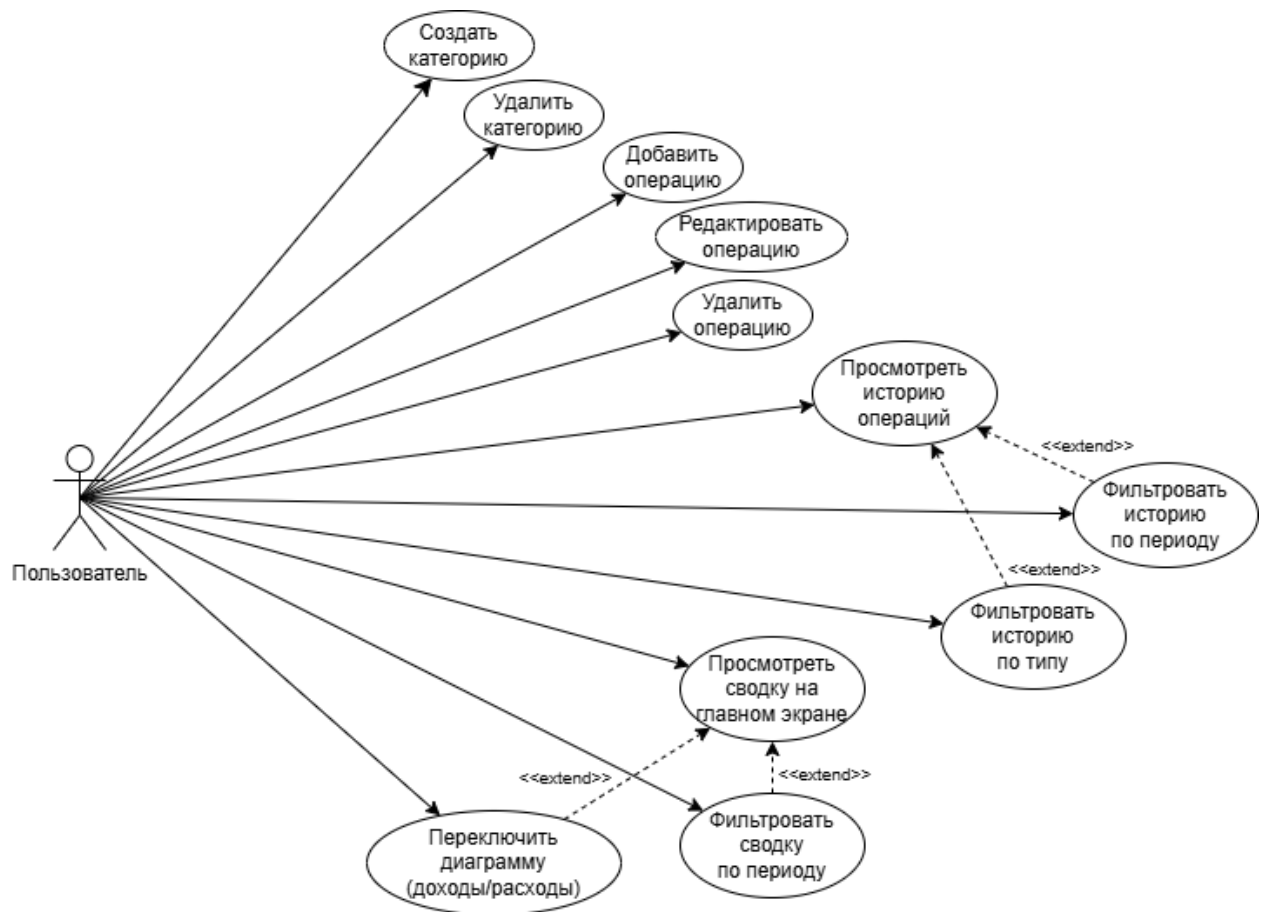


Рисунок 2.1 — Диаграмма прецедентов разрабатываемого проекта

2.1.2. Диаграмма последовательности

Диаграмма включает 5 объектов – Пользователь, Экран приложения (UI), Менеджер состояния (ViewModel), Хранилище данных (Repository), База данных SQLite.

Результат создания диаграммы представлен на Рисунок 2.2.

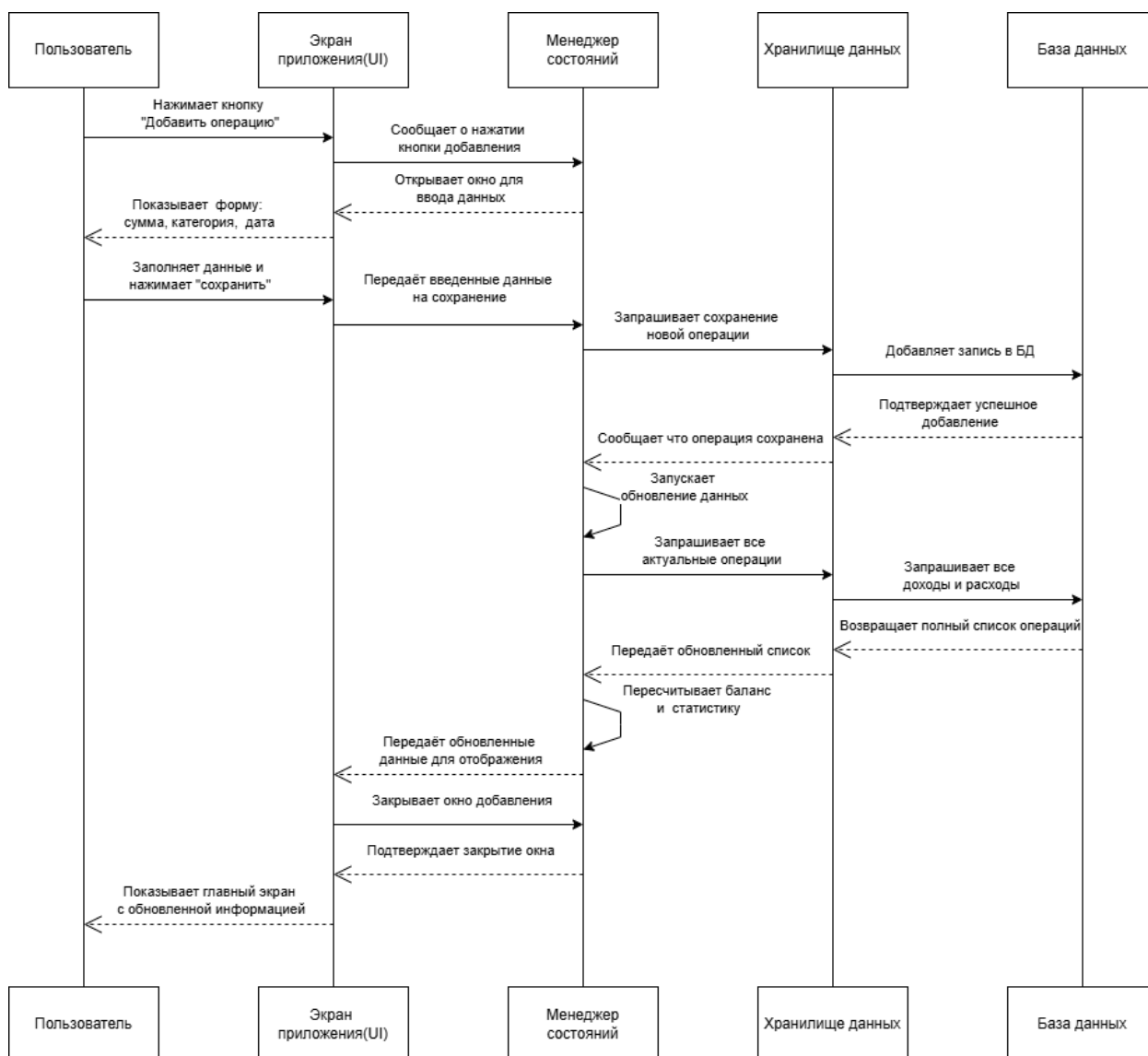


Рисунок 2.2 — Диаграмма последовательности разрабатываемого проекта

2.2. Процесс работы в системе

Для детализации поведения системы и взаимодействия пользователя с ней на уровне конкретных шагов была разработана диаграмма бизнес-процесса (BPMN). В качестве примера был выбран ключевой сценарий использования — «Добавление новой финансовой операции», который наглядно демонстрирует путь пользователя и логику обработки данных в приложении. (Рисунок 2.3)

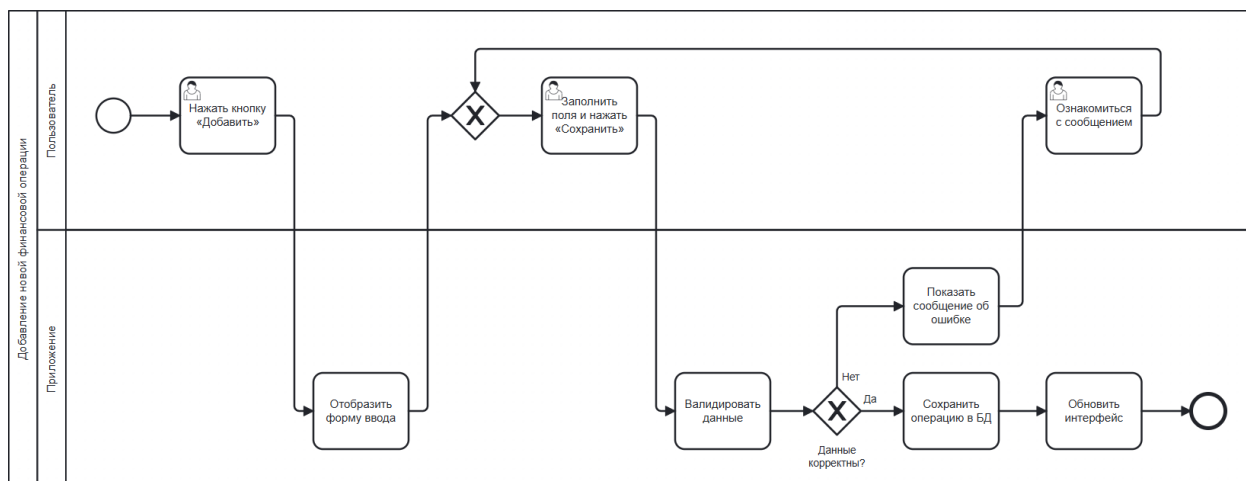


Рисунок 2.3 — Бизнес-процесс "Добавление новой финансовой операции"

2.3. Структурные диаграммы

2.3.1. Диаграмма классов

Диаграмма классов, представленная на Рисунок 2.4, отображает основные сущности приложения и их взаимоотношения, являясь основой для будущей реализации.

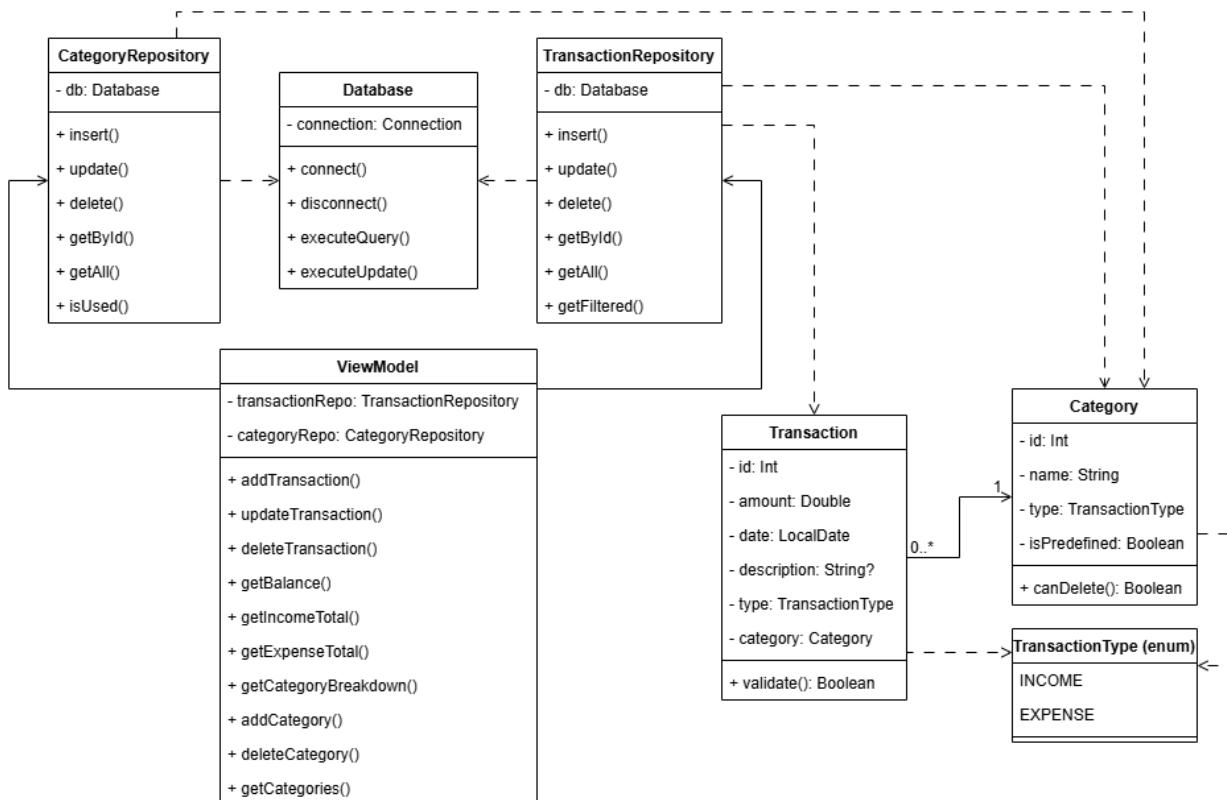


Рисунок 2.4 — Диаграмма классов

2.3.2. Диаграмма объектов

Для иллюстрации работы системы в конкретный момент времени была построена диаграмма объектов (Рисунок 2.5). Она показывает экземпляры классов, созданные в процессе выполнения пользовательского сценария, и значения их атрибутов.

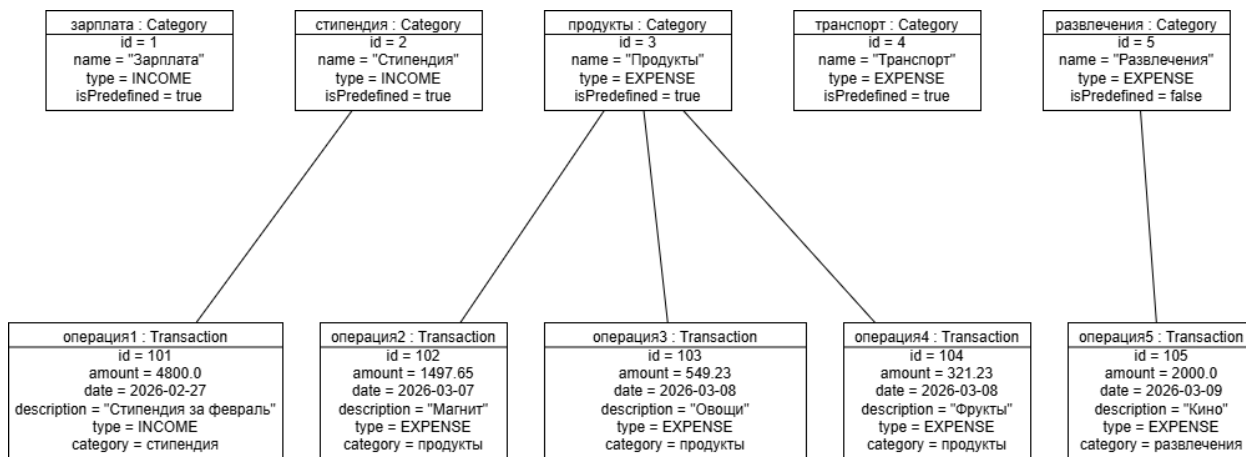


Рисунок 2.5 — Диаграмма объектов

2.4. Информационное взаимодействие компонентов системы

2.4.1. Текстовое описание

Взаимодействие начинается с того, что Пользователь системы инициирует работу, отправляя два основных типа запросов: Команды управления (на изменение данных) и Запросы на просмотр (на получение информации). Эти запросы могут сопровождаться Параметрами фильтрации для уточнения выборки. Все они поступают в ядро системы — Систему управления личными финансами, которая обрабатывает их и, при необходимости, транслирует в Запросы к БД для взаимодействия с базой данных.

В ответ на пользовательские действия система формирует Результаты запросов, которые затем преобразуются в Отображение данных для пользователя. Если в процессе обработки возникают ошибки (например, при некорректном Запросе на управление операциями или категориями), система генерирует Сообщения об ошибках, которые также выводятся пользователю.

Таким образом, замкнутый цикл "запрос — обработка — ответ" обеспечивает актуальность информации на экране.

Более детально взаимодействие раскрывается в специализированных сценариях. Так, Запрос на аналитику инициирует цепочку вычислений: сначала происходит Расчет финансовой сводки, на основе которого подготавливаются Данные для визуализации, и только потом выполняется Построение диаграмм. Результатом этого процесса является готовая Финансовая сводка и визуальные элементы, отображаемые пользователю.

Управление данными также структурировано по модулям. При выполнении Запроса на управление операциями система позволяет выполнять Добавление, Редактирование или Удаление операции, манипулируя соответствующими Данными операций. Аналогично, Запрос на управление категориями дает возможность Добавлять, Удалять или Просматривать категории, работая с Данными категорий. Ключевым результатом фильтрации и просмотра является формирование Отфильтрованного списка операций, который служит основой как для аналитики, так и для непосредственного ознакомления пользователя с историей транзакций.

2.4.2. Диаграмма DFD

На основании текстового описания информационного взаимодействия компонентов системы была построена диаграмма в нотации DFD. На рисунках ниже представлен верхний уровень диаграммы, декомпозиция верхнего уровня и функций внутри нее («Управление интерфейсом», «Управление операциями», «Управление категориями», «Аналитика и визуализация»):

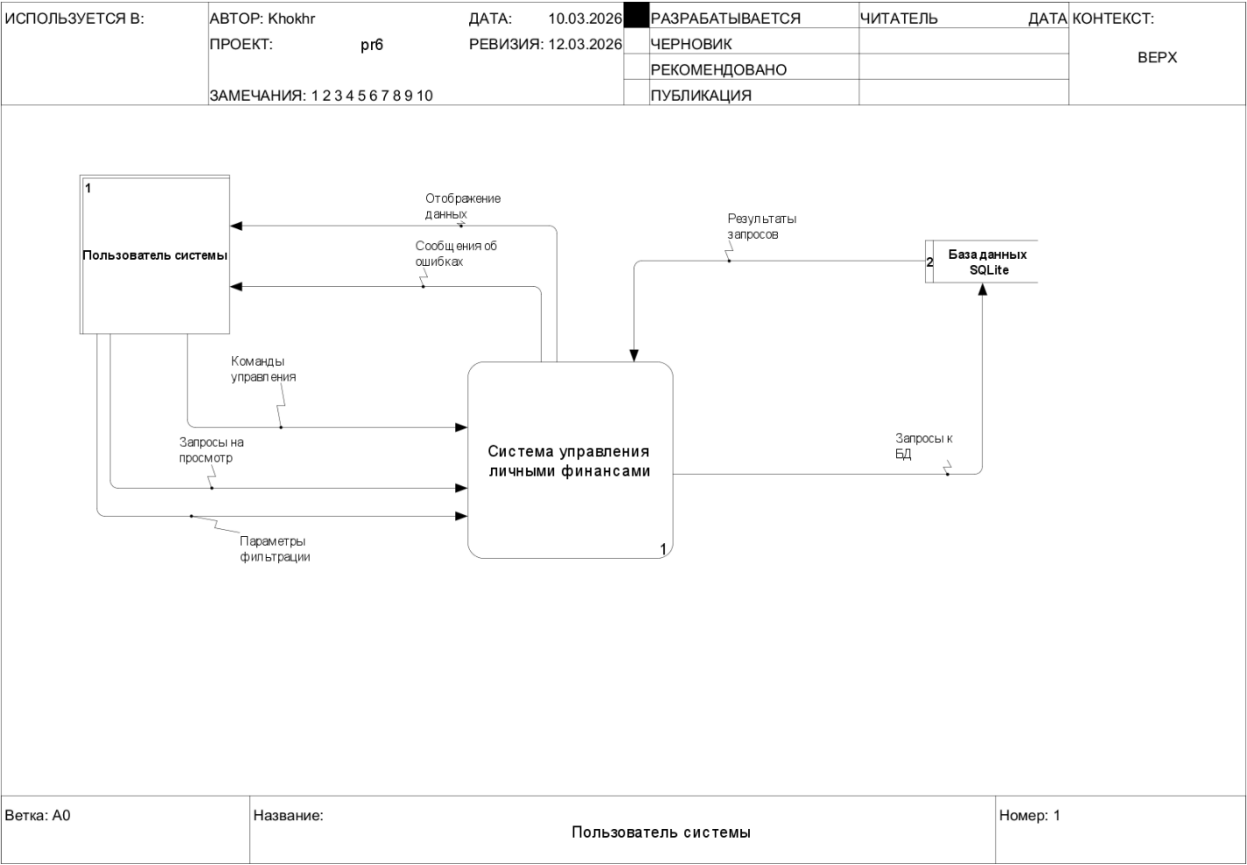


Рисунок 2.6 — Диаграмма DFD. Верхний уровень

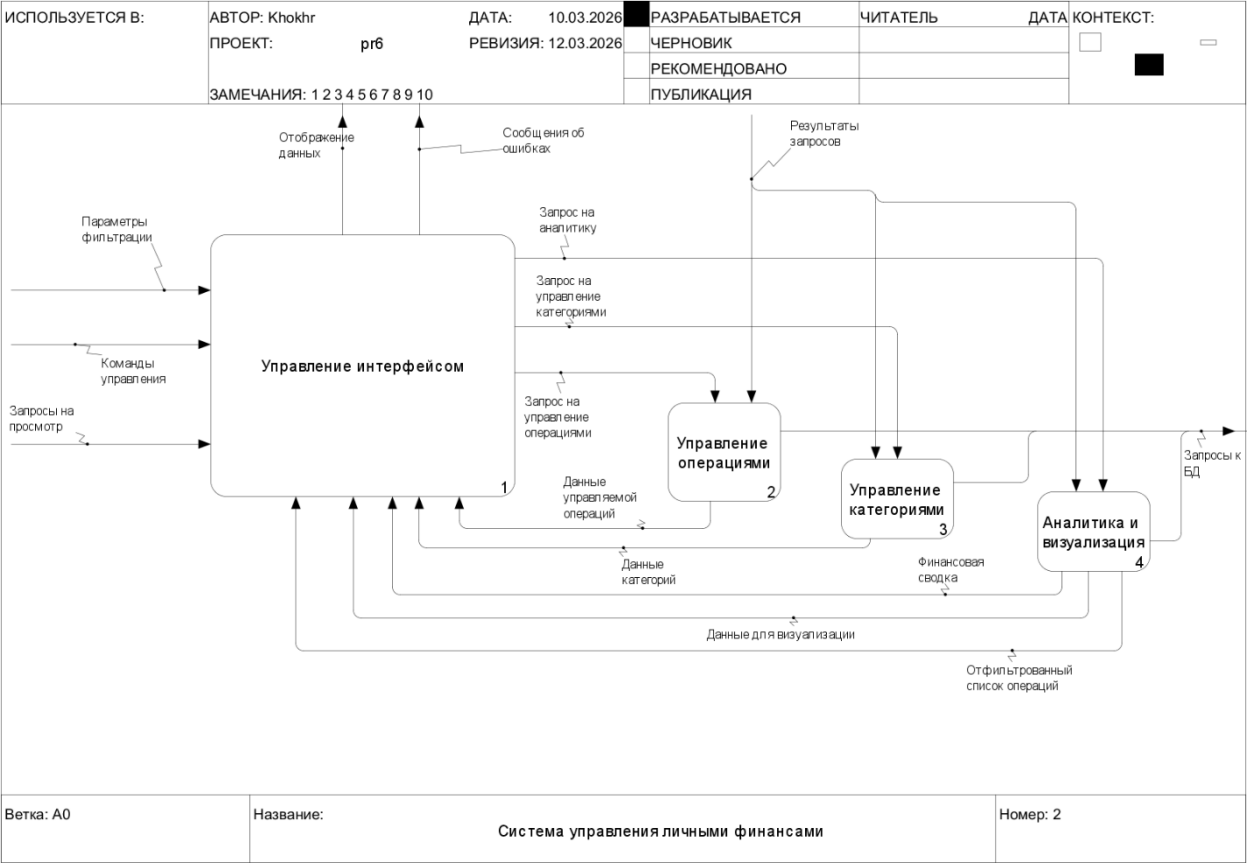


Рисунок 2.7 — Диаграмма DFD. Декомпозиция верхнего уровня

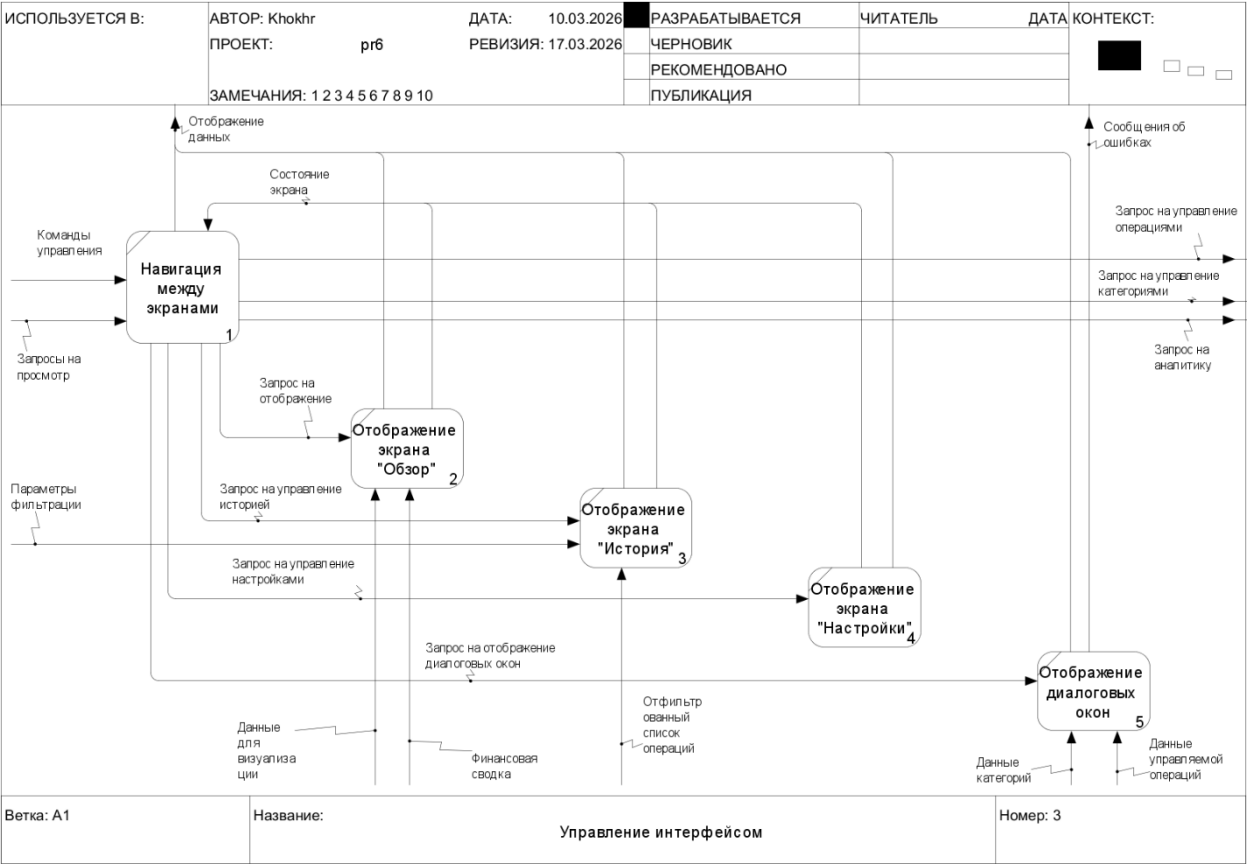


Рисунок 2.8 — Диаграмма DFD. Декомпозиция "Управление интерфейсом"



Рисунок 2.9 — Диаграмма DFD. Декомпозиция "Управление операциями"

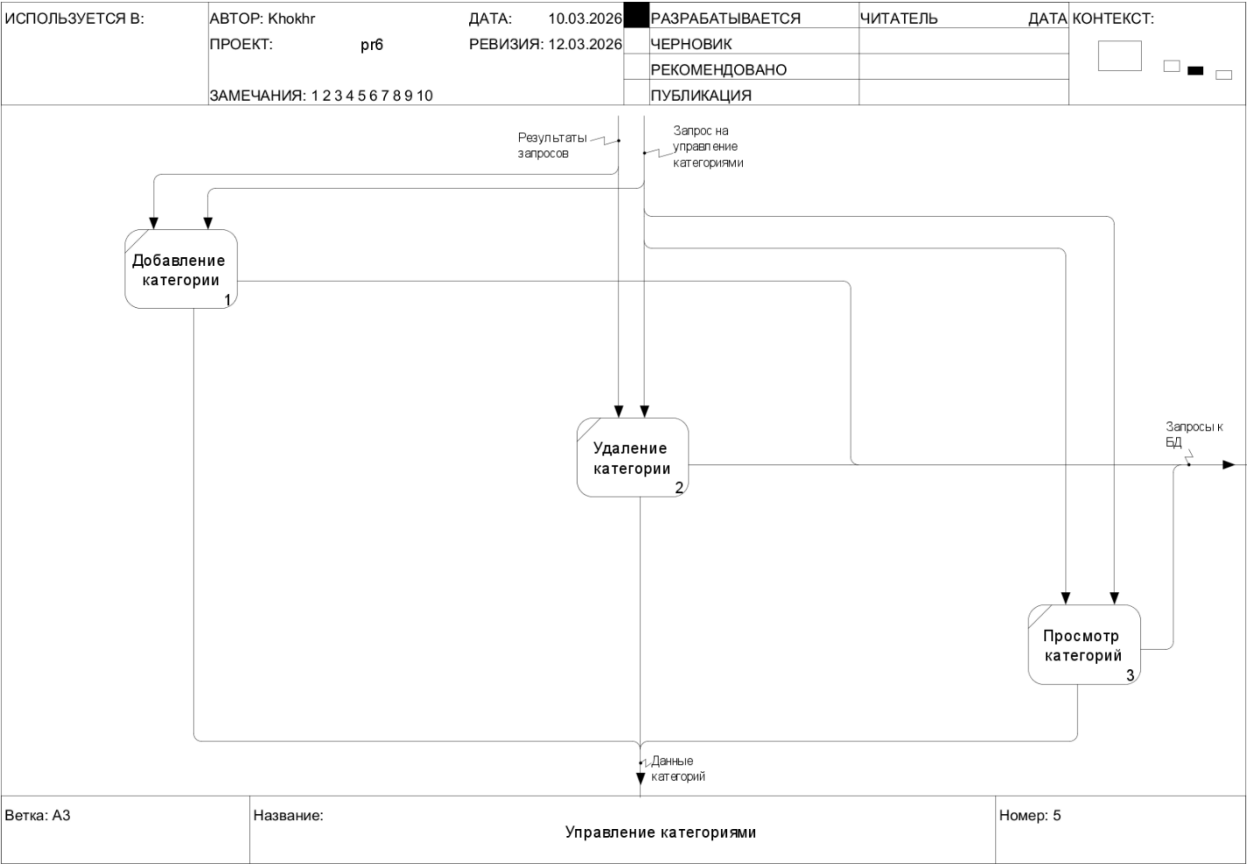


Рисунок 2.10 — Диаграмма DFD. Декомпозиция "Управление категориями"

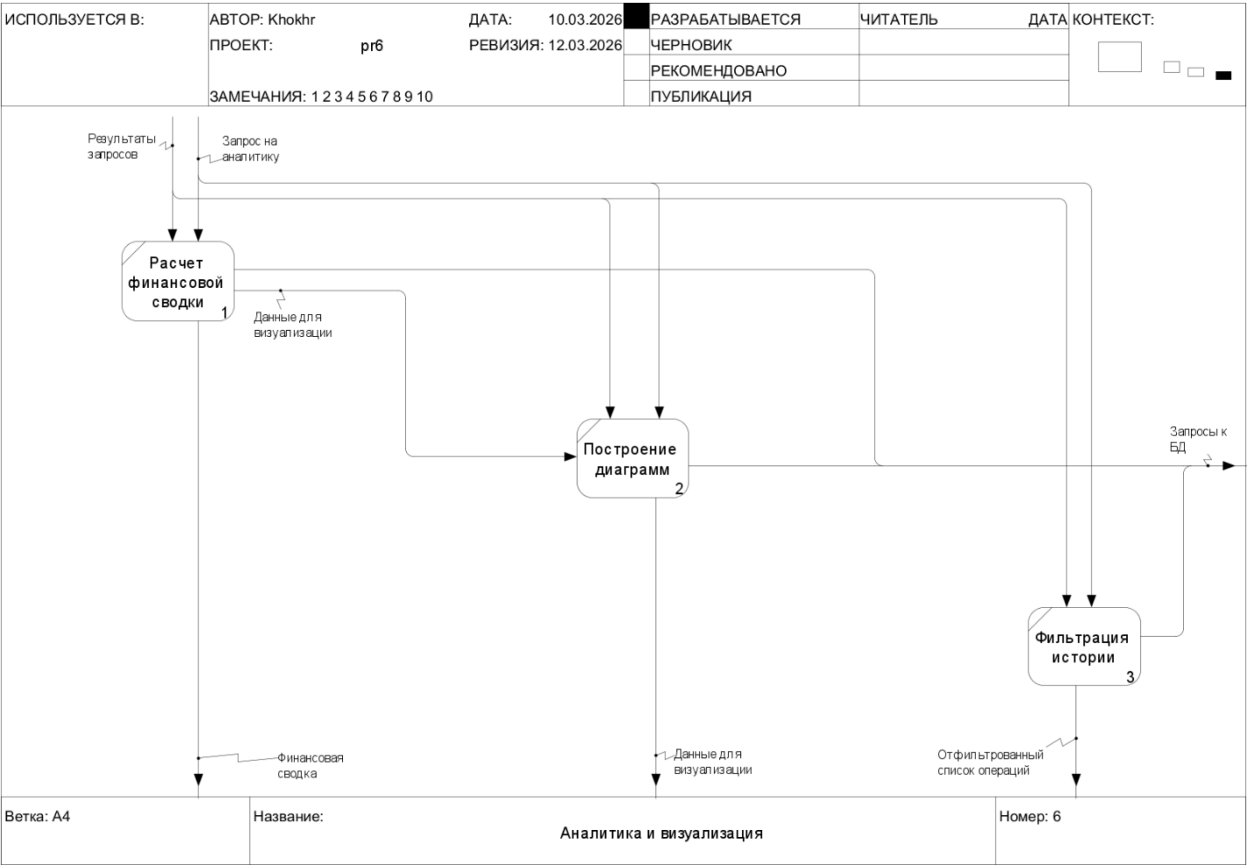


Рисунок 2.11 — Диаграмма DFD. Декомпозиция "Аналитика и визуализация"

2.5. Модель Базы данных

В качестве системы управления базами данных для офлайн-хранения информации выбрана SQLite — легковесная, встраиваемая реляционная база данных, идеально подходящая для десктопных приложений, работающих с локальными данными.

Логическая модель данных (Рисунок 2.12) состоит из четырех основных таблиц, связанных отношением "один ко многим":

1. Income_categories (Категории доходов):
 - id_income_category (INTEGER, PRIMARY KEY) — уникальный идентификатор категории.
 - income_category_name (TEXT, NOT NULL, UNIQUE) — название категории (например, "Зарплата", "Стипендия").
2. expenses_categories (Категории расходов):
 - id_expenses_category (INTEGER, PRIMARY KEY) — уникальный идентификатор категории.
 - expenses_category_name (TEXT, NOT NULL, UNIQUE) — название категории (например, "Еда", "Транспорт").
3. Income_transactions (Транзакции доходов):
 - id_income_transaction (INTEGER, PRIMARY KEY) — уникальный идентификатор транзакции.
 - income_transaction_name (TEXT) — описание транзакции (может быть NULL).
 - income_transaction_sum (REAL, NOT NULL) — сумма дохода.
 - id_income_category (INTEGER, NOT NULL, FOREIGN KEY) — ссылка на идентификатор категории дохода из таблицы Income_categories. Обеспечивает целостность данных на уровне БД.
 - income_transaction_date (DATE, NOT NULL) — дата операции в формате ГГГГ-ММ-ДД.

4. expenses_transactions (Транзакции расходов):
- id_expenses_transaction (INTEGER, PRIMARY KEY) — уникальный идентификатор транзакции.
 - expenses_transaction_name (TEXT) — описание транзакции (может быть NULL).
 - expenses_transaction_sum (REAL, NOT NULL) — сумма расхода.
 - id_expenses_category (INTEGER, NOT NULL, FOREIGN KEY) — ссылка на идентификатор категории расхода из таблицы expenses_categories.
 - expenses_transaction_date (DATE, NOT NULL) — дата операции.



Рисунок 2.12 — Логическая модель Базы данных

2.6. Архитектура приложения

Для проекта "Finlytics" выбрана чистая многослойная архитектура (Layered Architecture), реализованная в рамках паттерна MVVM (Model-View-

ViewModel) на клиентской стороне. Это классический и наиболее подходящий выбор для десктопного офлайн-приложения.

2.6.1. Обоснование выбора

1. Соответствие типу приложения: Разрабатываемое приложение — десктопное, офлайн, однопользовательское. Ему не требуется сложная распределенная архитектура (как SOA или микросервисы) или разделение на физические уровни (как трехуровневая архитектура). Многослойная архитектура идеально подходит для таких монолитных приложений, обеспечивая логическое разделение ответственности без излишнего усложнения инфраструктуры.

2. Принцип разделения ответственности (SoC): Архитектура четко разделяет приложение на слои: UI (View) отвечает только за отображение, ViewModel — за состояние и логику представления, а Model (Репозиторий и источник данных) — за бизнес-логику и данные. Это делает код понятным, тестируемым и легким в сопровождении.

3. Гибкость и расширяемость: Паттерн MVVM и многослойность обеспечивают гибкость. Мы можем изменить способ хранения данных, модифицируя только слой Repository и источники данных, не трогая при этом ViewModel и UI. Аналогично, изменение интерфейса не влияет на бизнес-логику.

4. Поддержка реактивности: Kotlin Flows, используемые в ViewModel, идеально вписываются в эту архитектуру, позволяя UI автоматически реагировать на изменения в состоянии (State).

5. Простота для команды: Архитектура понятна всем разработчикам, использующим Kotlin, и не требует изучения сложных распределенных концепций.

Таблица 2.1 – Программные решения и обоснования их выбора

Компонент архитектуры	Выбранное решение	Обоснование
-----------------------	-------------------	-------------

Язык реализации	Kotlin	Современный, лаконичный и безопасный язык для платформы JVM. Идеально подходит для разработки десктопных приложений с Compose для Desktop. Обеспечивает отличную совместимость с существующими Java-библиотеками.
UI-фреймворк (Слой View)	Jetpack Compose for Desktop	Современный декларативный фреймворк для создания нативных UI. Позволяет писать меньше кода, он более интуитивен и легче поддерживается по сравнению с устаревшими инструментами типа Swing или JavaFX. Идеально интегрируется с реактивной моделью MVVM через Kotlin Flow.
Управление состоянием	Kotlin Coroutines & Flow (ViewModel)	Flow обеспечивает реактивный поток данных. ViewModel подписывается на изменения в Repository (которые также могут быть представлены как Flow) и предоставляет StateFlow для UI. Coroutines используются для асинхронных операций с БД, не блокируя главный поток UI.
Слой данных (Repository)	Собственный класс FinanceRepository	Выступает единой точкой входа для всех данных. Абстрагирует источник данных (БД) от вышележащих слоев.
Слой источника данных	SQLite	Легковесная, встраиваемая реляционная база данных, идеально подходящая для локального хранения структурированных данных на одном устройстве. Не требует отдельного сервера и настройки, что критически важно для офлайн-приложения.
Работа с БД	Прямое использование JDBC (встроенные средства Kotlin/Java)	Для такого небольшого проекта использование ORM было бы избыточным. Прямые SQL-запросы с использованием PreparedStatement дают полный контроль над производительностью и безопасностью (защита от SQL-инъекций).
Сборка и зависимости	Gradle (Kotlin DSL)	Стандарт де-факто для сборки Kotlin-проектов. Kotlin DSL обеспечивает типовую безопасность и лучшую поддержку IDE по сравнению с Groovy DSL.
Логирование	Стандартный println (с настройкой UTF-8 в LoggingConfig)	Для отладки и мониторинга состояния приложения на этапе разработки. Простота использования достаточна для нужд проекта.
Визуализация (диаграммы)	Кастомный компонент на Canvas (PieChart.kt)	Compose для Desktop предоставляет мощный Canvas API, который позволяет полностью контролировать отрисовку. Создание собственной диаграммы дает гибкость в дизайне и избавляет от необходимости подключать тяжелые сторонние библиотеки.
Векторная графика	Собственный набор ImageVector (FinlyticsIconPack)	Использование встроенной в Compose возможности работы с векторной графикой. Иконки хранятся в виде кода (Kotlin-файлов), что делает их неотъемлемой частью

		приложения, упрощает версионирование и не требует внешних ресурсов.
Контроль версий	Git (платформа GitHub)	Обеспечивает совместную работу команды, отслеживание истории изменений и является стандартом в индустрии.

2.6.2. Диаграмма архитектуры

На Рисунок 2.13 представлена диаграмма, отражающая спроектированную и реализованную архитектуру приложения "Finlytics". Диаграмма иллюстрирует многослойную структуру с паттерном MVVM.

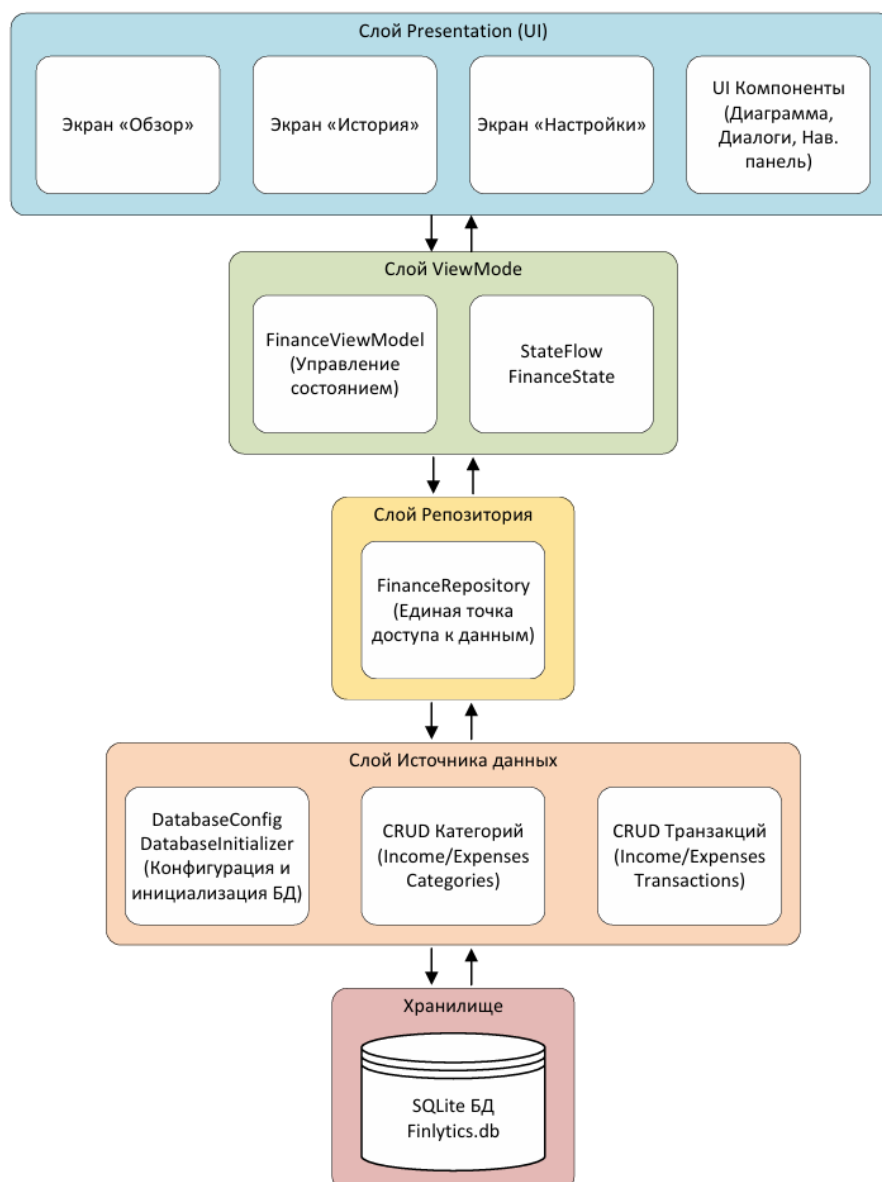


Рисунок 2.13 — Диаграмма архитектуры приложения «Finlytics»

2.7. Проектирование интерфейса

2.7.1. Наполнение экранов

Проектирование пользовательского интерфейса приложения "Finlytics" было выполнено в Figma с фокусом на создание интуитивно понятного, эстетичного и функционального пространства для управления личными финансами.

Были разработаны и визуализированы следующие ключевые экраны:

- **Главный экран ("Обзор"):** Представляет собой сводку финансового состояния. На нем отображается круговая диаграмма распределения и статистика расходов/доходов по категориям, ключевые показатели (общие доходы, расходы и баланс) за выбранный период, а также инструменты для фильтрации данных по времени.
- **Экран "История":** Содержит полный список всех финансовых операций в виде списка. Предусмотрена возможность фильтрации по временному промежутку и типу операции.
- **Экран "Настройки":** Позволяет пользователю управлять системой категорий — просматривать, добавлять, редактировать и удалять категории как для доходов, так и для расходов.

Для интерактивного управления данными были спроектированы всплывающие окна (pop-up):

- Добавление/редактирование финансовой операции.
- Добавление/редактирование категории.
- Подтверждение удаления операции или категории.

2.7.2. Обоснование дизайн-решений

1. Выбор темной темы (Основные цвета: #1E1E1E, #2C2C2C, #444444, #F4F4F4):

Темная тема соответствует современным тенденциям в дизайне программного обеспечения и снижает нагрузку на глаза, особенно при длительной работе с приложением, что актуально для десктопного ПО.

Глубокий фон (#1E1E1E) позволяет контенту и данным (таким как диаграммы и цифры) выступать на первый план. Цвета #2C2C2C и #444444 используются для разделения областей (карточек, панелей) и создания глубины интерфейса, не перегружая его.

Высококонтрастный текст (#F4F4F4) обеспечивает отличную читаемость на темном фоне.

2. Акцентный цвет (#2176FF):

Этот насыщенный синий цвет был выбран за его позитивные ассоциации с надежностью, спокойствием и профессионализмом, что важно для финансового приложения. Он используется для интерактивных элементов: кнопок ("Добавить операцию", "Сохранить"), активных полей ввода и выделения выбранного периода, что обеспечивает интуитивно понятную навигацию.

3. Цвета статусов (#4ADE80 для доходов, #EF4444 для расходов):

Зеленый (#4ADE80): Универсально ассоциируется с позитивом, успехом и ростом. Он интуитивно понятен для отображения доходов и положительного баланса.

Красный (#EF4444): Традиционный цвет для предупреждений и отрицательных значений. Он мгновенно сообщает пользователю о расходах и отрицательной динамике.

4. Вспомогательные цвета (для диаграмм и статистики):

Для круговой диаграммы и других элементов визуализации используется палитра контрастных, но гармонирующих с общей темой цветов. Это обеспечивает четкое различие между категориями расходов, делая диаграмму не только информативной, но и визуально привлекательной.

2.7.3. Макеты экранов и прототип

Результат создания макетов экранов и рор-уп представлен на рисунках ниже. Для демонстрации интерактивности и пользовательского потока был создан [кликабельный прототип в Figma](#).

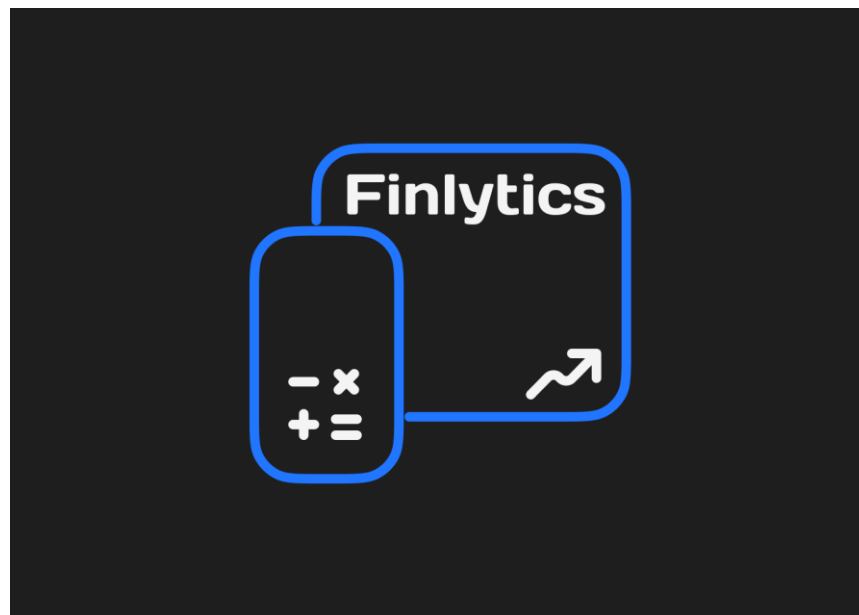


Рисунок 2.14 — Экран загрузки

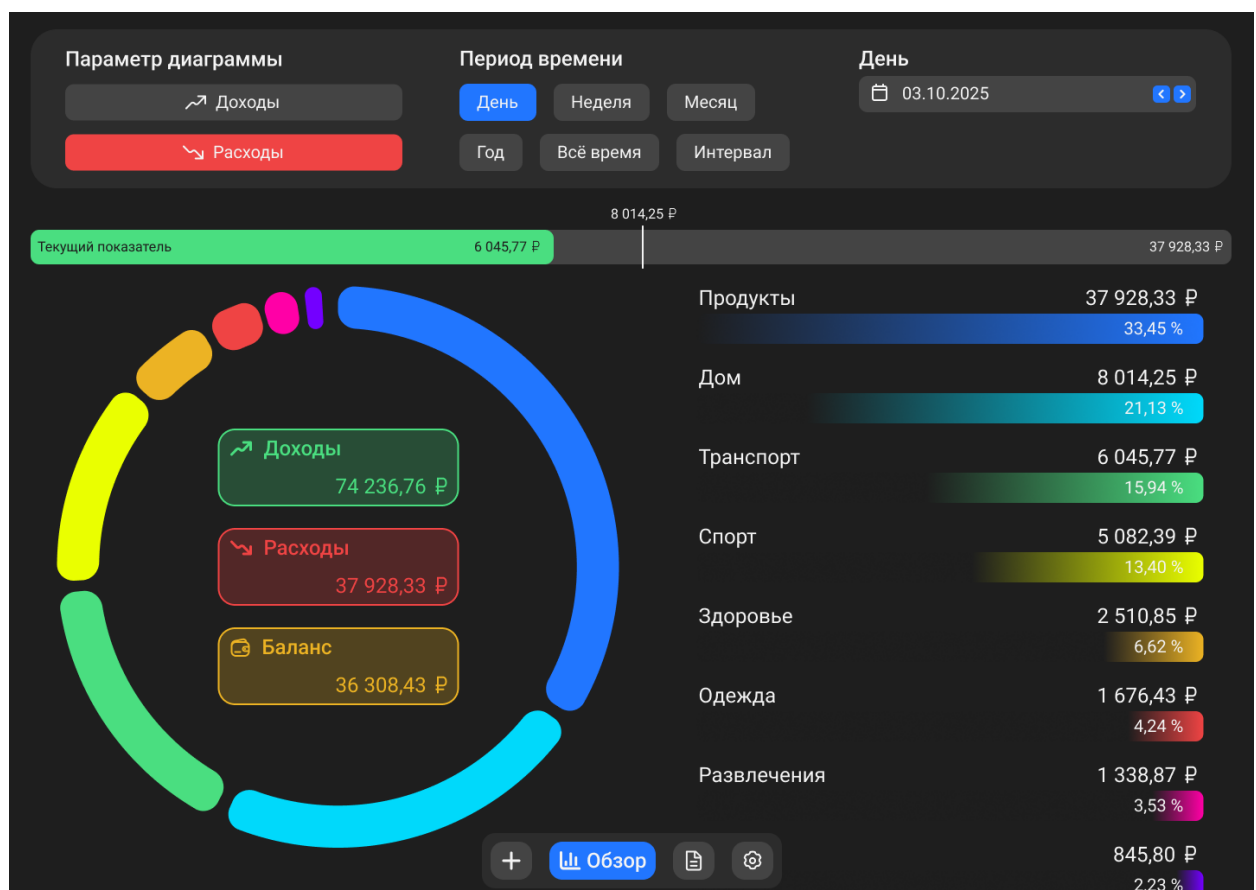


Рисунок 2.15 — Экран домашней страницы

3. РАЗРАБОТКА ПРИЛОЖЕНИЯ

3.1. Методология управления процессом разработки

Для проекта «Finlytics» была выбрана итерационная модель разработки, реализованная через гибкую методологию Kanban.

Обоснование выбора:

1. Соответствие типу проекта. Проект представляет собой десктопное офлайн-приложение с четко определенным, но нефинальным на момент старта набором функций. Итерационная модель позволяет разбить разработку на последовательные этапы (итерации), каждая из которых завершается получением работающего инкремента продукта. Это соответствует принципу MVP (Minimum Viable Product) и позволяет на ранних этапах продемонстрировать заказчику (преподавателю) работающий функционал для сбора обратной связи и корректировки требований.

2. Управление неопределенностью. Итерационный подход снижает риски, связанные с возможными ошибками в проектировании или неверно понятыми требованиями. Короткие итерации позволяют выявлять проблемы на ранних стадиях и оперативно их исправлять, не переписывая всю систему в конце разработки (что характерно для каскадной модели Waterfall).

3. Выбор Kanban в качестве реализации итерационного подхода. В рамках итерационной модели командой была выбрана методология Kanban. Этот выбор был обусловлен следующими факторами:

- Визуализация потока задач. В отчете представлена Kanban-доска, созданная в сервисе Miro (Рисунок 1.1). Доска четко разделена на колонки, отражающие этапы разработки. Это позволяет всем членам команды в любой момент видеть текущий статус каждой задачи и узкие места процесса.
- Гибкость и простота. Kanban не требует жестких временных рамок (как спринты в Scrum) и позволяет добавлять новые задачи по мере

необходимости. Это идеально подходит для студенческого проекта с фиксированным, но плавающим графиком, где приоритеты могут меняться в зависимости от успеваемости или появления новых идей.

- Акцент на завершенность. В Kanban задача считается выполненной только после прохождения всех этапов (разработка, тестирование, документирование), что гарантирует качество каждого реализованного функционального блока.

3.2. Инструменты разработки ПО

Для реализации проекта команда использовала современные инструменты, соответствующие выбранному стеку технологий (Kotlin/JVM, Compose for Desktop). Используемые инструменты приведены в Таблица 3.1.

Таблица 3.1 — Используемые инструменты разработки ПО

Инструмент	Назначение	Обоснование выбора
IntelliJ IDEA (Ultimate Edition)	Среда разработки (IDE)	Является флагманской IDE для разработки на Kotlin и Java. Обеспечивает глубокую интеграцию с языком, мощные инструменты рефакторинга, отладки и работы с системами сборки (Gradle).
Gradle (Kotlin DSL)	Система сборки проекта	Стандарт для Kotlin-проектов. Использование Kotlin DSL (build.gradle.kts) обеспечивает типовую безопасность и автодополнение кода в IDE, что упрощает настройку зависимостей и процесса сборки.
SQLite	Система управления базами данных (СУБД)	Легковесная, встраиваемая реляционная база данных, идеально подходящая для офлайн-приложений. Не требует отдельного сервера и настройки, обеспечивает высокую производительность и полную локальность хранения данных пользователя.
Git	Система контроля версий	Основа для командной разработки. Позволяет отслеживать изменения, работать над проектом параллельно, управлять версиями и восстанавливать предыдущие состояния кода.
GitHub	Хостинг репозитория	Обеспечивает централизованное хранение кода, удобный интерфейс для управления репозиторием, отслеживания задач и обсуждения изменений через Pull Requests.

Figma	Инструмент для UI/UX дизайна и прототипирования	Позволил на раннем этапе создать детальные макеты всех экранов и всплывающих окон, а также кликабельный прототип для согласования внешнего вида и логики работы интерфейса с командой и преподавателем до начала кодирования.
Miro	Инструмент для визуального планирования и управления задачами	Использован для создания Kanban-доски. Позволяет наглядно распределять задачи, отслеживать прогресс и проводить ретроспективы, что соответствует выбранной методологии Kanban.

3.3. Создание удаленного git репозитория

Для обеспечения эффективной совместной работы и контроля версий в процессе разработки проекта "Finlytics" был создан публичный репозиторий на платформе GitHub. Всем членам команды (Враженко Д.О., Павлов Н.С., Хохряков А.Ю.) предоставлены права на чтение и запись в репозиторий для обеспечения беспрепятственной совместной работы.

Результат создания репозитория разрабатываемого проекта представлен на Рисунок 3.1.

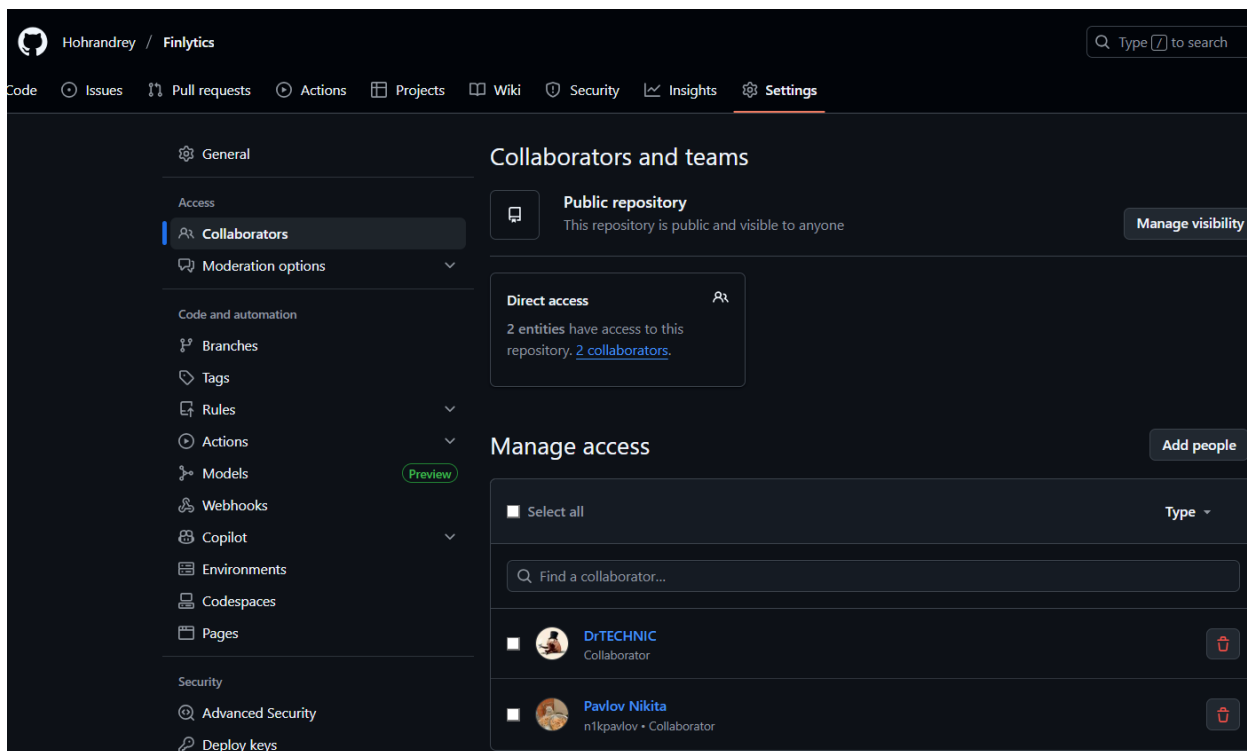


Рисунок 3.1 — Результат создания репозитория проекта

Ссылка на репозиторий: <https://github.com/Hohrandrey/Finlytics>

3.4. Настройка системы сборки Gradle

Листинг 1 – build.gradle.kts – Настройка системы сборки

```
import org.gradle.api.tasks.testing.logging.TestExceptionFormat
import org.gradle.api.tasks.testing.logging.TestLogEvent
import org.jetbrains.compose.desktop.application.dsl.TargetFormat
import org.jetbrains.kotlin.gradle.tasks.KotlinCompile

plugins {
    kotlin("jvm") version "1.9.24"
    id("org.jetbrains.compose") version "1.6.11"
    id("org.jetbrains.dokka") version "1.9.20"
}

group = "com.finlytics"
version = "1.0"

repositories {
    google()
    mavenCentral()
    maven("https://maven.pkg.jetbrains.space/public/p/compose/dev")
}

kotlin {
    jvmToolchain(21)
}

tasks.withType<KotlinCompile>().configureEach {
    kotlinOptions.jvmTarget = "21"
    kotlinOptions.freeCompilerArgs += listOf("-opt-in=kotlin.RequiresOptIn")
}

tasks.withType<JavaCompile> {
    options.encoding = "UTF-8"
}

tasks.withType<Test> {
    systemProperty("file.encoding", "UTF-8")
}

tasks.withType<JavaExec> {
    systemProperty("file.encoding", "UTF-8")
}

dependencies {
    implementation(compose.desktop.currentOs)
    implementation("org.xerial:sqlite-jdbc:3.46.1.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-swing:1.8.1")

    testImplementation(kotlin("test"))
    testImplementation("org.junit.jupiter:junit-jupiter:5.9.2")
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.8.1")
}

tasks.test {
    useJUnitPlatform()
    testLogging {
        events(TestLogEvent.PASSED, TestLogEvent.SKIPPED,
            TestLogEvent.FAILED)
        exceptionFormat = TestExceptionFormat.FULL
        showExceptions = true
    }
}
```

```

        showCauses = true
        showStackTraces = true
    }
    outputs.upToDateWhen { false }
}

tasks.register<Jar>("fatJar") {
    archiveClassifier.set("all")
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE

    manifest {
        attributes["Main-Class"] = "main.MainKt"
    }

    from({
        configurations.runtimeClasspath.get().map {
            if (it.isDirectory) it else zipTree(it)
        }
    })

    with(tasks.jar.get())
}

compose.desktop {
    application {
        mainClass = "main.MainKt"

        nativeDistributions {
            targetFormats(
                TargetFormat.Dmg,
                TargetFormat.Msi,
                TargetFormat.Deb
            )
            packageName = "Finlytics"
            packageVersion = "1.0.0"

            windows {
                menu = true
                console = true
            }

            linux {
                packageName = "finlytics"
            }
            macOS {
                bundleID = "com.finlytics.app"
            }
        }
    }
}

```

3.5. Реализация ключевых компонентов

3.5.1. Frontend

3.5.1.1. Структура слоя

Для реализации клиентской части проекта "Finlytics" использовался фреймворк Compose for Desktop, который позволяет создавать современные кроссплатформенные десктопные приложения на Kotlin с декларативным подходом к построению пользовательского интерфейса.

Основные реализованные компоненты и их функции:

1. Навигация и структура приложения:
 - Main.kt – точка входа в приложение, инициализирующая окно и настраивающая окружение Compose Desktop.
 - App.kt – главный компонент приложения, устанавливающий тему и стили Material Design;
 - AppNavigation.kt – компонент-маршрутизатор, управляющий отображением текущего экрана на основе состояния ViewModel;
 - NavigationBar.kt – навигационная панель с иконками для переключения между основными экранами приложения.
2. Основные экраны приложения:
 - OverviewScreen.kt – отображает финансовую сводку, круговую диаграмму распределения расходов по категориям, ключевые показатели (баланс, доходы, расходы) и предоставляет фильтры по временным периодам;
 - HistoryScreen.kt – показывает полный список финансовых операций с расширенными возможностями фильтрации по типу операций и временным периодам, поддерживает редактирование и удаление операций;

- `SettingsScreen.kt` – предоставляет интерфейс для управления категориями доходов и расходов: просмотр, добавление и удаление пользовательских категорий.
3. Компоненты пользовательского интерфейса:
- `PieChart.kt` – кастомный компонент для визуализации распределения расходов по категориям с использованием `Canvas API Compose`;
 - `OperationDialog.kt` – диалоговое окно для добавления и редактирования финансовых операций с валидацией вводимых данных;
 - `AddCategoryDialog.kt` – диалоговое окно для создания новых категорий доходов и расходов;
 - `GhostRaceProgressBar` – кастомный компонент для реализации модели «призрака» (сравнение показателей с предыдущим периодом);
4. Модель состояния и управление данными:
- `FinanceState.kt` – data-класс, представляющий иммутабельное состояние приложения для реактивного обновления UI;
 - `FinanceViewModel.kt` – `ViewModel`, реализующий бизнес-логику приложения и управляющий состоянием через `Flow`.
5. Ресурсы и визуальные элементы:
- `AppColors.kt` – объект, содержащий цветовую палитру приложения для поддержки единого визуального стиля;
 - `FinlyticsIconPack` – полная система векторных иконок приложения, состоящая из:
 - `__FinlyticsIconPack.kt` – основной объект-контейнер, предоставляющий доступ ко всем иконкам приложения;
 - 16 файлов векторных иконок (`Add.kt`, `Check.kt`, `Close.kt`, `Date.kt`, `Delete.kt`, `Edit.kt`, `Expenses.kt`, `History.kt`, `Income.kt`, `Left.kt`, `Minus.kt`, `Plus.kt`, `Right.kt`, `Settings.kt`, `Statistic.kt`,

Wallet.kt) – набор кастомных векторных изображений для всех элементов интерфейса.

3.5.1.2. Листинги компонентов интерфейса:

3.5.1.2.1. Структура и навигация:

Листинг 2 – Main.kt – Точка входа в приложение

```
package main

import androidx.compose.desktop.ui.tooling.preview.Preview
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material.MaterialTheme
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Window
import androidx.compose.ui.window.rememberWindowState
import androidx.compose.ui.window.application
import repository.FinanceRepository
import ui.App
import ui.components.AddCategoryDialog
import ui.components.OperationDialog
import utils.LoggingConfig
import viewmodel.FinanceViewModel

/**
 * Точка входа в приложение "Finlytics" – менеджер личных финансов.
 *
 * Приложение позволяет пользователям отслеживать доходы и расходы,
 * управлять категориями, просматривать статистику и историю операций.
 *
 * Основные функции:
 * - Добавление, редактирование и удаление финансовых операций
 * - Управление категориями доходов и расходов
 * - Просмотр статистики с фильтрацией по периодам
 * - Круговая диаграмма распределения расходов/доходов
 * - История всех операций
 * - Сравнительный прогресс-бар для анализа изменений
 *
 * @author Finlytics Team
 * @version 1.0.0
 */
@Composable
@Preview
fun AppPreview() {
    val repository = remember { FinanceRepository() }
    val viewModel = remember { FinanceViewModel(repository) }

    MaterialTheme {
        Column(modifier = Modifier.fillMaxSize()) {
            App(viewModel)

            if (viewModel.showOperationDialog) {
                OperationDialog(viewModel)
            }
        }
    }
}
```

```

        if (viewModel.showAddCategoryDialog) {
            AddCategoryDialog(viewModel)
        }
    }
}

/**
 * Главная функция приложения.
 * Инициализирует окно, настраивает логирование UTF-8 и запускает UI.
 *
 * Размер окна по умолчанию: 1440x1024 пикселей.
 * Минимальный размер окна: 800x600 пикселей.
 */
fun main() = application {
    // Логирование для корректной работы с UTF-8
    LoggingConfig.setupLogging()

    val windowState = rememberWindowState(width = 1440.dp, height = 1024.dp)

    Window(
        onCloseRequest = ::exitApplication,
        title = "Finlytics - Менеджер личных финансов",
        state = windowState,
        undecorated = false,
        resizable = true
    ) {
        // Минимальный размер окна для удобства использования
        window.minimumSize = java.awt.Dimension(800, 600)
        AppPreview()
    }
}

```

Листинг 3 – App.kt – Главный компонент приложения

```

package ui

import androidx.compose.material.MaterialTheme
import androidx.compose.runtime.Composable
import viewModel.FinanceViewModel

/**
 * Главный компонент приложения, устанавливающий тему и стили.
 * Использует MaterialTheme для единообразного оформления всех компонентов.
 *
 * @param viewModel ViewModel для управления состоянием приложения
 */
@Composable
fun App(viewModel: FinanceViewModel) {
    MaterialTheme {
        AppNavigation(viewModel)
    }
}

```

Листинг 4 – AppNavigation.kt – Компонент навигации

```

package ui

import androidx.compose.runtime.Composable
import ui.screens.HistoryScreen
import ui.screens.OverviewScreen
import ui.screens.SettingsScreen
import viewModel.FinanceViewModel

```

```

/**
 * Компонент навигации между экранами приложения.
 * В зависимости от текущего экрана в ViewModel отображает соответствующий
 * экран.
 *
 * @param viewModel ViewModel с информацией о текущем экране и данными
 */
@Composable
fun AppNavigation(viewModel: FinanceViewModel) {
    when (viewModel.currentScreen) {
        "Overview" -> OverviewScreen(viewModel)
        "History" -> HistoryScreen(viewModel)
        "Settings" -> SettingsScreen(viewModel)
    }
}

```

Листинг 5 – NavigationBar.kt – Навигационная панель

```

package ui.components

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.Icon
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.foundation.interaction.MutableInteractionSource
import ui.theme.AppColors
import ui.theme.icons.FinlyticsIconPack
import ui.theme.icons.finallyticsiconpack.*
import viewModel.FinanceViewModel

/**
 * Нижняя навигационная панель приложения.
 * Содержит кнопки для переключения между основными экранами и добавления
 * новой операции.
 *
 * Кнопки:
 * - Добавить - открывает диалог создания новой операции
 * - Обзор - переход на главный экран со статистикой
 * - История - переход на экран истории операций
 * - Настройки - переход на экран управления категориями
 *
 * @param viewModel ViewModel для управления навигацией и состоянием
 */
@Composable
fun NavigationBar(viewModel: FinanceViewModel) {
    val currentScreen = viewModel.currentScreen

    Row(
        modifier = Modifier
            .wrapContentSize()
            .background(AppColors.DarkGreyColor, RoundedCornerShape(15.dp))
            .padding(10.dp),
        horizontalArrangement = Arrangement.Center,

```

```

        verticalAlignment = Alignment.CenterVertically
    ) {
        NavigationButton(
            text = "Добавить",
            isSelected = currentScreen == "AddNew",
            onClick = { viewModel.showAddOperation() },
            Icon = FinlyticsIconPack.Add
        )

        Spacer(Modifier.width(20.dp))

        NavigationButton(
            text = "Обзор",
            isSelected = currentScreen == "Overview",
            onClick = { viewModel.navigateTo("Overview") },
            Icon = FinlyticsIconPack.Statistic
        )

        Spacer(Modifier.width(20.dp))

        NavigationButton(
            text = "История",
            isSelected = currentScreen == "History",
            onClick = { viewModel.navigateTo("History") },
            Icon = FinlyticsIconPack.History
        )

        Spacer(Modifier.width(20.dp))

        NavigationButton(
            text = "Настройки",
            isSelected = currentScreen == "Settings",
            onClick = { viewModel.navigateTo("Settings") },
            Icon = FinlyticsIconPack.Settings
        )
    }
}

/**
 * Отдельная кнопка в навигационной панели.
 * При выборе кнопки отображается только иконка, а при активном состоянии -
 * также текст.
 *
 * @param text Текст кнопки
 * @param isSelected Активна ли кнопка (соответствует текущему экрану)
 * @param onClick Callback при нажатии
 * @param Icon Иконка кнопки
 */
@Composable
fun NavigationButton(
    text: String,
    isSelected: Boolean,
    onClick: () -> Unit,
    Icon: ImageVector
) {
    val backgroundColor = if (isSelected) AppColors.BlueColor else AppColors.LightGreyColor
    val textColor = AppColors.LightColor
    val iconColor = AppColors.LightColor
    val interactionSource = remember { MutableInteractionSource() }

    Box(
        modifier = Modifier
            .wrapContentWidth()

```

```

        .background(backgroundColors, RoundedCornerShape(15.dp))
        .padding(horizontal = 10.dp, vertical = 8.dp)
        .clickable(
            interactionSource = interactionSource,
            indication = null,
            onClick = onClick
        ),
        contentAlignment = Alignment.Center
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(
                imageVector = Icon,
                contentDescription = text,
                modifier = Modifier.size(28.dp),
                tint = iconColor
            )

            if (isSelected) {
                Spacer(Modifier.width(5.dp))
                Text(
                    text = text,
                    style = TextStyle(
                        fontSize = 24.sp,
                        fontWeight = FontWeight.Medium,
                        color = textColor,
                        letterSpacing = 0.sp
                    ),
                    maxLines = 1
                )
            }
        }
    }
}

```

3.5.1.2.2. Основные экраны:

Листинг 6 – OverviewScreen.kt – Экран финансовой сводки

```

package ui.screens

import androidx.compose.foundation.*
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

```

```

import java.time.LocalDate
import java.time.format.DateTimeFormatter
import java.time.temporal.TemporalAdjusters
import kotlin.math.min
import ui.components.GhostRaceProgressBar
import ui.components.NavigationBar
import ui.components.PieChart
import ui.theme.AppColors
import ui.theme.icons.FinlyticsIconPack
import ui.theme.icons.finlyticsiconpack.*
import viewmodel.FinanceViewModel

/**
 * Экран обзора финансовой статистики.
 * Отображает круговую диаграмму распределения расходов/доходов,
 * финансовую сводку (баланс, доходы, расходы) и список категорий.
 *
 * Основные функции:
 * - Фильтрация данных по типу (Доходы/Расходы)
 * - Фильтрация по временному периоду (День/Неделя/Месяц/Год/Всё время)
 * - Отображение круговой диаграммы распределения по категориям
 * - Отображение финансовой сводки (доходы, расходы, баланс)
 * - Сравнение расходов с предыдущим периодом ("Гонка с призракoм")
 * - Список категорий с процентами от общей суммы
 *
 * @param viewModel ViewModel для управления финансовыми данными
 */
@Composable
fun OverviewScreen(viewModel: FinanceViewModel) {
    val state by viewModel.state.collectAsState()

    // Состояния для фильтров
    var selectedFilter by remember { mutableStateOf("Расходы") }
    var selectedPeriod by remember { mutableStateOf("Всё время") }
    var selectedDate by remember { mutableStateOf(LocalDate.now()) }

    // Форматирование даты для отображения
    val dateFormatter = DateTimeFormatter.ofPattern("dd.MM.yyyy")
    val monthYearFormatter = DateTimeFormatter.ofPattern("MM.yyyy")
    val yearFormatter = DateTimeFormatter.ofPattern("yyyy")

    /**
     * Фильтрация всех операций за выбранный период (без учёта типа).
     * Используется для расчёта общей статистики (баланс, доходы, расходы).
     */
    val allOperationsForPeriod = remember(state.operations, selectedPeriod,
selectedDate) {
        filterOperationsByPeriod(
            operations = state.operations,
            period = selectedPeriod,
            selectedDate = selectedDate
        )
    }

    /**
     * Фильтрация операций по типу и периоду.
     * Используется для построения круговой диаграммы и списка категорий.
     */
    val filteredOperationsByType = remember(state.operations,
selectedFilter, selectedPeriod, selectedDate) {
        filterOperationsByType(
            operations = state.operations,
            filterType = selectedFilter,
            period = selectedPeriod,

```

```

        selectedDate = selectedDate
    )
}

// Вычисляем статистику на основе ВСЕХ операций за период (вне зависимости от фильтра)
val totalIncome = allOperationsForPeriod.filter { it.type == "Доход" }.sumOf { it.amount }
val totalExpenses = allOperationsForPeriod.filter { it.type == "Расход" }.sumOf { it.amount }
val balance = totalIncome - totalExpenses

// Гонка с Призраком за предыдущий такой же выбранный период
val currentPeriodExpenses = totalExpenses
val previousPeriodExpenses = remember(selectedPeriod, selectedDate, state.operations) {
    if (selectedPeriod == "Всё время") 0.0
    else {
        val previousDate = when (selectedPeriod) {
            "День" -> selectedDate.minusDays(1)
            "Неделя" -> selectedDate.minusWeeks(1)
            "Месяц" -> selectedDate.minusMonths(1)
            "Год" -> selectedDate.minusYears(1)
            else -> selectedDate
        }
        val previousPeriodOps =
            filterOperationsByPeriod(state.operations, selectedPeriod, previousDate)
        previousPeriodOps.filter { it.type == "Расход" }.sumOf {
            it.amount
        }
    }
}
val ghostRacePercent = if (previousPeriodExpenses > 0)
    currentPeriodExpenses / previousPeriodExpenses else 0.0

// Группируем операции по категориям в зависимости от выбранного фильтра (только для диаграммы)
val operationsByCategory = remember(selectedFilter, filteredOperationsByType) {
    when (selectedFilter) {
        "Доходы" -> {
            filteredOperationsByType
                .filter { it.type == "Доход" }
                .groupBy { it.category }
                .mapValues { entry -> entry.value.sumOf { op ->
                    op.amount
                } }
        }
        "Расходы" -> {
            filteredOperationsByType
                .filter { it.type == "Расход" }
                .groupBy { it.category }
                .mapValues { entry -> entry.value.sumOf { op ->
                    op.amount
                } }
        }
        else -> emptyMap()
    }
}

// Подготавливаем список категорий для отображения с процентами
val categoryList = operationsByCategory.entries.sortedByDescending {
    it.value
}

/**
 * Получение отображаемого текста даты в зависимости от выбранного периода.

```

```

* Для дня: DD.MM.YYYY
* Для недели: DD.MM.YYYY - DD.MM.YYYY
* Для месяца: MM.YYYY
* Для года: YYYY
*/
val displayDateText = remember(selectedPeriod, selectedDate) {
    when (selectedPeriod) {
        "День" -> selectedDate.format(dateFormatter)
        "Неделя" -> {
            val weekStart =
selectedDate.with(TemporalAdjusters.previousOrSame(java.time.DayOfWeek.MONDAY))
            val weekEnd = weekStart.plusDays(6)
            "${weekStart.format(dateFormatter)} -
${weekEnd.format(dateFormatter)}"
        }
        "Месяц" -> selectedDate.format(monthYearFormatter)
        "Год" -> selectedDate.format(yearFormatter)
        else -> ""
    }
}

// Отладочная информация о данных
LaunchedEffect(allOperationsForPeriod, filteredOperationsByType,
operationsByCategory) {
    println("\n=== OVERVIEW SCREEN ===")
    println("Фильтр: $selectedFilter, Период: $selectedPeriod")
    println("Всего операций: ${state.operations.size}")
    println("Операций за период (все): ${allOperationsForPeriod.size}")
    println("Операций за период ($selectedFilter):
${filteredOperationsByType.size}")
    println("Текущий баланс: ${balance} руб.")
    println("Общие доходы: ${totalIncome} руб.")
    println("Общие расходы: ${totalExpenses} руб.")
    println("Категорий для диаграммы ($selectedFilter):
${operationsByCategory.size}")
    println("Данные для диаграммы: $operationsByCategory")
    println("=====\n")
}

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(AppColors.DarkColor)
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(horizontal = 25.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Секция настроек
        Box(
            modifier = Modifier
                .padding(top = 20.dp)
                .height(193.dp)
                .background(AppColors.DarkGreyColor,
RoundedCornerShape(30.dp))
        ) {
            Row(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(start = 40.dp, end = 40.dp, top = 15.dp),
                horizontalArrangement = Arrangement.SpaceBetween,

```



```

        verticalAlignment = Alignment.Top
    ) {
        // Фильтр по типу
        Column(
            modifier = Modifier.width(390.dp),
            verticalArrangement = Arrangement.spacedBy(15.dp)
        ) {
            Text(
                "Параметр отображения",
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )

            // Кнопка "Доходы"
            FilterButton(
                text = "Доходы",
                isSelected = selectedFilter == "Доходы",
                icon = true,
                modifier = Modifier.width(390.dp).height(42.dp),
                onClick = { selectedFilter = "Доходы" }
            )

            // Кнопка "Расходы"
            FilterButton(
                text = "Расходы",
                isSelected = selectedFilter == "Расходы",
                icon = true,
                isIncome = false,
                modifier = Modifier.width(390.dp).height(42.dp),
                onClick = { selectedFilter = "Расходы" }
            )
        }

        // Период времени
        Column(
            verticalArrangement = Arrangement.spacedBy(15.dp)
        ) {
            Text(
                "Период времени",
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )

            // Кнопки периодов
            Row(
                modifier = Modifier.width(350.dp),
                horizontalArrangement = Arrange-
ment.spacedBy(20.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                PeriodButton(
                    text = "День",
                    isSelected = selectedPeriod == "День",
                    onClick = {
                        selectedPeriod = "День"
                        selectedDate = LocalDate.now()
                    }
                )
                PeriodButton(
                    text = "Неделя",
                    isSelected = selectedPeriod == "Неделя",
                    onClick = {

```

```

                selectedPeriod = "Неделя"
                selectedDate = LocalDate.now()
            }
        )
        PeriodButton(
            text = "Месяц",
            isSelected = selectedPeriod == "Месяц",
            onClick = {
                selectedPeriod = "Месяц"
                selectedDate = LocalDate.now()
            }
        )
    }

    Row(
        modifier = Modifier.width(350.dp),
        horizontalArrangement = Arrange-
ment.spacedBy(20.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        PeriodButton(
            text = "Год",
            isSelected = selectedPeriod == "Год",
            onClick = {
                selectedPeriod = "Год"
                selectedDate = LocalDate.now()
            }
        )
        PeriodButton(
            text = "Всё время",
            isSelected = selectedPeriod == "Всё время",
            onClick = {
                selectedPeriod = "Всё время"
            }
        )
    }

    // Настройки даты
    if (selectedPeriod != "Всё время") {
        Column(
            verticalArrangement = Arrange-
ment.spacedBy(15.dp)
        ) {
            Text(
                selectedPeriod,
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )

            // Поле ввода даты
            Box(
                modifier = Modifier
                    .width(390.dp)
                    .height(42.dp)
                    .background(AppColors.LightGreyColor,
RoundedCornerShape(10.dp))
            ) {
                Row(
                    modifier = Modifier.fillMaxSize(),
                    verticalAlignment = Alignment.CenterVer-
tically
                ) {

```

```

        Pack.Date,

        Spacer(Modifier.width(16.dp))

        Icon(
            imageVector = FinlyticsIcon-

            contentDescription = "date",
            modifier = Modifier.size(18.dp),
            tint = AppColors.LightColor
        )

        Spacer(Modifier.width(18.dp))

        Text(
            displayDateText,
            fontSize = 20.sp,
            fontWeight = FontWeight.Normal,
            color = AppColors.LightColor
        )

        Spacer(Modifier.weight(1f))

        // Кнопки навигации
        Row(
            modifier = Modifier.padding(end =

            horizontalArrangement = Arrange-

            verticalAlignment = Alignment.Cen-

        ) {
            IconButton(
                onClick = {
                    // Предыдущий период
                    selectedDate = when (select-

                        "День" -> selected-

                        "Неделя" -> selected-

                        "Месяц" -> selected-

                        "Год" -> selected-

                        else -> selectedDate
                    }
                },
                modifier = Modifier
                    .background(AppColors.Blue-
                    .size(20.dp)
            ) {
                Icon(
                    imageVector = FinlyticsIcon-

                    contentDescription = "previ-

                    modifier = Modi-

                    tint = AppColors.LightColor
                )
            }

            IconButton(
                onClick = {

```

```

// Следующий период
selectedDate = when (selectedDate) {
    "День" -> selectedDate.plusDays(1)
    "Неделя" -> selectedDate.plusWeeks(1)
    "Месяц" -> selectedDate.plusMonths(1)
    "Год" -> selectedDate.plusYears(1)
    else -> selectedDate
}
},
modifier = Modifier
    .background(AppColors.BlueColor, RoundedCornerShape(5.dp))
    .size(20.dp)
) {
    Icon(
        imageVector = FinlyticsIconPack.Right,
        "next_period",
        modifier = Modifier.size(20.dp),
        tint = AppColors.LightColor
    )
}
}
}
}
} else {
    // Пустой колонка для выравнивания, когда поле даты скрыто
    Spacer(modifier = Modifier.width(390.dp))
}
}

Spacer(Modifier.height(40.dp))

// Основной контент
Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(bottom = 100.dp),
    horizontalArrangement = Arrangement.spacedBy(90.dp)
) {
    // Диаграмма и статистика (левая часть)
    Box(
        modifier = Modifier.size(650.dp),
        contentAlignment = Alignment.TopStart
    ) {
        // Диаграмма для выбранного фильтра
        Box(
            modifier = Modifier
                .size(650.dp)
                .align(Alignment.TopCenter)
        ) {
            if (operationsByCategory.isNotEmpty()) {
                PieChart(
                    operationsByCategory,

```



```

        contentDescription = "income",
        modifier = Modifier.size(22.dp),
        tint = AppColors.GreenColor
    )
    Text(
        "Доходы",
        fontSize = 24.sp,
        fontWeight = FontWeight.Medium,
        color = AppColors.GreenColor
    )
}
Spacer(Modifier.height(8.dp))
Row(
    vertically,
    horizontalArrangement = Arrange-
ment.spacedBy(10.dp)
) {
    Spacer(Modifier.weight(1f))
    Text(
        String.format("%.2f", totalIncome),
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.GreenColor
    )
    // Иконка валюты
    Text(
        text = "₽",
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.GreenColor
    )
}
}

// Расходы за период
Surface(
    color = AppColors.RedColor.copy(alpha = 0.25f),
    shape = RoundedCornerShape(20.dp),
    border = BorderStroke(2.dp, AppColors.RedColor)
) {
    Column(
        modifier = Modifier
            .height(90.dp)
            .fillMaxWidth()
            .padding(horizontal = 16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Row(
            vertically,
            horizontalArrangement = Arrange-
ment.spacedBy(15.dp)
        ) {
            // Иконка расходов
            Icon(
                imageVector = FinlyticsIconPack.Ex-
penses,
                contentDescription = "expenses",
                modifier = Modifier.size(22.dp),
                tint = AppColors.RedColor
            )
            Text(

```

```

        "Расходы",
        fontSize = 24.sp,
        fontWeight = FontWeight.Medium,
        color = AppColors.RedColor
    )
}
Spacer(Modifier.height(8.dp))
Row(
    vertically,
    horizontalArrangement = Arrange-
ment.spacedBy(10.dp)
) {
    Spacer(Modifier.weight(1f))
    Text(
        String.format("%.2f", totalEx-
penses),
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.RedColor
    )
    // Иконка валюты
    Text(
        text = "₽",
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.RedColor
    )
}
}

// Баланс за период
Surface(
    color = AppColors.YellowColor.copy(alpha =
0.25f),
    shape = RoundedCornerShape(20.dp),
    border = BorderStroke(2.dp, AppColors.Yellow-
Color)
) {
    Column(
        modifier = Modifier
            .height(90.dp)
            .fillMaxWidth()
            .padding(horizontal = 16.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Row(
            vertically,
            horizontalArrangement = Arrange-
ment.spacedBy(15.dp)
        ) {
            // Иконка кошелька
            Icon(
                let,
                imageVector = FinlyticsIconPack.Wal-
let,
                contentDescription = "wallet",
                modifier = Modifier.size(20.dp),
                tint = AppColors.YellowColor
            )
            Text(
                "Баланс",
                fontSize = 24.sp,

```

```

        fontWeight = FontWeight.Medium,
        color = AppColors.YellowColor
    )
}
Spacer(Modifier.height(8.dp))
Row(
    verticalAlignment = Alignment.CenterVer-
tically,
    horizontalArrangement = Arrange-
ment.spacedBy(10.dp)
) {
    Spacer(Modifier.weight(1f))
    Text(
        String.format("%.2f", balance),
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.YellowColor
    )
    // Иконка валюты
    Text(
        text = "₽",
        fontSize = 24.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.YellowColor
    )
}
}
}
}

// Гонка с призракoм + Список категорий
Column(
    modifier = Modifier
        .weight(1f)
        .fillMaxHeight()
) {
    // Прогресс-бар гонки (только если выбран не "Всё
время")
    if (selectedPeriod != "Всё время") {
        GhostRaceProgressBar(
            currentExpenses = currentPeriodExpenses,
            previousExpenses = previousPeriodExpenses,
            percent = ghostRacePercent,
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(25.dp))
    }

    LazyColumn(
        verticalArrangement = if (categoryList.isEmpty()) {
            Arrangement.Center
        } else {
            Arrangement.spacedBy(25.dp)
        }
    ) {
        if (categoryList.isEmpty()) {
            item {
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(200.dp),
                    contentAlignment = Alignment.Center
                ) {

```



```

// Сообщение об отсутствии данных для
выбранного фильтра
Text(
    if (filteredOperationsByType.isEmpty()) "Нет операций за выбранный период" else "Нет ${selectedFilter.lowercase()} за выбранный период",
    fontSize = 20.sp,
    color = AppColors.LightColor.copy(alpha = 0.5f),
)
}
} else {
    val totalForCategoryType = if (selectedFilter == "Доходы") {
        filteredOperationsByType
            .filter { it.type == "Доход" }
            .sumOf { it.amount }
    } else {
        filteredOperationsByType
            .filter { it.type == "Расход" }
            .sumOf { it.amount }
    }

    val gradients = if (selectedFilter == "Доходы")
{
        listOf(
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor1)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor2)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor3)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor4)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor5)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor6)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor7)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.IncomeColor8))
        )
    } else {
        listOf(
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor1)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor2)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor3)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor4)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor5)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor6)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor7)),
            Brush.horizontalGradient(listOf(AppColors.DarkColor, AppColors.ExpensesColor8))
        )
    }
}
}

```

```

items(categoryList.size) { index ->
    val (category, amount) = categoryList[index]
    val percentage = if (totalForCategoryType >
0) {
        (amount / totalForCategoryType) * 100
    } else {
        0.0
    }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .height(67.dp)
    ) {
        Row(
            verticalAlignment = Alignment.Cen-
terVertically

        ) {
            // Название категории
            Text(
                category,
                fontSize = 24.sp,
                fontWeight = FontWeight.Normal,
                color = AppColors.LightColor
            )

            Spacer(Modifier.weight(1f))

            // Сумма
            Row(
                horizontalArrangement = Arrange-
ment.spacedBy(4.dp)

            ) {
                Text(
                    String.format("%.2f",
amount),

                    fontSize = 24.sp,
                    fontWeight = FontWeight.Nor-
mal,

                    color = AppColors.LightColor
                )

                // Иконка валюты
                Text(
                    text = "₽",
                    fontSize = 24.sp,
                    fontWeight = FontWeight.Nor-
mal,

                    color = AppColors.LightColor
                )
            }

            Spacer(Modifier.height(8.dp))

            Box(modifier = Modifier.height(35.dp)) {
                // Прогресс-бар
                val normalizedPercentage = min(per-
centage, 100.0)

                val progressWidth = normalizedPer-
centage / 100.0

                Box(
                    modifier = Modifier

```

```

        .fillMaxWidth()
        .height(35.dp)
        .background(
            AppColors.DarkColor,
            shape = RoundedCorner-
Shape(9.dp)
        )
    ) {
        // Градиентная часть прогресс-
        бара
        Box(
            modifier = Modifier
                .align(Alignment.Cen-
terEnd)
                .fillMaxWidth(fraction =
progressWidth.toFloat())
                .height(35.dp)
                .background(
                    gradi-
ents.getOrNull(index) { gradients.last() },
                    shape = Rounded-
CornerShape(9.dp)
                )
        )
    }

    // Процент
    Text(
        String.format("%.2f", percent-
age) + " %",
        fontSize = 20.sp,
        fontWeight = FontWeight.Normal,
        color = AppColors.LightColor,
        modifier = Modifier
            .align(Alignment.CenterEnd)
            .padding(end = 8.dp)
    )
}

}

}

}

}

}

}

}

// Навигационная панель
Box(
    modifier = Modifier
        .padding(bottom = 10.dp)
        .align(Alignment.BottomCenter)
) {
    NavigationBar(viewModel)
}
}

}

/**
 * Фильтрует операции по временному периоду.
 *
 * Поддерживаемые периоды:
 * - "День": все операции за выбранную дату
 * - "Неделя": все операции с понедельника по воскресенье выбранной недели
 * - "Месяц": все операции с первого по последний день выбранного месяца

```

```

* - "Год": все операции с первого по последний день выбранного года
* - "Всё время": все операции без фильтрации по дате
*
* @param operations Список всех операций для фильтрации
* @param period Выбранный период ("День", "Неделя", "Месяц", "Год", "Всё
время")
* @param selectedDate Опорная дата для расчёта границ периода
* @return Отфильтрованный список операций, отсортированный по убыванию даты
*/
private fun filterOperationsByPeriod(
    operations: List<models.Operation>,
    period: String,
    selectedDate: LocalDate
): List<models.Operation> {
    return when (period) {
        "День" -> {
            operations.filter { it.date == selectedDate }
        }
        "Неделя" -> {
            val weekStart = selectedDate.with(TemporalAdjusters.previousOrSame(java.time.DayOfWeek.MONDAY))
            val weekEnd = weekStart.plusDays(6)
            operations.filter {
                val opDate = it.date
                opDate in weekStart..weekEnd
            }
        }
        "Месяц" -> {
            val monthStart = selectedDate.withDayOfMonth(1)
            val monthEnd = selectedDate.with(TemporalAdjusters.lastDayOfMonth())
            operations.filter {
                val opDate = it.date
                opDate in monthStart..monthEnd
            }
        }
        "Год" -> {
            val yearStart = selectedDate.withDayOfYear(1)
            val yearEnd = selectedDate.with(TemporalAdjusters.lastDayOfYear())
            operations.filter {
                val opDate = it.date
                opDate in yearStart..yearEnd
            }
        }
        else -> operations // "Всё время"
    }.sortedByDescending { it.date }
}

/**
* Фильтрует операции по типу и временному периоду.
*
* Сначала применяется фильтрация по типу операции (Доходы/Расходы),
* затем фильтрация по выбранному временному периоду.
*
* @param operations Список всех операций для фильтрации
* @param filterType Тип фильтра ("Доходы", "Расходы" или другое значение
для всех операций)
* @param period Выбранный период ("День", "Неделя", "Месяц", "Год", "Всё
время")
* @param selectedDate Опорная дата для расчёта границ периода
* @return Отфильтрованный список операций, отсортированный по убыванию даты
*/
private fun filterOperationsByType(

```

```

        operations: List<models.Operation>,
        filterType: String,
        period: String,
        selectedDate: LocalDate
    ): List<models.Operation> {
        // Фильтрация по типу операции
        val filteredByType = when (filterType) {
            "Доходы" -> operations.filter { it.type == "Доход" }
            "Расходы" -> operations.filter { it.type == "Расход" }
            else -> operations
        }

        // Фильтрация по периоду времени
        return when (period) {
            "День" -> {
                filteredByType.filter { it.date == selectedDate }
            }
            "Неделя" -> {
                val weekStart =
                    selectedDate.with(TemporalAdjusters.previousOrSame(java.time.DayOfWeek.MONDAY))
                val weekEnd = weekStart.plusDays(6)
                filteredByType.filter {
                    val opDate = it.date
                    opDate in weekStart..weekEnd
                }
            }
            "Месяц" -> {
                val monthStart = selectedDate.withDayOfMonth(1)
                val monthEnd =
                    selectedDate.with(TemporalAdjusters.lastDayOfMonth())
                filteredByType.filter {
                    val opDate = it.date
                    opDate in monthStart..monthEnd
                }
            }
            "Год" -> {
                val yearStart = selectedDate.withDayOfYear(1)
                val yearEnd =
                    selectedDate.with(TemporalAdjusters.lastDayOfYear())
                filteredByType.filter {
                    val opDate = it.date
                    opDate in yearStart..yearEnd
                }
            }
            else -> filteredByType // "Всё время"
        }.sortedByDescending { it.date }
    }
}

/**
 * Кнопка фильтрации для выбора типа операций.
 *
 * Отображает кнопку с иконкой (для доходов/расходов) или без неё.
 * Цвет кнопки меняется в зависимости от выбранного состояния:
 * - Для "Доходов": зелёный цвет
 * - Для "Расходов": красный цвет
 * - Для "Всех": синий цвет
 * - Невыбранное состояние: серый цвет
 *
 * @param text Текст кнопки
 * @param isSelected Выбрана ли кнопка в данный момент
 * @param modifier Модификатор для настройки внешнего вида
 * @param icon Флаг отображения иконки
 * @param isAll Флаг, является ли кнопка кнопкой "Все"

```

```

    * @param isIncome Флаг, является ли кнопка кнопкой доходов (для определения
цвета)
    * @param onClick Callback при нажатии на кнопку
    */
@Composable
private fun FilterButton(
    text: String,
    isSelected: Boolean,
    modifier: Modifier = Modifier,
    icon: Boolean = false,
    isAll: Boolean = false,
    isIncome: Boolean = true,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = modifier,
        shape = RoundedCornerShape(10.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (!isSelected) AppColors.LightGreyColor
            else if (isAll) AppColors.BlueColor
            else if (isIncome) AppColors.GreenColor
            else AppColors.RedColor
        ),
        elevation = null
    ) {
        if (icon) {
            Row(
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically
            ) {
                if (isIncome) {
                    Icon(
                        imageVector = FinlyticsIconPack.Income,
                        contentDescription = "income",
                        modifier = Modifier.size(22.dp),
                        tint = AppColors.LightColor
                    )
                } else {
                    Icon(
                        imageVector = FinlyticsIconPack.Expenses,
                        contentDescription = "expenses",
                        modifier = Modifier.size(22.dp),
                        tint = AppColors.LightColor
                    )
                }
                Spacer(Modifier.width(12.dp))
                Text(
                    text,
                    fontSize = 20.sp,
                    fontWeight = FontWeight.Normal,
                    color = AppColors.LightColor
                )
            }
        } else {
            Text(
                text,
                fontSize = 20.sp,
                fontWeight = FontWeight.Normal,
                color = AppColors.LightColor
            )
        }
    }
}

```

```

/**
 * Кнопка выбора временного периода.
 *
 * Отображает кнопку с текстом периода. Цвет кнопки меняется на синий при
 * выборе.
 *
 * @param text Текст кнопки (День, Неделя, Месяц, Год, Всё время)
 * @param isSelected Выбрана ли кнопка в данный момент
 * @param onClick Callback при нажатии на кнопку
 */
@Composable
private fun PeriodButton(
    text: String,
    isSelected: Boolean,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = Modifier.defaultMinSize(minWidth = 70.dp),
        shape = RoundedCornerShape(10.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (isSelected) AppColors.BlueColor else
AppColors.LightGreyColor
        ),
        elevation = null
    ) {
        Text(
            text,
            fontSize = 20.sp,
            fontWeight = FontWeight.Normal,
            color = AppColors.LightColor
        )
    }
}

```

Листинг 7 – HistoryScreen.kt – Экран истории операций

```

package ui.screens

import androidx.compose.foundation.*
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.*
import androidx.compose.material.Icon
import androidx.compose.material.IconButton
import androidx.compose.runtime.*
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import java.time.LocalDate
import java.time.format.DateTimeFormatter
import java.time.temporal.TemporalAdjusters
import models.Operation
import ui.components.NavigationBar
import ui.theme.AppColors

```

```

import ui.theme.icons.FinlyticsIconPack
import ui.theme.icons.finallyticsiconpack.*
import viewmodel.FinanceViewModel

/**
 * Экран "История" приложения.
 * Отображает список всех финансовых операций с возможностью фильтрации.
 *
 * Основные функции:
 * - Фильтрация по типу операций (Все/Доходы/Расходы)
 * - Фильтрация по временному периоду (День/Неделя/Месяц/Год/Всё время)
 * - Редактирование операции (открывает диалог OperationDialog)
 * - Удаление операции с подтверждением (единый стиль с остальными диало-
    гами)
 *
 * Каждая операция отображается с:
 * - Датой операции
 * - Категорией с цветной рамкой
 * - Описанием (или "Без описания")
 * - Суммой с иконкой плюса/минуса
 * - Кнопками редактирования и удаления
 *
 * @param viewModel ViewModel для управления данными
 */
@Composable
fun HistoryScreen(viewModel: FinanceViewModel) {
    val state by viewModel.state.collectAsState()

    // Состояния для фильтров
    var selectedFilter by remember { mutableStateOf("Все") }
    var selectedPeriod by remember { mutableStateOf("Всё время") }
    var selectedDate by remember { mutableStateOf(LocalDate.now()) }
    var operationToDelete by remember { mutableStateOf<Operation?>(null) }

    // Форматирование даты для отображения
    val dateFormatter = DateTimeFormatter.ofPattern("dd.MM.yyyy")
    val monthYearFormatter = DateTimeFormatter.ofPattern("MM.yyyy")
    val yearFormatter = DateTimeFormatter.ofPattern("yyyy")

    /**
     * Фильтрация операций по выбранным критериям.
     * Сначала применяется фильтр по типу, затем по временному периоду.
     */
    val filteredOperations = remember(state.operations, selectedFilter, se-
lectedPeriod, selectedDate) {
        filterOperations(
            operations = state.operations,
            filterType = selectedFilter,
            period = selectedPeriod,
            selectedDate = selectedDate
        )
    }

    // Отладочная информация об операциях
    LaunchedEffect(state.operations, filteredOperations) {
        println("\n=== HISTORY SCREEN ===")
        println("Всего операций в базе: ${state.operations.size}")
        println("Отфильтровано операций: ${filteredOperations.size}")
        println("Фильтр: $selectedFilter, Период: $selectedPeriod")
        println("Операций доходов: ${filteredOperations.count { it.type ==
"Доход" }}")
        println("Операций расходов: ${filteredOperations.count { it.type ==
"Расход" }}")
        if (filteredOperations.isNotEmpty()) {

```



```

println("Последние 3 отфильтрованные операции:")
filteredOperations.take(3).forEach { op ->
    println("    - ${op.type}: ${op.category} - ${op.amount} руб.
(${op.date})")
}
}
println("=====\n")
}

/**
 * Получение отображаемого текста даты в зависимости от выбранного пери-
ода.
 * Для дня: DD.MM.YYYY
 * Для недели: DD.MM.YYYY - DD.MM.YYYY
 * Для месяца: MM.YYYY
 * Для года: YYYY
 */
val displayDateText = remember(selectedPeriod, selectedDate) {
    when (selectedPeriod) {
        "День" -> selectedDate.format(dateFormatter)
        "Неделя" -> {
            val weekStart =
selectedDate.with(TemporalAdjusters.previousOrSame(java.time.DayOfWeek.MONDA
Y))

            val weekEnd = weekStart.plusDays(6)
            "${weekStart.format(dateFormatter)} -
${weekEnd.format(dateFormatter)}"
        }
        "Месяц" -> selectedDate.format(monthYearFormatter)
        "Год" -> selectedDate.format(yearFormatter)
        else -> ""
    }
}

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(AppColors.DarkColor)
) {
    // Контент страницы
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 25.dp)
    ) {
        // Секция настроек
        Box(
            modifier = Modifier
                .padding(top = 20.dp)
                .height(193.dp)
                .background(AppColors.DarkGreyColor,
RoundedCornerShape(30.dp))
        ) {
            Row(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(start = 40.dp, end = 40.dp, top = 15.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.Top
            ) {
                // Фильтр по типу
                Column(
                    modifier = Modifier.width(390.dp),
                    verticalArrangement = Arrangement.spacedBy(15.dp)
                )
            }
        }
    }
}

```

```

    ) {
        Text(
            "Параметр отображения",
            fontSize = 24.sp,
            fontWeight = FontWeight.Medium,
            color = AppColors.LightColor
        )

        Row(
            horizontalArrangement =
Arrangement.spacedBy(15.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Кнопка "Все"
            FilterButton(
                text = "Все",
                isSelected = selectedFilter == "Все",
                isAll = true,
                modifier =
Modifier.width(85.dp).height(98.dp),
                onClick = { selectedFilter = "Все" }
            )

            Column(
                verticalArrangement =
Arrangement.spacedBy(15.dp)
            ) {
                // Кнопка "Доходы"
                FilterButton(
                    text = "Доходы",
                    isSelected = selectedFilter == "Доходы",
                    icon = true,
                    modifier =
Modifier.width(290.dp).height(42.dp),
                    onClick = { selectedFilter = "Доходы" }
                )

                // Кнопка "Расходы"
                FilterButton(
                    text = "Расходы",
                    isSelected = selectedFilter == "Рас-
ходы",
                    icon = true,
                    isIncome = false,
                    modifier =
Modifier.width(290.dp).height(42.dp),
                    onClick = { selectedFilter = "Расходы" }
                )
            }
        }
    }

    // Период времени
    Column(
        verticalArrangement = Arrangement.spacedBy(15.dp)
    ) {
        Text(
            "Период времени",
            fontSize = 24.sp,
            fontWeight = FontWeight.Medium,
            color = AppColors.LightColor
        )

        // Кнопки периодов

```

```

        Row(
            modifier = Modifier.width(350.dp),
            horizontalArrangement = Arrange-
ment.spacedBy(20.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            PeriodButton(
                text = "День",
                isSelected = selectedPeriod == "День",
                onClick = {
                    selectedPeriod = "День"
                    selectedDate = LocalDate.now()
                }
            )
            PeriodButton(
                text = "Неделя",
                isSelected = selectedPeriod == "Неделя",
                onClick = {
                    selectedPeriod = "Неделя"
                    selectedDate = LocalDate.now()
                }
            )
            PeriodButton(
                text = "Месяц",
                isSelected = selectedPeriod == "Месяц",
                onClick = {
                    selectedPeriod = "Месяц"
                    selectedDate = LocalDate.now()
                }
            )
        }

        Row(
            modifier = Modifier.width(350.dp),
            horizontalArrangement = Arrange-
ment.spacedBy(20.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            PeriodButton(
                text = "Год",
                isSelected = selectedPeriod == "Год",
                onClick = {
                    selectedPeriod = "Год"
                    selectedDate = LocalDate.now()
                }
            )
            PeriodButton(
                text = "Всё время",
                isSelected = selectedPeriod == "Всё время",
                onClick = {
                    selectedPeriod = "Всё время"
                }
            )
        }
    }

    // Настройки даты
    if (selectedPeriod != "Всё время") {
        Column(
            verticalArrangement = Arrange-
ment.spacedBy(15.dp)
        ) {
            Text(
                selectedPeriod,

```

```

        fontSize = 24.sp,
        fontWeight = FontWeight.Medium,
        color = AppColors.LightColor
    )

    // Поле ввода даты
    Box(
        modifier = Modifier
            .width(390.dp)
            .height(42.dp)
            .background(AppColors.LightGreyColor,
RoundedCornerShape(10.dp))
    ) {
        Row(
            modifier = Modifier.fillMaxSize(),
            verticalAlignment = Alignment.CenterVer-
tically
        ) {
            Spacer(Modifier.width(16.dp))

            Icon(
                imageVector = FinlyticsIcon-
Pack.Date,
                contentDescription = "date",
                modifier = Modifier.size(18.dp),
                tint = AppColors.LightColor
            )

            Spacer(Modifier.width(18.dp))

            Text(
                displayDateText,
                fontSize = 20.sp,
                fontWeight = FontWeight.Normal,
                color = AppColors.LightColor
            )

            Spacer(Modifier.weight(1f))

            // Кнопки навигации
            Row(
                modifier = Modifier.padding(end =
6.dp),
                horizontalArrangement = Arrange-
ment.spacedBy(4.dp),
                verticalAlignment = Alignment.Cen-
terVertically
            ) {
                IconButton(
                    onClick = {
                        selectedDate = when (select-
edPeriod) {
                            "День" -> selected-
Date.minusDays(1)
                            "Неделя" -> selected-
Date.minusWeeks(1)
                            "Месяц" -> selected-
Date.minusMonths(1)
                            "Год" -> selected-
Date.minusYears(1)
                            else -> selectedDate
                        }
                    },
                    modifier = Modifier

```

```

        Color, RoundedCornerShape(5.dp))
        .background(AppColors.Blue-
        .size(20.dp)
    ) {
        Icon(
            imageVector = FinlyticsIcon-
            contentDescription = "previ-
            modifier = Modi-
            tint = AppColors.LightColor
        )
    }

    IconButton(
        onClick = {
            selectedDate = when (select-
                "День" -> selected-
                "Неделя" -> selected-
                "Месяц" -> selected-
                "Год" -> selected-
                else -> selectedDate
            )
        },
        modifier = Modifier
        .background(AppColors.Blue-
        .size(20.dp)
    ) {
        Icon(
            imageVector = FinlyticsIcon-
            contentDescription =
            modifier = Modi-
            tint = AppColors.LightColor
        )
    }
}

}

}

} else {
    // Пустой колонка для выравнивания, когда поле даты
    скрыто
    Spacer(modifier = Modifier.width(390.dp))
}

}

// Основной контент
Box(
    modifier = Modifier
        .fillMaxSize()
        .padding(top = 243.dp)
) {
    if (filteredOperations.isEmpty()) {

```

```

        // Сообщение, если операций нет
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center
        ) {
            Text(
                "Нет операций за выбранный период",
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightGreyColor
            )
        }
    } else {
        // Список операций
        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.spacedBy(20.dp)
        ) {
            items(filteredOperations) { operation ->
                TransactionItem(
                    operation = operation,
                    onEditClick = { viewModel.showEditOpera-
tion(operation) },
                    onDeleteClick = { operationToDelete = opera-
tion }
                )

                // Разделитель (кроме последнего элемента)
                if (operation != filteredOperations.last()) {
                    Divider(
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(1.dp),
                        color = AppColors.LightGreyColor
                    )
                }
            }
        }
    }
}

// Навигационная панель
Box(
    modifier = Modifier
        .padding(bottom = 10.dp)
        .align(Alignment.BottomCenter)
) {
    NavigationBar(viewModel)
}

// Диалог подтверждения удаления (обновленное оформление)
if (operationToDelete != null) {
    val operation = operationToDelete!!
    val isIncome = operation.type == "Доход"
    val typeColor = if (isIncome) AppColors.GreenColor else AppCol-
ors.RedColor
    val dateFormatter = DateTimeFormatter.ofPattern("dd.MM.yyyy")

    AlertDialog(
        onDismissRequest = { operationToDelete = null },
        title = {
            Text(

```

```

        "Подтверждение удаления",
        color = AppColors.LightColor,
        fontWeight = FontWeight.Medium,
        fontSize = 20.sp
    )
},
text = {
    Column(
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            "Вы уверены, что хотите удалить эту операцию?",
            color = AppColors.LightColor,
            fontSize = 14.sp
        )

        // Карточка с информацией об операции
        Surface(
            color = typeColor.copy(alpha = 0.15f),
            shape = RoundedCornerShape(12.dp),
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(
                modifier = Modifier.padding(16.dp),
                verticalArrangement = Arrange-
ment.spacedBy(12.dp)
            ) {
                // Тип и категория
                Row(
                    verticalAlignment = Alignment.CenterVerti-
cally,
                    horizontalArrangement = Arrange-
ment.spacedBy(8.dp)
                ) {
                    // Иконка типа операции
                    Icon(
                        imageVector = if (isIncome) Fin-
lyticsIconPack.Income else FinlyticsIconPack.Expenses,
                        contentDescription = operation.type,
                        tint = typeColor,
                        modifier = Modifier.size(20.dp)
                    )

                    Text(
                        operation.type,
                        color = typeColor,
                        fontSize = 14.sp,
                        fontWeight = FontWeight.Medium
                    )

                    Text(
                        "•",
                        color = typeColor.copy(alpha = 0.5f),
                        fontSize = 12.sp
                    )

                    Text(
                        operation.category,
                        color = typeColor,
                        fontSize = 14.sp
                    )
                }

                // Сумма

```

```

        Row(
            verticalAlignment = Alignment.Bottom,
            horizontalArrangement = Arrange-
ment.spacedBy(4.dp)
        ) {
            Text(
                String.format("%.2f", operation.amount),
                color = AppColors.LightColor,
                fontSize = 24.sp,
                fontWeight = FontWeight.Bold
            )
            Text(
                "₹",
                color = AppColors.LightColor,
                fontSize = 20.sp,
                fontWeight = FontWeight.Medium
            )
        }

        Divider(
            color = typeColor.copy(alpha = 0.15f),
            modifier = Modifier.padding(vertical = 4.dp)
        )

        // Описание (если есть)
        if (!operation.name.isNullOrEmpty()) {
            Row(
                verticalAlignment = Alignment.CenterVer-
tically,
                horizontalArrangement = Arrange-
ment.spacedBy(8.dp)
            ) {
                Text(
                    operation.name,
                    color = AppColors.LightColor,
                    fontSize = 13.sp
                )
            }
        }

        // Дата
        Row(
            verticalAlignment = Alignment.CenterVerti-
cally,
            horizontalArrangement = Arrange-
ment.spacedBy(8.dp)
        ) {
            Icon(
                imageVector = FinlyticsIconPack.Date,
                contentDescription = "date",
                tint = AppColors.LightColor,
                modifier = Modifier.size(14.dp)
            )
            Text(
                operation.date.format(dateFormatter),
                color = AppColors.LightColor,
                fontSize = 12.sp
            )
        }
    }
}

// Предупреждение
Surface(

```



```

        color = AppColors.RedColor.copy(alpha = 0.1f),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Row(
            modifier = Modifier.padding(12.dp),
            horizontalArrangement = Arrange-
ment.spacedBy(8.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                "Это действие нельзя отменить. Операция бу-
дет удалена безвозвратно.",
                color = AppColors.RedColor,
                fontSize = 12.sp
            )
        }
    }
},
confirmButton = {
    Button(
        onClick = {
            viewModel.deleteOperation(operation)
            operationToDelete = null
        },
        colors = ButtonDefaults.buttonColors(background-color =
AppColors.RedColor),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            "Удалить",
            color = AppColors.LightColor,
            fontWeight = FontWeight.Medium
        )
    }
},
dismissButton = {
    TextButton(
        onClick = { operationToDelete = null },
        colors = ButtonDefaults.buttonColors(background-color =
AppColors.LightGreyColor),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Отмена", color = AppColors.LightColor)
    }
},
backgroundColor = AppColors.DarkGreyColor,
shape = RoundedCornerShape(24.dp),
modifier = Modifier.widthIn(max = 800.dp)
)
}
}

/**
 * Фильтрует операции по типу и временному периоду.
 *
 * Сначала применяется фильтрация по типу операции (Все/Доходы/Расходы),
 * затем фильтрация по выбранному временному периоду.
 *
 * @param operations Список всех операций для фильтрации
 * @param filterType Тип фильтра ("Все", "Доходы", "Расходы")

```

```

    * @param period Выбранный период ("День", "Неделя", "Месяц", "Год", "Всё
    время")
    * @param selectedDate Опорная дата для расчёта границ периода
    * @return Отфильтрованный список операций, отсортированный по убыванию даты
    */
private fun filterOperations(
    operations: List<Operation>,
    filterType: String,
    period: String,
    selectedDate: LocalDate
): List<Operation> {
    // Фильтрация по типу операции
    val filteredByType = when (filterType) {
        "Доходы" -> operations.filter { it.type == "Доход" }
        "Расходы" -> operations.filter { it.type == "Расход" }
        else -> operations // "Все"
    }

    // Фильтрация по периоду времени
    return when (period) {
        "День" -> {
            filteredByType.filter { it.date == selectedDate }
        }
        "Неделя" -> {
            val weekStart =
selectedDate.with(TemporalAdjusters.previousOrSame(java.time.DayOfWeek.MONDAY))
            val weekEnd = weekStart.plusDays(6)
            filteredByType.filter {
                val opDate = it.date
                opDate in weekStart..weekEnd
            }
        }
        "Месяц" -> {
            val monthStart = selectedDate.withDayOfMonth(1)
            val monthEnd =
selectedDate.with(TemporalAdjusters.lastDayOfMonth())
            filteredByType.filter {
                val opDate = it.date
                opDate in monthStart..monthEnd
            }
        }
        "Год" -> {
            val yearStart = selectedDate.withDayOfYear(1)
            val yearEnd =
selectedDate.with(TemporalAdjusters.lastDayOfYear())
            filteredByType.filter {
                val opDate = it.date
                opDate in yearStart..yearEnd
            }
        }
        else -> filteredByType
    }.sortedByDescending { it.date }
}

/**
 * Кнопка фильтрации для выбора типа операций.
 *
 * Отображает кнопку с иконкой (для доходов/расходов) или без неё.
 * Цвет кнопки меняется в зависимости от выбранного состояния:
 * - Для "Доходов": зелёный цвет
 * - Для "Расходов": красный цвет
 * - Для "Всех": синий цвет
 * - Невыбранное состояние: серый цвет

```

```

*
* @param text Текст кнопки
* @param isSelected Выбрана ли кнопка в данный момент
* @param modifier Модификатор для настройки внешнего вида
* @param icon Флаг отображения иконки
* @param isAll Флаг, является ли кнопка кнопкой "Все"
* @param isIncome Флаг, является ли кнопка кнопкой доходов (для определения
цвета)
* @param onClick Callback при нажатии на кнопку
*/
@Composable
private fun FilterButton(
    text: String,
    isSelected: Boolean,
    modifier: Modifier = Modifier,
    icon: Boolean = false,
    isAll: Boolean = false,
    isIncome: Boolean = true,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = modifier,
        shape = RoundedCornerShape(10.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (!isSelected) AppColors.LightGreyColor
            else if (isAll) AppColors.BlueColor
            else if (isIncome) AppColors.GreenColor
            else AppColors.RedColor
        ),
        elevation = null
    ) {
        if (icon) {
            Row(
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically
            ) {
                if (isIncome) {
                    Icon(
                        imageVector = FinlyticsIconPack.Income,
                        contentDescription = "income",
                        modifier = Modifier.size(22.dp),
                        tint = AppColors.LightColor
                    )
                } else {
                    Icon(
                        imageVector = FinlyticsIconPack.Expenses,
                        contentDescription = "expenses",
                        modifier = Modifier.size(22.dp),
                        tint = AppColors.LightColor
                    )
                }
                Spacer(Modifier.width(12.dp))
                Text(
                    text,
                    fontSize = 20.sp,
                    fontWeight = FontWeight.Normal,
                    color = AppColors.LightColor
                )
            }
        } else {
            Text(
                text,
                fontSize = 20.sp,

```

```

        fontWeight = FontWeight.Normal,
        color = AppColors.LightColor
    )
}
}

/**
 * Кнопка выбора временного периода.
 *
 * Отображает кнопку с текстом периода. Цвет кнопки меняется на синий при
 * выборе.
 *
 * @param text Текст кнопки (День, Неделя, Месяц, Год, Всё время)
 * @param isSelected Выбрана ли кнопка в данный момент
 * @param onClick Callback при нажатии на кнопку
 */
@Composable
private fun PeriodButton(
    text: String,
    isSelected: Boolean,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = Modifier.defaultMinSize(minWidth = 70.dp),
        shape = RoundedCornerShape(10.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (isSelected) AppColors.BlueColor else
AppColors.LightGreyColor
        ),
        elevation = null
    ) {
        Text(
            text,
            fontSize = 20.sp,
            fontWeight = FontWeight.Normal,
            color = AppColors.LightColor
        )
    }
}

/**
 * Элемент списка транзакций (операций).
 *
 * Отображает детальную информацию о финансовой операции:
 * - Дата слева
 * - Категория с цветной рамкой
 * - Описание (или "Без описания")
 * - Сумма с иконкой плюса/минуса
 * - Кнопки редактирования и удаления
 *
 * @param operation Операция для отображения
 * @param onEditClick Callback при нажатии на кнопку редактирования
 * @param onDeleteClick Callback при нажатии на кнопку удаления
 */
@Composable
private fun TransactionItem(
    operation: Operation,
    onEditClick: () -> Unit,
    onDeleteClick: () -> Unit
) {
    val formatter = DateTimeFormatter.ofPattern("dd.MM.yyyy")
    val dateText = operation.date.format(formatter)

```

```

        val isIncome = operation.type == "Доход"
        val typeColor = if (isIncome) AppColors.GreenColor else
AppColors.RedColor

        Box(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentHeight()
                .padding(bottom = 20.dp)
        ) {
            // Дата слева
            Text(
                text = "[ $dateText ]",
                fontSize = 20.sp,
                fontWeight = FontWeight.Light,
                color = AppColors.LightColor,
                modifier = Modifier
                    .align(Alignment.TopStart)
                    .offset(x = 0.dp, y = 4.dp)
            )

            // Контент операции
            Column(
                modifier = Modifier
                    .width(1120.dp)
                    .align(Alignment.CenterStart)
                    .offset(x = 150.dp)
                    .wrapContentHeight(),
                verticalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                // Категория с цветным фоном
                Box(
                    modifier = Modifier
                        .wrapContentWidth()
                        .height(32.dp)
                        .border(
                            width = 1.dp,
                            color = typeColor,
                            shape = RoundedCornerShape(10.dp)
                        )
                    .background(
                        color = typeColor.copy(alpha = 0.25f),
                        shape = RoundedCornerShape(10.dp)
                    )
                    .padding(horizontal = 10.dp, vertical = 4.dp),
                    contentAlignment = Alignment.Center
                ) {
                    Text(
                        text = operation.category,
                        fontSize = 20.sp,
                        fontWeight = FontWeight.Normal,
                        color = typeColor
                    )
                }

                // Описание
                Text(
                    text = if (operation.name == "" || operation.name == null)
"Без описания" else operation.name,
                    fontSize = 20.sp,
                    fontWeight = FontWeight.Light,
                    color = AppColors.LightColor
                )
            }
        }
    }
}

```

```

        // Сумма
        Row(
            horizontalArrangement = Arrangement.spacedBy(10.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Иконка плюса/минуса
            Box(
                modifier = Modifier.size(14.dp),
                contentAlignment = Alignment.Center
            ) {
                if (isIncome) {
                    Icon(
                        imageVector = FinlyticsIconPack.Plus,
                        contentDescription = "plus",
                        modifier = Modifier.size(14.dp),
                        tint = AppColors.LightColor
                    )
                } else {
                    Icon(
                        imageVector = FinlyticsIconPack.Minus,
                        contentDescription = "minus",
                        modifier = Modifier.size(14.dp),
                        tint = AppColors.LightColor
                    )
                }
            }

            Text(
                text = String.format("%.2f", operation.amount).replace('.', ','),
                fontSize = 32.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )

            // Символ рубля
            Text(
                text = "₽",
                fontSize = 32.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )
        }

        // Кнопки действий справа
        Row(
            modifier = Modifier.align(Alignment.CenterEnd),
            horizontalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            // Кнопка редактирования
            IconButton(
                onClick = onEditClick,
                modifier = Modifier.size(50.dp)
            ) {
                Box(
                    modifier = Modifier
                        .fillMaxSize()
                        .background(AppColors.YellowColor, RoundedCornerShape(10.dp)),
                    contentAlignment = Alignment.Center
                ) {
                    Icon(
                        imageVector = FinlyticsIconPack.Edit,

```



```

* - Удаление категорий с подтверждением
* - Проверка наличия связанных операций перед удалением
*
* @param viewModel ViewModel для управления данными и навигацией
*/
@Composable
fun SettingsScreen(viewModel: FinanceViewModel) {
    val state by viewModel.state.collectAsState()

    // Состояния для диалогов
    var showDeleteConfirmation by remember { mutableStateOf(false) }
    var categoryToDelete by remember { mutableStateOf

```



```

        onDeleteClick = { category -> onDeleteCategory(category,
false) },

        isIncome = false,
        modifier = Modifier.weight(1f)
    )
}

// Навигационная панель
Box(
    modifier = Modifier
        .padding(bottom = 10.dp)
        .align(Alignment.BottomCenter)
) {
    NavigationBar(viewModel)
}

// Диалог добавления категории
if (viewModel.showAddCategoryDialog) {
    AddCategoryDialog(viewModel)
}

// Диалог подтверждения удаления категории
if (showDeleteConfirmation && categoryToDelete != null) {
    val (categoryName, isIncome) = categoryToDelete!!
    val categoryType = if (isIncome) "доходов" else "расходов"
    val categoryColor = if (isIncome) AppColors.GreenColor else
AppColors.RedColor

    // Проверяем, есть ли операции с этой категорией
    val hasOperations = if (isIncome) {
        state.operations.any { it.type == "Доход" && it.category ==
categoryName }
    } else {
        state.operations.any { it.type == "Расход" && it.category ==
categoryName }
    }

    AlertDialog(
        onDismissRequest = {
            showDeleteConfirmation = false
            categoryToDelete = null
        },
        title = {
            Text(
                "Подтверждение удаления",
                color = AppColors.LightColor,
                fontWeight = FontWeight.Medium,
                fontSize = 20.sp
            )
        },
        text = {
            Column(
                verticalArrangement = Arrangement.spacedBy(12.dp)
            ) {
                Text(
                    "Вы уверены, что хотите удалить категорию?",
                    color = AppColors.LightColor,
                    fontSize = 14.sp
                )
            }

            // Выделенная категория
            Surface(

```

```

        color = categoryColor.copy(alpha = 0.15f),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text(
            text = "\"$categoryName\"",
            color = categoryColor,
            fontSize = 18.sp,
            fontWeight = FontWeight.Medium,
            modifier = Modifier.padding(12.dp)
        )
    }

    if (hasOperations) {
        // Предупреждение о наличии операций
        Surface(
            color = AppColors.RedColor.copy(alpha = 0.15f),
            shape = RoundedCornerShape(8.dp)
        ) {
            Column(
                modifier = Modifier.padding(12.dp)
            ) {
                Text(
                    "⚠ Внимание!",
                    color = AppColors.RedColor,
                    fontSize = 14.sp,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(4.dp))
                Text(
                    "Существуют операции с этой категорией."
                    " +
                    "Удаление категории невозможно,
                    пока есть связанные операции.",
                    color = AppColors.RedColor,
                    fontSize = 13.sp
                )
            }
        }
    } else {
        Text(
            "Эта категория не используется ни в одной опера-
            ции и может быть безопасно удалена.",
            color = AppColors.LightColor.copy(alpha = 0.7f),
            fontSize = 13.sp
        )
    }
},
confirmButton = {
    Button(
        onClick = {
            if (!hasOperations) {
                viewModel.deleteCategory(categoryName, isIncome)
            }
            showDeleteConfirmation = false
            categoryToDelete = null
        },
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (hasOperations)
AppColors.LightGreyColor else AppColors.RedColor
        ),
        enabled = !hasOperations,
        shape = RoundedCornerShape(8.dp)
    )
}

```

```

        ) {
            Text(
                "Удалить",
                color = if (hasOperations)
AppColors.LightColor.copy(alpha = 0.5f) else AppColors.LightColor,
                fontWeight = FontWeight.Medium
            )
        }
    },
    dismissButton = {
        TextButton(
            onClick = {
                showDeleteConfirmation = false
                categoryToDelete = null
            },
            colors = ButtonDefaults.buttonColors(background-color =
AppColors.LightGreyColor),
            shape = RoundedCornerShape(8.dp)
        ) {
            Text("Отмена", color = AppColors.LightColor)
        }
    },
    backgroundColor = AppColors.DarkGreyColor,
    shape = RoundedCornerShape(16.dp)
)
}

/**
 * Панель управления категориями определенного типа (доходы или расходы).
 *
 * Отображает заголовок с количеством категорий, кнопку добавления
 * и список категорий с возможностью удаления каждой.
 *
 * @param title Заголовок панели
 * @param count Количество категорий
 * @param categories Список названий категорий
 * @param onAddClick Callback при нажатии на кнопку добавления
 * @param onDeleteClick Callback при удалении категории (передаётся название
категории)
 * @param isIncome Тип категории (для цветового оформления)
 * @param modifier Модификатор для настройки внешнего вида
 */
@Composable
fun CategoryPanel(
    title: String,
    count: Int,
    categories: List<String>,
    onAddClick: () -> Unit,
    onDeleteClick: (String) -> Unit,
    isIncome: Boolean,
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier
            .fillMaxHeight()
            .clip(RoundedCornerShape(40.dp))
            .background(AppColors.DarkGreyColor)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(40.dp)
        ) {

```

```

        // Заголовок панели
        PanelHeader(
            title = title,
            count = count,
            onAddClick = onAddClick
        )

        Spacer(modifier = Modifier.height(25.dp))

        // Список категорий
        if (categories.isEmpty()) {
            Box(
                modifier = Modifier.fillMaxSize(),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    text = "Нет категорий",
                    color = AppColors.LightColor.copy(alpha = 0.5f),
                    fontSize = 16.sp
                )
            }
        } else {
            CategoryList(
                categories = categories,
                onDeleteClick = onDeleteClick,
                isIncome = isIncome
            )
        }
    }
}

/**
 * Заголовок панели категорий.
 *
 * Отображает название панели, счётчик категорий и кнопку добавления.
 *
 * @param title Название панели
 * @param count Количество категорий
 * @param onAddClick Callback при нажатии на кнопку добавления
 */
@Composable
fun PanelHeader(
    title: String,
    count: Int,
    onAddClick: () -> Unit
) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(13.dp)
        ) {
            Text(
                text = title,
                color = AppColors.LightColor,
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                letterSpacing = 0.sp
            )

```

```

        // Счетчик категорий
        Box(
            modifier = Modifier
                .clip(RoundedCornerShape(8.dp))
                .background(AppColors.LightGreyColor)
                .padding(horizontal = 10.dp, vertical = 2.dp)
        ) {
            Text(
                text = count.toString(),
                color = AppColors.LightColor,
                fontSize = 16.sp,
                fontWeight = FontWeight.Medium,
                letterSpacing = 0.sp
            )
        }
    }

    // Кнопка добавления
    AddButton(onClick = onAddClick)
}

/**
 * Кнопка добавления категории.
 *
 * Отображает синюю кнопку с иконкой плюса.
 *
 * @param onClick Callback при нажатии на кнопку
 */
@Composable
fun AddButton(onClick: () -> Unit) {
    Box(
        modifier = Modifier
            .size(30.dp)
            .clip(RoundedCornerShape(8.dp))
            .background(AppColors.BlueColor)
            .clickable(
                interactionSource = remember { MutableInteractionSource() },
                indication = null,
                onClick = onClick
            ),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = FinlyticsIconPack.Add,
            contentDescription = "add",
            modifier = Modifier.size(28.dp),
            tint = AppColors.LightColor
        )
    }
}

/**
 * Список категорий с поддержкой прокрутки.
 *
 * Отображает список категорий с возможностью удаления каждой.
 *
 * @param categories Список названий категорий
 * @param onDeleteClick Callback при удалении категории
 * @param isIncome Тип категории (для определения цвета)
 */
@Composable
fun CategoryList(
    categories: List<String>,

```

```

        onDeleteClick: (String) -> Unit,
        isIncome: Boolean
    ) {
        val lazyListState = rememberLazyListState()
        val categoryColor = if (isIncome) AppColors.GreenColor else
AppColors.RedColor

        LazyColumn(
            state = lazyListState,
            modifier = Modifier.fillMaxHeight(),
            verticalArrangement = Arrangement.spacedBy(15.dp)
        ) {
            items(categories) { category ->
                CategoryItem(
                    category = category,
                    onDeleteClick = { onDeleteClick(category) },
                    categoryColor = categoryColor
                )
            }
        }
    }
}

/**
 * Элемент списка категорий.
 *
 * Отображает название категории и кнопку удаления.
 *
 * @param category Название категории
 * @param onDeleteClick Callback при нажатии на кнопку удаления
 * @param categoryColor Цвет для выделения категории (зелёный для доходов,
красный для расходов)
 */
@Composable
fun CategoryItem(
    category: String,
    onDeleteClick: () -> Unit,
    categoryColor: androidx.compose.ui.graphics.Color
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(50.dp)
    ) {
        // Фон элемента
        Box(
            modifier = Modifier
                .fillMaxSize()
                .clip(RoundedCornerShape(15.dp))
                .background(AppColors.LightGreyColor)
        )

        // Название категории
        Text(
            text = category,
            color = AppColors.LightColor,
            fontSize = 20.sp,
            fontWeight = FontWeight.Normal,
            letterSpacing = 0.sp,
            modifier = Modifier
                .align(Alignment.CenterStart)
                .padding(start = 25.dp)
        )

        // Кнопка удаления (справа)

```

```

        Box(
            modifier = Modifier
                .align(Alignment.CenterEnd)
                .padding(end = 15.dp)
        ) {
            DeleteButton(onClick = onDeleteClick)
        }
    }
}

/**
 * Кнопка удаления категории.
 *
 * Отображает красную кнопку с иконкой корзины.
 *
 * @param onClick Callback при нажатии на кнопку
 */
@Composable
fun DeleteButton(onClick: () -> Unit) {
    Box(
        modifier = Modifier
            .size(28.dp)
            .clip(RoundedCornerShape(8.dp))
            .background(AppColors.RedColor)
            .clickable(
                interactionSource = remember { MutableInteractionSource() },
                indication = null,
                onClick = onClick
            ),
        contentAlignment = Alignment.Center
    ) {
        Icon(
            imageVector = FinlyticsIconPack.Delete,
            contentDescription = "delete",
            modifier = Modifier.size(28.dp),
            tint = AppColors.LightColor
        )
    }
}

```

3.5.1.2.3. Компоненты интерфейса:

Листинг 9 – PieChart.kt – Компонент круговой диаграммы

```

package ui.components

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.*
import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.*
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlin.math.*
import ui.theme.AppColors

/**

```

```

    * Компонент для отображения круговой диаграммы распределения расходов по
    категориям.
    * Отображает сегменты, пропорциональные суммам расходов в каждой категории.
    * В центре диаграммы расположено пустое пространство (donut-стиль).
    *
    * @param data Карта данных, где ключ - название категории, значение - сумма
    расходов
    * @param modifier Модификатор для настройки внешнего вида компонента
    * @param colors Список цветов для сегментов диаграммы (по умолчанию исполь-
    зует палитру из дизайна)
    */
@Composable
fun PieChart(
    data: Map<String, Double>,
    modifier: Modifier = Modifier,
    colors: List<Color> = listOf(
        AppColors.IncomeColor1,
        AppColors.IncomeColor2,
        AppColors.IncomeColor3,
        AppColors.IncomeColor4,
        AppColors.IncomeColor5,
        AppColors.IncomeColor6,
        AppColors.IncomeColor7,
        AppColors.IncomeColor8,
        AppColors.ExpensesColor1,
        AppColors.ExpensesColor2,
        AppColors.ExpensesColor3,
        AppColors.ExpensesColor4,
        AppColors.ExpensesColor5,
        AppColors.ExpensesColor6,
        AppColors.ExpensesColor7,
        AppColors.ExpensesColor8
    )
) {
    println("\n=== PIE CHART ===")
    println("Полученные данные: $data")
    println("Количество категорий: ${data.size}")
    println("Данные не пустые: ${data.isNotEmpty()}")

    // Фильтруем данные, оставляя только категории с положительными значени-
    ями
    val filteredData = data.filterValues { it > 0 }
        .toList()
        .sortedByDescending { it.second }
    val total = filteredData.sumOf { it.second }

    println("Отфильтрованные данные: $filteredData")
    println("Общая сумма расходов: $total")

    if (filteredData.isNotEmpty()) {
        println("Детализация по категориям:")
        filteredData.forEach { (category, amount) ->
            val percentage = (amount / total * 100)
            println("    - $category: $amount руб.
            (${("%.1f".format(percentage))}%)")
        }
    }
    println("===== \n")

    // Если данных нет, показываем сообщение
    if (total == 0.0 || filteredData.isEmpty()) {
        Box(
            modifier = modifier,
            contentAlignment = Alignment.Center
        )
    }
}

```



```

    ) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            // Иконка пустой диаграммы (круг)
            Canvas(
                modifier = Modifier.size(100.dp)
            ) {
                drawCircle(
                    color = Color.LightGray.copy(alpha = 0.3f),
                    style = Stroke(width = 4f)
                )
            }
            Text(
                "Нет данных по расходам",
                color = Color.LightGray.copy(alpha = 0.8f),
                fontSize = 16.sp
            )
        }
    }
    return
}

Box(
    modifier = modifier,
    contentAlignment = Alignment.Center
) {
    // Основная круговая диаграмма
    Canvas(
        modifier = Modifier
            .fillMaxSize()
            .aspectRatio(1f)
    ) {
        val canvasSize = min(size.width, size.height)
        val radius = canvasSize / 2 - 20
        val centerOffset = Offset(size.width / 2, size.height / 2)

        // Вычисляем углы для каждого сегмента
        var startAngle = -90f

        filteredData.forEachIndexed { index, (_, amount) ->
            val sweepAngle = (amount / total * 360).toFloat()

            // Рисуем сегмент
            drawArc(
                color = colors.getOrNull(index) { colors.get(8 - index)

                startAngle = startAngle,
                sweepAngle = sweepAngle,
                useCenter = true,
                topLeft = Offset(
                    centerOffset.x - radius,
                    centerOffset.y - radius
                ),
                size = Size(radius * 2, radius * 2)
            )

            startAngle += sweepAngle
        }

        // Рисуем центральное отверстие (donut-стиль)
        val centerCircleRadius = radius * 0.9f
    }
}

```

```

        drawCircle(
            color = AppColors.DarkColor,
            center = centerOffset,
            radius = centerCircleRadius
        )
    }
}
}

```

Листинг 10 – OperationDialog.kt – Диалог операций

```

package ui.components

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.runtime.LaunchedEffect
import models.Operation
import viewmodel.FinanceViewModel
import java.time.LocalDate
import java.time.format.DateTimeParseException
import ui.theme.AppColors
import androidx.compose.foundation.background
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.draw.clip
import ui.theme.icons.FinlyticsIconPack
import ui.theme.icons.finlyticsiconpack.*

/**
 * Диалоговое окно для добавления или редактирования финансовой операции.
 * Поддерживает ввод всех параметров операции: тип, сумму, описание, категорию и дату.
 *
 * Функциональность:
 * - Выбор типа операции (Доход/Расход) с цветовой индикацией
 * - Ввод суммы с валидацией (положительное число)
 * - Выбор даты с быстрыми кнопками (Сегодня/Вчера/Завтра)
 * - Выбор категории из списка (динамически обновляется при изменении типа)
 * - Ввод описания (необязательно)
 * - Валидация всех полей перед сохранением
 *
 * Валидация:
 * - Сумма: положительное число, не может быть пустым или нулевым
 * - Категория: обязательно должна быть выбрана
 * - Дата: корректный формат ГГГГ-ММ-ДД, не более чем на 1 день в будущее
 *
 * @param viewModel ViewModel для управления операциями
 */
@Composable
fun OperationDialog(viewModel: FinanceViewModel) {
    val state by viewModel.state.collectAsState()

    // Отладочная информация о редактируемой операции
    LaunchedEffect(viewModel.editingOperation) {

```

```

        println("\n=== OPERATION DIALOG ===")
        println("Режим: ${if (viewModel.editingOperation == null) "Добавле-
ние" else "Редактирование"}")
        println("Операция для редактирования:
${viewModel.editingOperation}")
        println("===== \n")
    }

    var type by remember { mutableStateOf(viewModel.editingOperation?.type
?: "Расход") }
    var amount by remember {
mutableStateOf(viewModel.editingOperation?.amount?.toString() ?: "") }
    var name by remember { mutableStateOf(viewModel.editingOperation?.name
?: "") }
    var category by remember {
mutableStateOf(viewModel.editingOperation?.category ?: "") }
    var dateText by remember {
mutableStateOf(viewModel.editingOperation?.date?.toString() ?:
LocalDate.now().toString()) }
    var dateError by remember { mutableStateOf("") }
    var amountError by remember { mutableStateOf("") }
    var categoryError by remember { mutableStateOf("") }

    val categories = if (type == "Доход") state.incomeCategories else
state.expenseCategories
    val isIncome = type == "Доход"
    val typeColor = if (isIncome) AppColors.GreenColor else
AppColors.RedColor

    // Преобразуем текст в LocalDate
    val selectedDate = remember(dateText) {
        try {
            LocalDate.parse(dateText)
        } catch (e: DateTimeParseException) {
            null
        }
    }

    AlertDialog(
        onDismissRequest = {
            viewModel.hideOperationDialog()
            amountError = ""
            categoryError = ""
            dateError = ""
        },
        title = {
            Text(
                if (viewModel.editingOperation == null) "Новая операция"
else "Редактирование операции",
                color = AppColors.LightColor,
                fontWeight = FontWeight.Medium,
                fontSize = 20.sp
            )
        },
        text = {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                verticalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                // Выбор типа операции (Доход/Расход)
                Column(
                    verticalArrangement = Arrangement.spacedBy(8.dp)

```

```

    ) {
        Text(
            "Тип операции",
            color = AppColors.LightColor,
            fontSize = 14.sp,
            fontWeight = FontWeight.Medium
        )

        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // Кнопка "Доход"
            TypeButton(
                text = "Доход",
                isSelected = type == "Доход",
                color = AppColors.GreenColor,
                modifier = Modifier.weight(1f),
                onClick = {
                    type = "Доход"
                    amountError = ""
                    categoryError = ""
                }
            )

            // Кнопка "Расход"
            TypeButton(
                text = "Расход",
                isSelected = type == "Расход",
                color = AppColors.RedColor,
                modifier = Modifier.weight(1f),
                onClick = {
                    type = "Расход"
                    amountError = ""
                    categoryError = ""
                }
            )
        }
    }

    // Сумма и дата в одной строке
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        // Поле для ввода суммы
        Column(
            modifier = Modifier.weight(1f),
            verticalArrangement = Arrangement.spacedBy(4.dp)
        ) {
            Text(
                "Сумма (₽)",
                color = AppColors.LightColor,
                fontSize = 14.sp,
                fontWeight = FontWeight.Medium
            )

            OutlinedTextField(
                value = amount,
                onValueChange = {
                    amount = it
                    amountError = ""
                },
                placeholder = {

```

```

        Text(
            "0.00",
            color = AppColors.LightColor.copy(alpha
= 0.5f)
        )
    },
    modifier = Modifier.fillMaxWidth(),
    isError = amountError.isNotEmpty(),
    singleLine = true,
    colors = TextFieldDefaults.outlinedTextFieldCol-
ors(
        focusedBorderColor = typeColor,
        unfocusedBorderColor = AppColors.Light-
GreyColor,

        cursorColor = typeColor,
        textColor = AppColors.LightColor
    )
)
}

// Поле для ввода даты
Column(
    modifier = Modifier.weight(1f),
    verticalArrangement = Arrangement.spacedBy(4.dp)
) {
    Text(
        "Дата",
        color = AppColors.LightColor,
        fontSize = 14.sp,
        fontWeight = FontWeight.Medium
    )

    OutlinedTextField(
        value = dateText,
        onValueChange = {
            dateText = it
            dateError = ""
            try {
                LocalDate.parse(it)
                dateError = ""
            } catch (e: DateTimeParseException) {
                if (it.isNotEmpty() && it.length > 5) {
                    dateError = "Формат: ГГГГ-ММ-ДД"
                }
            }
        },
        placeholder = {
            Text(
                "ГГГГ-ММ-ДД",
                color = AppColors.LightColor.copy(alpha
= 0.5f)
            )
        },
        modifier = Modifier.fillMaxWidth(),
        isError = dateError.isNotEmpty(),
        colors = TextFieldDefaults.outlinedTextFieldCol-
ors(
            focusedBorderColor = typeColor,
            unfocusedBorderColor = AppColors.Light-
GreyColor,

            cursorColor = typeColor,
            textColor = AppColors.LightColor
        )
    )
}

```

```

    }
}

// Отображение ошибок суммы и даты в одной строке
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(12.dp)
) {
    // Ошибка суммы
    if (amountError.isNotEmpty()) {
        Surface(
            color = AppColors.RedColor.copy(alpha = 0.15f),
            shape = RoundedCornerShape(8.dp),
            modifier = Modifier.weight(1f)
        ) {
            Text(
                text = amountError,
                color = AppColors.RedColor,
                fontSize = 11.sp,
                modifier = Modifier.padding(6.dp)
            )
        }
    } else {
        Spacer(modifier = Modifier.weight(1f))
    }

    // Ошибка даты
    if (dateError.isNotEmpty()) {
        Surface(
            color = AppColors.RedColor.copy(alpha = 0.15f),
            shape = RoundedCornerShape(8.dp),
            modifier = Modifier.weight(1f)
        ) {
            Text(
                text = dateError,
                color = AppColors.RedColor,
                fontSize = 11.sp,
                modifier = Modifier.padding(6.dp)
            )
        }
    } else {
        Spacer(modifier = Modifier.weight(1f))
    }
}

// Быстрые кнопки для выбора дат
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    listOf("Сегодня", "Вчера", "Завтра").forEach { label ->
        QuickDateButton(
            label = label,
            isSelected = when (label) {
                "Сегодня" -> selectedDate == LocalDate.now()
                "Вчера" -> selectedDate == Local-
Date.now().minusDays(1)
                "Завтра" -> selectedDate == Local-
Date.now().plusDays(1)
                else -> false
            },
            onClick = {
                val newDate = when (label) {
                    "Сегодня" -> LocalDate.now()

```

```

        "Вчера" -> LocalDate.now().minusDays(1)
        "Завтра" -> LocalDate.now().plusDays(1)
        else -> LocalDate.now()
    }
    dateText = newDate.toString()
    dateError = ""
},
modifier = Modifier.weight(1f)
)
}
}

// Поле для ввода описания
Column(
    verticalArrangement = Arrangement.spacedBy(4.dp)
) {
    Text(
        "Описание",
        color = AppColors.LightColor,
        fontSize = 14.sp,
        fontWeight = FontWeight.Medium
    )

    OutlinedTextField(
        value = name,
        onValueChange = { name = it },
        placeholder = {
            Text(
                "Необязательно",
                color = AppColors.LightColor.copy(alpha =
0.5f)
            )
        },
        modifier = Modifier.fillMaxWidth(),
        colors = TextFieldDefaults.outlinedTextFieldColors(
            focusedBorderColor = typeColor,
            unfocusedBorderColor = AppColors.LightGreyColor,
            cursorColor = typeColor,
            textColor = AppColors.LightColor
        )
    )
}

// Выбор категории
Column(
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    Text(
        "Категория",
        color = AppColors.LightColor,
        fontSize = 14.sp,
        fontWeight = FontWeight.Medium
    )

    if (categories.isNotEmpty()) {
        // Список категорий
        LazyColumn(
            modifier = Modifier
                .fillMaxWidth()
                .heightIn(max = 180.dp)
                .clip(RoundedCornerShape(12.dp))
                .background(AppColors.LightGreyColor),
            verticalArrangement = Arrangement.spacedBy(4.dp)
        ) {

```

```

        items(categories) { item ->
            CategoryItem(
                name = item,
                isSelected = category == item,
                onClick = {
                    category = item
                    categoryError = ""
                },
                color = typeColor
            )
        }
    }

    if (categoryError.isNotEmpty()) {
        Surface(
            color = AppColors.RedColor.copy(alpha =
0.15f),

            shape = RoundedCornerShape(8.dp),
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(
                text = categoryError,
                color = AppColors.RedColor,
                fontSize = 12.sp,
                modifier = Modifier.padding(8.dp)
            )
        }
    }
} else {
    // Если категорий нет
    Surface(
        color = AppColors.RedColor.copy(alpha = 0.15f),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier.padding(12.dp),
            horizontalAlignment =
Alignment.CenterHorizontally
        ) {
            Text(
                "⚠ Нет доступных категорий",
                color = AppColors.RedColor,
                fontSize = 13.sp,
                fontWeight = FontWeight.Medium
            )
            Spacer(modifier = Modifier.height(8.dp))
            Button(
                onClick = {
                    viewModel.showAddCategory(type ==
"Доход")
                },
                colors =
ButtonDefaults.buttonColors(backgroundColor = AppColors.BlueColor),
                shape = RoundedCornerShape(8.dp)
            ) {
                Text(
                    "Добавить категорию",
                    color = AppColors.LightColor,
                    fontSize = 13.sp
                )
            }
        }
    }
}
}

```



```

    }
    },
    confirmButton = {
        Button(
            onClick = {
                // Валидация суммы
                val sum = amount.toDoubleOrNull()
                if (sum == null || sum <= 0) {
                    amountError = "Введите сумму > 0"
                    return@Button
                }

                // Валидация категории
                if (category.isEmpty()) {
                    categoryError = "Выберите категорию"
                    return@Button
                }

                // Валидация даты
                val parsedDate = try {
                    LocalDate.parse(dateText)
                } catch (e: DateTimeParseException) {
                    dateError = "Неверный формат даты"
                    return@Button
                }

                // Дополнительная проверка: дата не должна быть слишком
далеко в будущем
                if (parsedDate.isAfter(LocalDate.now().plusDays(1))) {
                    dateError = "Дата не может быть более чем на 1 день
в будущем"

                    return@Button
                }

                val op = Operation(
                    id = viewModel.editingOperation?.id ?: 0,
                    type = type,
                    amount = sum,
                    category = category,
                    date = parsedDate,
                    name = name.ifBlank { null }
                )

                viewModel.saveOperation(op)
            },
            colors = ButtonDefaults.buttonColors(backgroundColor =
typeColor),
            enabled = amount.isNotEmpty() && category.isNotEmpty() &&
dateText.isNotEmpty() && selectedDate != null,
            shape = RoundedCornerShape(8.dp),
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(
                if (viewModel.editingOperation == null) "Создать опера-
цию" else "Сохранить изменения",
                color = AppColors.LightColor,
                fontWeight = FontWeight.Medium
            )
        }
    },
    dismissButton = {
        TextButton(

```

```

        onClick = {
            viewModel.hideOperationDialog()
            amountError = ""
            categoryError = ""
            dateError = ""
        },
        colors = ButtonDefaults.buttonColors(backgroundColor =
AppColors.LightGreyColor),
        shape = RoundedCornerShape(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Отмена", color = AppColors.LightColor)
    }
},
backgroundColor = AppColors.DarkGreyColor,
shape = RoundedCornerShape(24.dp),
modifier = Modifier.widthIn(max = 800.dp)
)
}

/**
 * Кнопка выбора типа операции (Доход/Расход).
 *
 * @param text Текст кнопки ("Доход" или "Расход")
 * @param isSelected Выбрана ли кнопка в данный момент
 * @param color Цвет кнопки при выборе (зелёный для доходов, красный для
расходов)
 * @param modifier Модификатор для настройки внешнего вида
 * @param onClick Callback при нажатии на кнопку
 */
@Composable
private fun TypeButton(
    text: String,
    isSelected: Boolean,
    color: Color,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = modifier.height(48.dp),
        shape = RoundedCornerShape(10.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (isSelected) color else
AppColors.LightGreyColor
        ),
        elevation = null
    ) {
        Text(
            text,
            color = if (isSelected) AppColors.LightColor else
AppColors.LightColor,
            fontSize = 16.sp,
            fontWeight = if (isSelected) FontWeight.Medium else
FontWeight.Normal
        )
    }
}

/**
 * Кнопка быстрого выбора даты (Сегодня, Вчера, Завтра).
 *
 * @param label Текст кнопки
 * @param isSelected Выбрана ли дата, соответствующая кнопке

```

```

    * @param onClick Callback при нажатии на кнопку
    * @param modifier Модификатор для настройки внешнего вида
    */
@Composable
private fun QuickDateButton(
    label: String,
    isSelected: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Button(
        onClick = onClick,
        modifier = modifier.height(36.dp),
        shape = RoundedCornerShape(8.dp),
        colors = ButtonDefaults.buttonColors(
            backgroundColor = if (isSelected) AppColors.BlueColor else
AppColors.LightGreyColor
        ),
        elevation = null
    ) {
        Text(
            label,
            color = AppColors.LightColor,
            fontSize = 13.sp,
            fontWeight = if (isSelected) FontWeight.Medium else
FontWeight.Normal
        )
    }
}

/**
 * Элемент списка категорий.
 *
 * Отображает название категории и иконку выбора при выделении.
 *
 * @param name Название категории
 * @param isSelected Выбрана ли категория
 * @param onClick Callback при нажатии на элемент
 * @param color Цвет выделения (зелёный для доходов, красный для расходов)
 */
@Composable
private fun CategoryItem(
    name: String,
    isSelected: Boolean,
    onClick: () -> Unit,
    color: Color
) {
    Surface(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        shape = RoundedCornerShape(8.dp),
        color = if (isSelected) color.copy(alpha = 0.2f) else
Color.Transparent
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 12.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = name,

```

```

        color = if (isSelected) color else AppColors.LightColor,
        fontSize = 14.sp,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
        modifier = Modifier.weight(1f)
    )

    if (isSelected) {
        Icon(
            imageVector = FinlyticsIconPack.Check,
            contentDescription = "selected",
            tint = color,
            modifier = Modifier.size(18.dp)
        )
    }
}
}
}

```

Листинг 11 – *AddCategoryDialog.kt* – Диалог добавления категорий

```

package ui.components

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import ui.theme.AppColors
import viewmodel.FinanceViewModel

/**
 * Диалоговое окно для добавления новой категории (доходов или расходов).
 *
 * Позволяет пользователю ввести название новой категории.
 *
 * Функциональность:
 * - Ввод названия категории с валидацией (не может быть пустым)
 * - Автоматическая проверка на пустое название
 * - Цветовое оформление в зависимости от типа категории (зелёный для дохо-
 * дов, красный для расходов)
 * - Сохранение категории через ViewModel
 *
 * @param viewModel ViewModel для управления категориями (содержит информа-
 * цию о типе добавляемой категории)
 */
@Composable
fun AddCategoryDialog(viewModel: FinanceViewModel) {
    var categoryName by remember { mutableStateOf("") }
    var error by remember { mutableStateOf("") }

    // Определяем тип категории для отображения в заголовке
    val categoryType = if (viewModel.isIncomeCategory) "доходов" else "рас-
    ходов"
    val categoryColor = if (viewModel.isIncomeCategory) AppColors.GreenColor
    else AppColors.RedColor

    AlertDialog(
        onDismissRequest = {

```

```

        viewModel.hideCategoryDialog()
        categoryName = ""
        error = ""
    },
    title = {
        Text(
            "Добавить категорию $categoryType",
            color = AppColors.LightColor,
            fontWeight = FontWeight.Medium,
            fontSize = 20.sp
        )
    },
    text = {
        Column(
            modifier = Modifier.fillMaxWidth(),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            // Пояснительный текст
            Text(
                "Введите название новой категории:",
                color = AppColors.LightColor.copy(alpha = 0.8f),
                fontSize = 14.sp
            )

            // Поле ввода
            OutlinedTextField(
                value = categoryName,
                onValueChange = {
                    categoryName = it
                    error = ""
                },
                label = {
                    Text(
                        "Название категории",
                        color = if (error.isNotEmpty())
AppColors.RedColor else AppColors.LightColor.copy(alpha = 0.7f)
                    )
                },
                modifier = Modifier.fillMaxWidth(),
                singleLine = true,
                isError = error.isNotEmpty(),
                colors = TextFieldDefaults.outlinedTextFieldColors(
                    focusedBorderColor = categoryColor,
                    unfocusedBorderColor = AppColors.LightGreyColor,
                    cursorColor = categoryColor,
                    textColor = AppColors.LightColor
                )
            )

            // Сообщение об ошибке
            if (error.isNotEmpty()) {
                Surface(
                    color = AppColors.RedColor.copy(alpha = 0.15f),
                    shape = RoundedCornerShape(8.dp)
                ) {
                    Text(
                        text = error,
                        color = AppColors.RedColor,
                        fontSize = 13.sp,
                        modifier = Modifier.padding(8.dp)
                    )
                }
            }
        }
    }
}

```

```

        // Подсказка
        Text(
            "Название должно быть уникальным и не может быть пу-
стым",
            color = AppColors.LightColor.copy(alpha = 0.5f),
            fontSize = 11.sp
        )
    },
    confirmButton = {
        Button(
            onClick = {
                // Валидация: проверяем, что название не пустое
                if (categoryName.trim().isEmpty()) {
                    error = "Введите название категории"
                    return@Button
                }

                // Сохраняем категорию через ViewModel
                viewModel.saveCategory(categoryName.trim())
                categoryName = ""
            },
            colors = ButtonDefaults.buttonColors(backgroundColor =
categoryColor),
            enabled = categoryName.isNotEmpty(),
            shape = RoundedCornerShape(8.dp)
        ) {
            Text(
                "Добавить",
                color = AppColors.LightColor,
                fontWeight = FontWeight.Medium
            )
        }
    },
    dismissButton = {
        TextButton(
            onClick = {
                viewModel.hideCategoryDialog()
                categoryName = ""
                error = ""
            },
            colors = ButtonDefaults.buttonColors(backgroundColor =
AppColors.LightGreyColor),
            shape = RoundedCornerShape(8.dp)
        ) {
            Text(
                "Отмена",
                color = AppColors.LightColor
            )
        }
    },
    backgroundColor = AppColors.DarkGreyColor,
    shape = RoundedCornerShape(16.dp)
)
}

```

Листинг 12 – *GhostRaceProgressBar.kt* – Компонент реализации призрака

```

package ui.components

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.Surface

```

```

import androidx.compose.material.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import ui.theme.AppColors

/**
 * Компонент прогресс-бара для функционала "Тонка с призраком".
 * Отображает сравнение расходов текущего выбранного периода с расходами за
 * аналогичный предыдущий период.
 *
 * Концепция "Тонка с призраком" помогает пользователю отслеживать,
 * насколько изменились расходы
 * по сравнению с предыдущим аналогичным периодом (день назад, неделю назад,
 * месяц назад или год назад).
 *
 * Прогресс-бар заполняется в зависимости от отношения текущих расходов к
 * расходам предыдущего периода.
 *
 * Цветовая индикация:
 * - Зелёный: менее 80% от лимита (расходы снизились значительно)
 * - Жёлтый: от 80% до 100% (расходы близки к уровню предыдущего периода)
 * - Красный: более 100% (расходы превысили уровень предыдущего периода)
 *
 * Визуально компонент содержит:
 * - Заголовок "Тонка с призраком" и текущий процент
 * - Горизонтальный прогресс-бар с числовой меткой посередине
 * - Строку с суммами: потрачено и лимит прошлого периода
 *
 * @param currentExpenses Сумма расходов за текущий выбранный период
 * @param previousExpenses Сумма расходов за аналогичный предыдущий период
 * (лимит)
 * @param percent Отношение текущих расходов к предыдущим (currentExpenses /
 * previousExpenses)
 * @param modifier Модификатор для настройки внешнего вида и расположения
 * компонента
 */
@Composable
fun GhostRaceProgressBar(
    currentExpenses: Double,
    previousExpenses: Double,
    percent: Double,
    modifier: Modifier = Modifier
) {
    // Цвет прогресса в зависимости от процента выполнения
    val progressColor = when {
        percent < 0.8 -> AppColors.GreenColor
        percent <= 1.0 -> AppColors.YellowColor
        else -> AppColors.RedColor
    }

    // Заполнение не может превышать 100%
    val fillFraction = if (percent > 1.0) 1.0f else percent.toFloat()

    Surface(
        color = AppColors.DarkGreyColor,
        shape = RoundedCornerShape(20.dp),
        modifier = modifier
    ) {
        Column(
            modifier = Modifier

```

```

        .fillMaxWidth()
        .padding(20.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        // Заголовок и процент
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = "Гонка с призраком",
                fontSize = 24.sp,
                fontWeight = FontWeight.Medium,
                color = AppColors.LightColor
            )
            Text(
                text = String.format("%.1f%", percent * 100),
                fontSize = 24.sp,
                fontWeight = FontWeight.Bold,
                color = progressColor
            )
        }

        // Прогресс-бар с меткой посередине
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(40.dp)
                .clip(RoundedCornerShape(20.dp))
                .background(AppColors.LightGreyColor.copy(alpha = 0.3f))
        ) {
            // Заполненная часть
            Box(
                modifier = Modifier
                    .fillMaxWidth(fraction = fillFraction)
                    .fillMaxHeight()
                    .background(progressColor)
            )
        }

        // Строка с суммами
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text(
                text = "Потрачено: ${String.format("%.2f",
currentExpenses)} ₸",
                fontSize = 16.sp,
                color = progressColor,
                fontWeight = FontWeight.Medium
            )
            Text(
                text = "Лимит прошлого периода: ${String.format("%.2f",
previousExpenses)} ₸",
                fontSize = 16.sp,
                color = AppColors.LightColor
            )
        }
    }
}
}

```


3.5.1.2.4. Состояние и управление:

Листинг 13 – *FinanceState.kt* – Состояние приложения

```
package viewmodel

import models.Operation

/**
 * Класс состояния приложения, содержащий все данные, необходимые для отображения UI.
 * Состояние является иммутабельным (immutable) и обновляется через Flow в архитектуре MVVM.
 *
 * @property operations Список всех финансовых операций (доходы и расходы)
 * @property totalIncome Общая сумма всех доходов за выбранный период
 * @property totalExpenses Общая сумма всех расходов за выбранный период
 * @property balance Текущий баланс (доходы минус расходы)
 * @property expensesByCategory Распределение расходов по категориям для построения диаграмм
 * @property incomeCategories Список доступных категорий доходов
 * @property expenseCategories Список доступных категорий расходов
 * @property editingCategory Категория, находящаяся в процессе редактирования (в текущей реализации не используется)
 */
data class FinanceState(
    val operations: List<Operation> = emptyList(),
    val totalIncome: Double = 0.0,
    val totalExpenses: Double = 0.0,
    val balance: Double = 0.0,
    val expensesByCategory: Map<String, Double> = emptyMap(),
    val incomeCategories: List<String> = emptyList(),
    val expenseCategories: List<String> = emptyList(),
    val editingCategory: String? = null
)
```

Листинг 14 – *FinanceViewModel.kt* – *ViewModel* приложения

```
package viewmodel

import androidx.compose.runtime.*
import java.text.DecimalFormat
import java.text.DecimalFormatSymbols
import java.util.Locale
import kotlin.math.roundToInt
import kotlin.math.pow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import models.Operation
import repository.FinanceRepository

/**
 * ViewModel для управления состоянием приложения и бизнес-логикой.
 * Следует архитектуре MVVM, обеспечивает реактивное обновление UI.
 *
 * @param repo Репозиторий для доступа к данным финансовых операций и категорий
 */
class FinanceViewModel(private val repo: FinanceRepository) {
    // Создаем DecimalFormat с явным указанием локали для корректного
```

```

парсинга
    private val decimalFormat = DecimalFormat("#.##",
DecimalFormatSymbols(Locale.US))

    /**
     * Хранит текущее состояние приложения в виде реактивного потока.
     * Используется для обновления UI при изменении данных.
     */
    private val _state = MutableStateFlow(FinanceState())
    val state: StateFlow<FinanceState> = _state.asStateFlow()

    /**
     * Текущий экран приложения.
     * Возможные значения: "Overview", "History", "Settings".
     */
    var currentScreen by mutableStateOf("Overview")
    private set

    /** Флаг отображения диалога добавления/редактирования операции. */
    var showOperationDialog by mutableStateOf(false)
    private set

    /** Флаг отображения диалога добавления категории. */
    var showAddCategoryDialog by mutableStateOf(false)
    private set

    /** Операция, находящаяся в процессе редактирования. null - режим созда-
ния новой операции. */
    var editingOperation by mutableStateOf<Operation?>(null)
    private set

    /** Флаг, указывающий тип добавляемой категории: true - доходы, false -
расходы. */
    var isIncomeCategory by mutableStateOf(true)
    private set

    /**
     * Инициализация репозитория и загрузка начальных данных.
     * Вызывается при создании ViewModel.
     */
    init {
        refresh()
    }

    /**
     * Пересчитывает статистику на основе списка операций.
     *
     * Вычисляет общую сумму доходов и расходов, баланс,
     * а также группирует расходы по категориям для построения диаграмм.
     *
     * @param operations Список операций для анализа
     */
    private fun calculateStatistics(operations: List<Operation>) {
        // Суммируем доходы и расходы
        val income = operations.filter { it.type == "Доход" }.sumOf {
it.amount }
        val expenses = operations.filter { it.type == "Расход" }.sumOf {
it.amount }

        // Группируем расходы по категориям для диаграммы
        val expenseOperations = operations.filter { it.type == "Расход" }
        val expByCat = expenseOperations
            .groupBy { it.category }
            .mapValues { entry ->

```

```

        val sum = entry.value.sumOf { op -> op.amount }
        sum
    }

    // Подробная отладочная информация
    println("\n=== CALCULATE STATISTICS ===")
    println("Всего операций: ${operations.size}")
    println("Операций доходов: ${operations.count { it.type == "Доход"
}}}"))

    println("Операций расходов: ${expenseOperations.size}")
    println("Общая сумма доходов: $income руб.")
    println("Общая сумма расходов: $expenses руб.")
    println("Баланс: ${income - expenses} руб.")

    if (expByCat.isNotEmpty()) {
        println("Детализация расходов по категориям:")
        expByCat.forEach { (category, amount) ->
            println("    - $category: $amount руб.")
        }
    } else {
        println("Расходы по категориям: нет данных")
    }
    println("=====\\n")

    _state.value = _state.value.copy(
        operations = operations,
        totalIncome = income,
        totalExpenses = expenses,
        balance = income - expenses,
        expensesByCategory = expByCat,
        incomeCategories = repo.getIncomeCategories(),
        expenseCategories = repo.getExpenseCategories()
    )
}

/**
 * Обновляет данные из репозитория и пересчитывает статистику.
 * Вызывается при изменении данных (добавление, редактирование, удаление
операций или категорий).
 */
fun refresh() {
    println("Обновление данных...")
    val allOps = repo.getAllOperations()
    println("Операций загружено: ${allOps.size}")
    calculateStatistics(allOps)
}

/**
 * Открывает диалог добавления новой операции.
 * Сбрасывает editingOperation в null для режима создания.
 */
fun showAddOperation() {
    println("Показать диалог добавления операции")
    editingOperation = null
    showOperationDialog = true
}

/**
 * Открывает диалог редактирования существующей операции.
 *
 * @param op Операция для редактирования
 */
fun showEditOperation(op: Operation) {
    println("Показать диалог редактирования операции: $op")
}

```

```

        editingOperation = op
        showOperationDialog = true
    }

    /**
     * Закрывает диалог операции и сбрасывает связанные состояния.
     */
    fun hideOperationDialog() {
        println("Скрыть диалог операции")
        showOperationDialog = false
        editingOperation = null
    }

    /**
     * Сохраняет операцию: добавляет новую или обновляет существующую.
     * Автоматически обновляет данные приложения после сохранения.
     *
     * @param op Операция для сохранения (при id == 0 - добавление, иначе -
    обновление)
     */
    fun saveOperation(op: Operation) {
        println("Сохранение операции: $op")
        try {
            if (op.id == 0) {
                repo.addOperation(op)
            } else {
                repo.updateOperation(op)
            }
            refresh()
            hideOperationDialog()
        } catch (e: Exception) {
            println("Ошибка при сохранении операции: ${e.message}")
        }
    }

    /**
     * Удаляет операцию из базы данных.
     *
     * @param op Операция для удаления
     */
    fun deleteOperation(op: Operation) {
        println("Удаление операции: $op")
        try {
            repo.deleteOperation(op.id, op.type)
            refresh()
        } catch (e: Exception) {
            println("Ошибка при удалении операции: ${e.message}")
        }
    }

    /**
     * Открывает диалог добавления новой категории.
     *
     * @param isIncome Тип добавляемой категории (true - доходы, false -
    расходы)
     */
    fun showAddCategory(isIncome: Boolean) {
        println("Показать диалог добавления категории. Тип: ${if (isIncome)
"доходы" else "расходы"}")
        isIncomeCategory = isIncome
        showAddCategoryDialog = true
    }

    /**

```

```

    * Закрывает диалог добавления категории.
    */
fun hideCategoryDialog() {
    println("Скрыть диалог категории")
    showAddCategoryDialog = false
}

/**
 * Сохраняет новую категорию в базу данных.
 * Проверяет уникальность имени категории перед добавлением.
 *
 * @param name Название новой категории
 */
fun saveCategory(name: String) {
    println("Сохранение категории: '$name'. Тип: ${if (isIncomeCategory)
"доходы" else "расходы"}")
    try {
        if (name.isBlank()) {
            println("Имя категории пустое")
            return
        }

        val trimmedName = name.trim()

        // Проверяем уникальность категории
        val existingCategories = if (isIncomeCategory)
            state.value.incomeCategories
        else
            state.value.expenseCategories

        if (existingCategories.any { it.equals(trimmedName, ignoreCase =
true) }) {
            println("Категория '$trimmedName' уже существует")
            return
        }

        val success = repo.addCategory(trimmedName, isIncomeCategory)
        println("Категория добавлена. Успех: $success")

        if (success) {
            refresh()
            hideCategoryDialog()
        } else {
            println("Не удалось добавить категорию")
        }
    } catch (e: Exception) {
        println("Ошибка при добавлении категории: ${e.message}")
    }
}

/**
 * Навигация между экранами приложения.
 *
 * @param screen Название целевого экрана ("Overview", "History",
"Settings")
 */
fun navigateTo(screen: String) {
    currentScreen = screen
}

/**
 * Удаляет категорию, если с ней не связано ни одной операции.
 *
 * @param name Название категории для удаления

```

```

        * @param isIncome Тип категории (true - доходы, false - расходы)
        */
        fun deleteCategory(name: String, isIncome: Boolean) {
            println("Удаление категории: '$name'. Тип: ${if (isIncome) "доходы"
            else "расходы"}")
            try {
                // Проверяем, есть ли операции с этой категорией
                val hasOperations = if (isIncome) {
                    state.value.operations.any { it.type == "Доход" &&
it.category == name }
                } else {
                    state.value.operations.any { it.type == "Расход" &&
it.category == name }
                }

                if (hasOperations) {
                    println("Нельзя удалить категорию '$name', так как есть опе-
рации с этой категорией")
                    return
                }

                repo.deleteCategory(name, isIncome)
                refresh()
            } catch (e: Exception) {
                println("Ошибка при удалении категории: ${e.message}")
                e.printStackTrace()
            }
        }

        /**
        * Округляет число до указанного количества знаков после запятой.
        *
        * @param decimals Количество знаков после запятой
        * @return Округленное значение
        */
        private fun Double.roundTo(decimals: Int): Double {
            val multiplier = 10.0.pow(decimals)
            return (this * multiplier).roundToInt() / multiplier
        }
    }

    /**
    * Расширение для округления Double до целого числа.
    *
    * @return Округленное до ближайшего целого значение
    */
    fun Double.roundToDouble(): Double = kotlin.math.round(this)

```

3.5.1.2.5. Ресурсы и визуальные элементы:

Листинг 15 – AppColors.kt – Цветовая палитра приложения

```

package ui.theme

import androidx.compose.ui.graphics.Color

/**
 * Объект, содержащий цветовую палитру приложения "Finlytics".
 * Определяет основные и вспомогательные цвета для темной темы интерфейса.
 *
 * Цветовая схема:
 * - Основные цвета: темный фон, серые оттенки для элементов

```

```

* - Акцентные цвета: синий для выделения, красный для расходов, зеленый для
доходов, желтый для баланса
* - Цвета для диаграмм: 8 цветов для доходов и 8 цветов для расходов
*
* @author Finlytics Team
* @since 1.0.0
*/
object AppColors {
    // Основные цвета интерфейса
    val DarkColor = Color(0xFF1E1E1E)           // Основной фон приложения
    val DarkGreyColor = Color(0xFF2C2C2C)        // Фон панелей и карточек
    val LightGreyColor = Color(0xFF444444)        // Фон кнопок и элементов ввода
    val LightColor = Color(0xFFFF4F4F4)         // Основной цвет текста и иконок

    // Акцентные цвета
    val BlueColor = Color(0xFF2176FF)           // Цвет выделения, активных эле-
ментов
    val RedColor = Color(0xFFEF4444)            // Цвет для расходов, кнопка
удаления
    val GreenColor = Color(0xFF4ADE80)          // Цвет для доходов
    val YellowColor = Color(0xFFECB324)         // Цвет для баланса, предупре-
ждений

    // Цветовая палитра для круговой диаграммы доходов (от синего к желтому)
    val IncomeColor1 = Color(0xFF1565C0)       // Темно-синий
    val IncomeColor2 = Color(0xFF42A5F5)       // Синий
    val IncomeColor3 = Color(0xFF29B6F6)       // Голубой
    val IncomeColor4 = Color(0xFF26A69A)       // Бирюзовый
    val IncomeColor5 = Color(0xFF66BB6A)       // Зеленый
    val IncomeColor6 = Color(0xFF9CCC65)       // Светло-зеленый
    val IncomeColor7 = Color(0xFFD4E157)       // Лаймовый
    val IncomeColor8 = Color(0xFFFFFCA28)      // Желтый

    // Цветовая палитра для круговой диаграммы расходов (от красного к фио-
летовому)
    val ExpensesColor1 = Color(0xFFD32F2F)      // Темно-красный
    val ExpensesColor2 = Color(0xFFFFF5722)    // Оранжевый
    val ExpensesColor3 = Color(0xFFFFF9800)     // Оранжево-желтый
    val ExpensesColor4 = Color(0xFFFFFB74D)     // Светло-оранжевый
    val ExpensesColor5 = Color(0xFFFFF06292)    // Розовый
    val ExpensesColor6 = Color(0xFFFB68C8)      // Фиолетовый
    val ExpensesColor7 = Color(0xFF7E57C2)      // Темно-фиолетовый
    val ExpensesColor8 = Color(0xFF5C6BC0)      // Индиго
}

```

Листинг 16 – *FinlyticsIconPack.kt* – Контейнер векторных иконок

```

package ui.theme.icons

import androidx.compose.ui.graphics.vector.ImageVector
import ui.theme.icons.finlyticsiconpack.Add
import ui.theme.icons.finlyticsiconpack.Check
import ui.theme.icons.finlyticsiconpack.Close
import ui.theme.icons.finlyticsiconpack.Date
import ui.theme.icons.finlyticsiconpack.Delete
import ui.theme.icons.finlyticsiconpack.Edit
import ui.theme.icons.finlyticsiconpack.Expenses
import ui.theme.icons.finlyticsiconpack.History
import ui.theme.icons.finlyticsiconpack.Income
import ui.theme.icons.finlyticsiconpack.Left
import ui.theme.icons.finlyticsiconpack.Minus
import ui.theme.icons.finlyticsiconpack.Plus
import ui.theme.icons.finlyticsiconpack.Right
import ui.theme.icons.finlyticsiconpack.Settings

```

```

import ui.theme.icons.finlyticsiconpack.Statistic
import ui.theme.icons.finlyticsiconpack.Wallet
import kotlin.collections.List as ____KtList

/**
 * Объект-контейнер для всех векторных иконок приложения "Finlytics".
 * Предоставляет доступ к коллекции всех иконок через свойство AllIcons.
 *
 * Доступные иконки:
 * - Add - иконка добавления (плюс)
 * - Close - иконка закрытия (крестик)
 * - Date - иконка календаря
 * - Delete - иконка удаления (корзина)
 * - Edit - иконка редактирования (карандаш)
 * - Expenses - иконка расходов (стрелка вниз)
 * - History - иконка истории (документ)
 * - Income - иконка доходов (стрелка вверх)
 * - Left - иконка навигации влево (стрелка)
 * - Minus - иконка минуса
 * - Plus - иконка плюса
 * - Right - иконка навигации вправо (стрелка)
 * - Settings - иконка настроек (шестеренка)
 * - Statistic - иконка статистики (график)
 * - Wallet - иконка кошелька
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
public object FinlyticsIconPack

private var __AllIcons: ____KtList<ImageVector>? = null

/**
 * Список всех доступных иконок в приложении.
 * Используется для превью и отладки.
 */
public val FinlyticsIconPack.AllIcons: ____KtList<ImageVector>
    get() {
        if (__AllIcons != null) {
            return __AllIcons!!
        }
        __AllIcons= listOf(Add, Check, Close, Date, Delete, Edit, Expenses, His-
        tory, Income, Left, Minus, Plus,
            Right, Settings, Statistic, Wallet)
        return __AllIcons!!
    }

```

Листинг 17 – Add.kt – Иконка "Добавить"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Добавить" – стилизованный знак плюса.

```



```

* Используется для кнопок добавления операций и категорий.
* 
* Размер: 28x28 dp
* Цвет: светлый (#F4F4F4)
* Стил: контурная иконка с закругленными краями
*/
public val FinlyticsIconPack.Add: ImageVector
    get() {
        if (_add != null) {
            return _add!!
        }
        _add = Builder(name = "Add", defaultWidth = 28.0.dp, defaultHeight =
28.0.dp, viewportWidth
        = 28.0f, viewportHeight = 28.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFFF4F4F4)), strokeLine-
            width = 3.0f, strokeLineCap = Round, strokeL-
            ineJoin = Miter,
            strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(14.0f, 4.9f)
                verticalLineTo(23.1f)
            }
            path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFFF4F4F4)), strokeLine-
            width = 3.0f, strokeLineCap = Round, strokeL-
            ineJoin = Miter,
            strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(4.9f, 14.0f)
                lineTo(23.1f, 14.0f)
            }
        }
        .build()
        return _add!!
    }

private var _add: ImageVector? = null

```

Листинг 18 – Check.kt – Иконка "Галочка"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
* Иконка "Галочка" – символ подтверждения выбора.
* Используется для обозначения выбранной категории в диалоге операции.
* 
* Размер: 18x18 dp
* Цвет: динамический (зависит от типа операции)
* Стил: контурная иконка с закругленными краями
*/
public val FinlyticsIconPack.Check: ImageVector
    get() {
        if (check != null) {

```

```

        return _check!!
    }
    _check = Builder(name = "Check", defaultWidth = 18.0.dp, defaultHeight = 18.0.dp,
        viewportWidth = 18.0f, viewportHeight = 18.0f).apply {
        path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFFF4F4)),
            strokeLineWidth = 2.5f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
            strokeLineMiter = 4.0f, pathFillType = NonZero) {
            moveTo(2.0f, 9.0f)
            lineTo(7.0f, 14.0f)
            lineTo(16.0f, 4.0f)
        }
    }
    .build()
    return _check!!
}

private var _check: ImageVector? = null

```

Листинг 19 – Close.kt – Иконка "Закрыть"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Закрыть" - стилизованный знак крестика (X).
 * Используется для закрытия диалоговых окон и модальных окон.
 *
 * Размер: 19x19 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Close: ImageVector
    get() {
        if (_close != null) {
            return _close!!
        }
        _close = Builder(name = "Close", defaultWidth = 19.0.dp, defaultHeight = 19.0.dp,
            viewportWidth = 19.0f, viewportHeight = 19.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFFF4F4)),
                strokeLineWidth = 3.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(1.5f, 1.5f)
                lineTo(17.333f, 17.333f)
                moveTo(1.5f, 17.333f)
                lineTo(17.333f, 1.5f)
            }
        }
    }
}

```

```

        .build()
        return _close!!
    }

private var _close: ImageVector? = null

```

Листинг 20 – Date.kt – Иконка "Дата"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Дата" – стилизованный календарь.
 * Используется для отображения и выбора даты в фильтрах и формах.
 *
 * Размер: 18x20 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными углами
 */
public val FinlyticsIconPack.Date: ImageVector
    get() {
        if (_date != null) {
            return _date!!
        }
        _date = Builder(name = "Date", defaultWidth = 18.0.dp, defaultHeight
= 20.0.dp,
            viewportWidth = 18.0f, viewportHeight = 20.0f).apply {
                path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFFF4F4)),
                    strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(1.0f, 7.0f)
                        horizontalLineTo(17.0f)
                        moveTo(1.0f, 7.0f)
                        verticalLineTo(15.8f)
                        curveTo(1.0f, 16.92f, 1.0f, 17.48f, 1.218f, 17.908f)
                        curveTo(1.41f, 18.284f, 1.715f, 18.59f, 2.092f, 18.782f)
                        curveTo(2.519f, 19.0f, 3.079f, 19.0f, 4.197f, 19.0f)
                        horizontalLineTo(13.803f)
                        curveTo(14.921f, 19.0f, 15.48f, 19.0f, 15.907f, 18.782f)
                        curveTo(16.284f, 18.59f, 16.59f, 18.284f, 16.782f, 17.908f)
                        curveTo(17.0f, 17.48f, 17.0f, 16.921f, 17.0f, 15.804f)
                        verticalLineTo(7.0f)
                        moveTo(1.0f, 7.0f)
                        verticalLineTo(6.2f)
                        curveTo(1.0f, 5.08f, 1.0f, 4.52f, 1.218f, 4.092f)
                        curveTo(1.41f, 3.715f, 1.715f, 3.41f, 2.092f, 3.218f)
                        curveTo(2.52f, 3.0f, 3.08f, 3.0f, 4.2f, 3.0f)
                        horizontalLineTo(5.0f)
                        moveTo(17.0f, 7.0f)
                        verticalLineTo(6.197f)
                        curveTo(17.0f, 5.079f, 17.0f, 4.519f, 16.782f, 4.092f)

```

```

        curveTo(16.59f, 3.715f, 16.284f, 3.41f, 15.907f, 3.218f)
        curveTo(15.48f, 3.0f, 14.92f, 3.0f, 13.8f, 3.0f)
        horizontalLineTo(13.0f)
        moveTo(13.0f, 1.0f)
        verticalLineTo(3.0f)
        moveTo(13.0f, 3.0f)
        horizontalLineTo(5.0f)
        moveTo(5.0f, 1.0f)
        verticalLineTo(3.0f)
    }
}

    .build()
    return _date!!
}

private var _date: ImageVector? = null

```

Листинг 21 – Delete.kt – Иконка "Удалить"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Удалить" – стилизованная корзина.
 * Используется для кнопок удаления операций и категорий.
 *
 * Размер: 28x28 dp
 * Цвет: светлый (#F4F4F4)
 * Стилль: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Delete: ImageVector
    get() {
        if (_delete != null) {
            return _delete!!
        }
        _delete = Builder(name = "Delete", defaultWidth = 28.0.dp, defaultHeight = 28.0.dp,
            viewportWidth = 28.0f, viewportHeight = 28.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F4)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(8.587f, 9.333f)
                verticalLineTo(20.4f)
                curveTo(8.587f, 21.505f, 9.482f, 22.4f, 10.587f, 22.4f)
                horizontalLineTo(17.787f)
                curveTo(18.891f, 22.4f, 19.787f, 21.505f, 19.787f, 20.4f)
                verticalLineTo(9.333f)
            }
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F4)),
                strokeLineWidth = 2.0f, strokeLineCap = Round,

```

```

strokeLineJoin = StrokeJoinRound,
    strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(6.72f, 9.333f)
    horizontalLineTo(21.653f)
}
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
        strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
        moveTo(9.52f, 9.333f)
        lineTo(11.387f, 5.6f)
        horizontalLineTo(16.987f)
        lineTo(18.853f, 9.333f)
    }
}
    .build()
    return _delete!!
}

private var _delete: ImageVector? = null

```

Листинг 22 – Edit.kt – Иконка "Редактировать"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Butt
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Редактировать" – стилизованный карандаш.
 * Используется для кнопок редактирования операций и категорий.
 *
 * Размер: 28x28 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Edit: ImageVector
    get() {
        if (_edit != null) {
            return _edit!!
        }
        _edit = Builder(name = "Edit", defaultWidth = 28.0.dp, defaultHeight
= 28.0.dp,
            viewportWidth = 28.0f, viewportHeight = 28.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = Miter,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(21.784f, 14.7f)
                verticalLineTo(17.512f)
                curveTo(21.784f, 20.274f, 19.545f, 22.512f, 16.784f,

```

```

22.512f)
        horizontalLineTo(11.16f)
        curveTo(8.399f, 22.512f, 6.16f, 20.274f, 6.16f, 17.512f)
        verticalLineTo(11.888f)
        curveTo(6.16f, 9.127f, 8.399f, 6.888f, 11.16f, 6.888f)
        horizontalLineTo(13.972f)
    }
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
        strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeLineJoin
= Miter,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
        moveTo(18.275f, 6.455f)
        curveTo(19.055f, 5.674f, 20.321f, 5.673f, 21.102f, 6.454f)
        lineTo(22.024f, 7.376f)
        curveTo(22.798f, 8.15f, 22.806f, 9.404f, 22.042f, 10.188f)
        lineTo(16.212f, 16.169f)
        curveTo(15.648f, 16.748f, 14.874f, 17.075f, 14.065f,
17.075f)
        lineTo(12.965f, 17.075f)
        curveTo(12.113f, 17.075f, 11.432f, 16.364f, 11.467f,
15.511f)
        lineTo(11.517f, 14.343f)
        curveTo(11.548f, 13.591f, 11.861f, 12.879f, 12.392f,
12.348f)
        lineTo(18.275f, 6.455f)
        close()
    }
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
        strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
        moveTo(17.209f, 7.642f)
        lineTo(20.694f, 11.126f)
    }
}
    .build()
    return _edit!!
}

private var _edit: ImageVector? = null

```

Листинг 23 – Expenses.kt – Иконка "Расходы"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Расходы" – стилизованная стрелка вниз.
 * Используется для обозначения расходов в фильтрах и статистике.
 *
 * Размер: 24x16 dp
 */

```

```

* Цвет: светлый (#F4F4F4)
* Стилль: контурная иконка с закругленными краями
*/
public val FinlyticsIconPack.Expenses: ImageVector
    get() {
        if (_expenses != null) {
            return _expenses!!
        }
        _expenses = Builder(name = "Expenses", defaultWidth = 24.0.dp, defaultHeight = 16.0.dp,
            viewportWidth = 24.0f, viewportHeight = 16.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFFF4F4)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(23.0f, 15.0f)
                lineTo(14.962f, 6.688f)
                curveTo(14.817f, 6.538f, 14.744f, 6.463f, 14.68f, 6.405f)
                curveTo(13.636f, 5.453f, 12.057f, 5.453f, 11.013f, 6.404f)
                curveTo(10.948f, 6.463f, 10.875f, 6.538f, 10.73f, 6.688f)
                curveTo(10.586f, 6.837f, 10.514f, 6.912f, 10.449f, 6.971f)
                curveTo(9.405f, 7.922f, 7.825f, 7.922f, 6.781f, 6.971f)
                curveTo(6.717f, 6.912f, 6.645f, 6.837f, 6.501f, 6.689f)
                lineTo(1.0f, 1.0f)
                moveTo(23.0f, 15.0f)
                lineTo(22.999f, 6.6f)
                moveTo(23.0f, 15.0f)
                horizontalLineTo(14.75f)
            }
        }.build()
        return _expenses!!
    }

private var _expenses: ImageVector? = null

```

Листинг 24 – History.kt – Иконка "История"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Butt
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "История" - стилизованный документ с линиями.
 * Используется для навигации на экран истории операций.
 *
 * Размер: 28x28 dp
 * Цвет: светлый (#F4F4F4)
 * Стилль: контурная иконка
 */
public val FinlyticsIconPack.History: ImageVector
    get() {

```

```

        if (_history != null) {
            return _history!!
        }
        _history = Builder(name = "History", defaultWidth = 28.0.dp, defaultHeight = 28.0.dp,
            viewportWidth = 28.0f, viewportHeight = 28.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(7.833f, 24.5f)
                curveTo(6.729f, 24.5f, 5.833f, 23.605f, 5.833f, 22.5f)
                verticalLineTo(3.5f)
                horizontalLineTo(16.333f)
                lineTo(22.167f, 9.333f)
                verticalLineTo(22.5f)
                curveTo(22.167f, 23.605f, 21.271f, 24.5f, 20.167f, 24.5f)
                horizontalLineTo(7.833f)
                close()
            }
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(15.167f, 3.5f)
                verticalLineTo(10.5f)
                horizontalLineTo(22.167f)
            }
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(10.5f, 15.167f)
                horizontalLineTo(17.5f)
            }
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(10.5f, 19.833f)
                horizontalLineTo(17.5f)
            }
        }
        .build()
        return _history!!
    }

private var _history: ImageVector? = null

```

Листинг 25 – *Income.kt* – Иконка "Доходы"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector

```



```

import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Доходы" - стилизованная стрелка вверх.
 * Используется для обозначения доходов в фильтрах и статистике.
 *
 * Размер: 24x16 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Income: ImageVector
    get() {
        if (_income != null) {
            return _income!!
        }
        _income = Builder(name = "Income", defaultWidth = 24.0.dp, defaultHeight = 16.0.dp,
            viewportWidth = 24.0f, viewportHeight = 16.0f).apply {
            path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                strokeLineWidth = 2.0f, strokeLineCap = Round, strokeLineJoin = StrokeJoinRound,
                strokeLineMiter = 4.0f, pathFillType = NonZero) {
                moveTo(23.0f, 1.0f)
                lineTo(14.962f, 9.313f)
                curveTo(14.817f, 9.462f, 14.744f, 9.537f, 14.68f, 9.595f)
                curveTo(13.636f, 10.547f, 12.057f, 10.547f, 11.013f, 9.596f)
                curveTo(10.948f, 9.537f, 10.875f, 9.462f, 10.73f, 9.312f)
                curveTo(10.586f, 9.163f, 10.514f, 9.088f, 10.449f, 9.029f)
                curveTo(9.405f, 8.078f, 7.825f, 8.078f, 6.781f, 9.029f)
                curveTo(6.717f, 9.088f, 6.645f, 9.163f, 6.501f, 9.311f)
                lineTo(1.0f, 15.0f)
                moveTo(23.0f, 1.0f)
                lineTo(22.999f, 9.4f)
                moveTo(23.0f, 1.0f)
                horizontalLineTo(14.75f)
            }
        }.build()
        return _income!!
    }

private var _income: ImageVector? = null

```

Листинг 26 – Left.kt – Иконка "Влево"

```

package ui.theme.icons.finallyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinallyticsIconPack

/**

```

```

* Иконка "Влево" - стрелка навигации влево.
* Используется для навигации по периодам (предыдущий день, неделя, месяц,
год).
*
* Размер: 20x20 dp
* Цвет: светлый (#F4F4F4)
* Стилль: контурная иконка с закругленными краями
*/
public val FinlyticsIconPack.Left: ImageVector
    get() {
        if (_left != null) {
            return _left!!
        }
        _left = Builder(name = "Left", defaultWidth = 20.0.dp, defaultHeight
= 20.0.dp,
            viewportWidth = 20.0f, viewportHeight = 20.0f).apply {
                path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFFF4F4F4)),
                    strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(11.667f, 14.167f)
                        lineTo(7.5f, 10.0f)
                    }
                path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFFF4F4F4)),
                    strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(7.5f, 10.0f)
                        lineTo(11.667f, 5.833f)
                    }
            }
        .build()
        return _left!!
    }

private var _left: ImageVector? = null

```

Листинг 27 – Minus.kt – Иконка "Минус"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
* Иконка "Минус" - горизонтальная черта.
* Используется для обозначения вычитания или отрицательного значения.
*
* Размер: 17x3 dp
* Цвет: светлый (#F4F4F4)
* Стилль: контурная иконка с закругленными краями
*/
public val FinlyticsIconPack.Minus: ImageVector
    get() {

```

```

        if (_minus != null) {
            return _minus!!
        }
        _minus = Builder(name = "Minus", defaultWidth = 17.0.dp, defaultHeight = 3.0.dp,
            viewportWidth = 17.0f, viewportHeight = 3.0f).apply {
                path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                    strokeLineWidth = 3.0f, strokeLineCap = Round, strokeLineJoin = Miter,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(1.5f, 1.5f)
                        lineTo(15.5f, 1.5f)
                    }
            }
        .build()
        return _minus!!
    }

private var _minus: ImageVector? = null

```

Листинг 28 – Plus.kt – Иконка "Плюс"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Плюс" - пересекающиеся горизонтальная и вертикальная линии.
 * Используется для обозначения добавления или положительного значения.
 *
 * Размер: 17x17 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Plus: ImageVector
    get() {
        if (_plus != null) {
            return _plus!!
        }
        _plus = Builder(name = "Plus", defaultWidth = 17.0.dp, defaultHeight = 17.0.dp,
            viewportWidth = 17.0f, viewportHeight = 17.0f).apply {
                path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                    strokeLineWidth = 3.0f, strokeLineCap = Round, strokeLineJoin = Miter,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(8.5f, 1.5f)
                        verticalLineTo(15.5f)
                    }
                path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F)),
                    strokeLineWidth = 3.0f, strokeLineCap = Round, strokeLineJoin = Miter,

```

```

        strokeLineMiter = 4.0f, pathFillType = NonZero) {
            moveTo(1.5f, 8.5f)
            horizontalLineTo(15.5f)
        }
    }
    .build()
    return _plus!!
}

private var _plus: ImageVector? = null

```

Листинг 29 – *Right.kt* – Иконка "Вправо"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Вправо" - стрелка навигации вправо.
 * Используется для навигации по периодам (следующий день, неделя, месяц,
 * год).
 *
 * Размер: 20x20 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: контурная иконка с закругленными краями
 */
public val FinlyticsIconPack.Right: ImageVector
    get() {
        if (_right != null) {
            return _right!!
        }
        _right = Builder(name = "Right", defaultWidth = 20.0.dp, de-
faultHeight = 20.0.dp,
            viewportWidth = 20.0f, viewportHeight = 20.0f).apply {
                path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
                    strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(8.333f, 14.167f)
                        lineTo(12.5f, 10.0f)
                    }
                path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F4)),
                    strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                        moveTo(12.5f, 10.0f)
                        lineTo(8.333f, 5.833f)
                    }
            }
        .build()
        return _right!!
    }
}

```

```
private var _right: ImageVector? = null
```

Листинг 30 – *Settings.kt* – Иконка "Настройки"

```
package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeCap.Companion.Butt
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Настройки" – стилизованная шестеренка с кружком внутри.
 * Используется для навигации на экран управления категориями и настройками.
 *
 * Размер: 28x28 dp
 * Цвет: светлый (#F4F4F4)
 * Стилль: заливка для основной формы, контур для шестеренки
 */
public val FinlyticsIconPack.Settings: ImageVector
    get() {
        if (_settings != null) {
            return _settings!!
        }
        _settings = Builder(name = "Settings", defaultWidth = 28.0.dp, defaultHeight = 28.0.dp,
            viewportWidth = 28.0f, viewportHeight = 28.0f).apply {
            path(fill = SolidColor(Color(0xFFF4F4F4)), stroke = null,
                strokeLineWidth = 0.0f,
                strokeLineCap = Butt, strokeLineJoin = Miter,
                strokeLineMiter = 4.0f,
                pathFillType = NonZero) {
                moveTo(14.062f, 3.652f)
                lineTo(14.063f, 2.652f)
                horizontalLineTo(14.062f)
                verticalLineTo(3.652f)
                close()
                moveTo(18.444f, 6.512f)
                lineTo(17.944f, 7.377f)
                lineTo(17.944f, 7.378f)
                lineTo(18.444f, 6.512f)
                close()
                moveTo(21.851f, 13.389f)
                lineTo(20.851f, 13.389f)
                verticalLineTo(13.389f)
                horizontalLineTo(21.851f)
                close()
                moveTo(18.443f, 20.264f)
                lineTo(17.943f, 19.397f)
                lineTo(17.943f, 19.397f)
                lineTo(18.443f, 20.264f)
                close()
                moveTo(14.062f, 23.128f)
                lineTo(14.062f, 24.128f)
            }
        }
    }
```

```

lineTo(14.063f, 24.128f)
lineTo(14.062f, 23.128f)
close()
moveTo(9.683f, 20.264f)
lineTo(10.183f, 19.397f)
lineTo(10.183f, 19.397f)
lineTo(9.683f, 20.264f)
close()
moveTo(6.274f, 13.389f)
lineTo(7.274f, 13.389f)
lineTo(7.274f, 13.389f)
lineTo(6.274f, 13.389f)
close()
moveTo(9.682f, 6.513f)
lineTo(10.182f, 7.379f)
lineTo(10.182f, 7.378f)
lineTo(9.682f, 6.513f)
close()
moveTo(11.06f, 4.655f)
lineTo(12.034f, 4.885f)
lineTo(11.06f, 4.655f)
close()
moveTo(11.677f, 3.946f)
lineTo(11.433f, 2.977f)
lineTo(11.677f, 3.946f)
close()
moveTo(7.565f, 19.948f)
lineTo(7.34f, 18.973f)
lineTo(7.565f, 19.948f)
close()
moveTo(6.656f, 19.709f)
lineTo(7.417f, 19.06f)
lineTo(6.656f, 19.709f)
close()
moveTo(11.679f, 22.834f)
lineTo(11.435f, 23.803f)
lineTo(11.679f, 22.834f)
close()
moveTo(11.062f, 22.124f)
lineTo(12.035f, 21.895f)
lineTo(11.062f, 22.124f)
close()
moveTo(17.062f, 22.123f)
lineTo(18.036f, 22.352f)
lineTo(17.062f, 22.123f)
close()
moveTo(16.445f, 22.833f)
lineTo(16.201f, 21.863f)
lineTo(16.445f, 22.833f)
close()
moveTo(21.466f, 19.711f)
lineTo(20.706f, 19.061f)
lineTo(21.466f, 19.711f)
close()
moveTo(23.25f, 10.165f)
lineTo(24.194f, 9.834f)
lineTo(23.25f, 10.165f)
close()
moveTo(22.902f, 11.147f)
lineTo(23.542f, 11.915f)
lineTo(22.902f, 11.147f)
close()
moveTo(21.463f, 7.065f)
lineTo(20.703f, 7.715f)

```

```

lineTo(21.463f, 7.065f)
close()
moveTo(7.568f, 6.827f)
lineTo(7.343f, 7.802f)
lineTo(7.568f, 6.827f)
close()
moveTo(22.903f, 15.631f)
lineTo(22.263f, 16.4f)
lineTo(22.903f, 15.631f)
close()
moveTo(17.065f, 4.655f)
lineTo(18.038f, 4.425f)
lineTo(17.065f, 4.655f)
close()
moveTo(14.062f, 3.652f)
lineTo(14.062f, 4.652f)
curveTo(14.802f, 4.652f, 15.519f, 4.744f, 16.204f, 4.916f)
lineTo(16.448f, 3.946f)
lineTo(16.692f, 2.977f)
curveTo(15.849f, 2.765f, 14.968f, 2.652f, 14.063f, 2.652f)
lineTo(14.062f, 3.652f)
close()
moveTo(17.065f, 4.655f)
lineTo(16.092f, 4.885f)
curveTo(16.331f, 5.896f, 16.97f, 6.815f, 17.944f, 7.377f)
lineTo(18.444f, 6.512f)
lineTo(18.944f, 5.646f)
curveTo(18.469f, 5.371f, 18.156f, 4.924f, 18.038f, 4.425f)
lineTo(17.065f, 4.655f)
close()
moveTo(18.444f, 6.512f)
lineTo(17.944f, 7.378f)
curveTo(18.838f, 7.894f, 19.851f, 8.015f, 20.78f, 7.801f)
lineTo(20.556f, 6.827f)
lineTo(20.332f, 5.852f)
curveTo(19.873f, 5.958f, 19.381f, 5.898f, 18.944f, 5.646f)
lineTo(18.444f, 6.512f)
close()
moveTo(21.463f, 7.065f)
lineTo(20.703f, 7.715f)
curveTo(21.398f, 8.528f, 21.946f, 9.468f, 22.307f, 10.497f)
lineTo(23.25f, 10.165f)
lineTo(24.194f, 9.834f)
curveTo(23.75f, 8.569f, 23.076f, 7.413f, 22.223f, 6.415f)
lineTo(21.463f, 7.065f)
close()
moveTo(22.902f, 11.147f)
lineTo(22.262f, 10.379f)
curveTo(21.402f, 11.096f, 20.851f, 12.178f, 20.851f,
13.389f)
lineTo(21.851f, 13.389f)
lineTo(22.851f, 13.39f)
curveTo(22.851f, 12.798f, 23.118f, 12.269f, 23.542f,
11.915f)
lineTo(22.902f, 11.147f)
close()
moveTo(21.851f, 13.389f)
horizontalLineTo(20.851f)
curveTo(20.851f, 14.602f, 21.403f, 15.683f, 22.263f, 16.4f)
lineTo(22.903f, 15.631f)
lineTo(23.543f, 14.863f)
curveTo(23.118f, 14.509f, 22.851f, 13.981f, 22.851f,
13.389f)
horizontalLineTo(21.851f)

```

```

close()
moveTo(23.251f, 16.612f)
lineTo(22.308f, 16.281f)
curveTo(21.948f, 17.309f, 21.4f, 18.249f, 20.706f, 19.061f)
lineTo(21.466f, 19.711f)
lineTo(22.227f, 20.36f)
curveTo(23.079f, 19.363f, 23.752f, 18.208f, 24.195f,
16.943f)
lineTo(23.251f, 16.612f)
close()
moveTo(20.558f, 19.949f)
lineTo(20.784f, 18.975f)
curveTo(19.853f, 18.76f, 18.838f, 18.881f, 17.943f, 19.397f)
lineTo(18.443f, 20.264f)
lineTo(18.943f, 21.13f)
curveTo(19.381f, 20.877f, 19.874f, 20.817f, 20.333f,
20.923f)
lineTo(20.558f, 19.949f)
close()
moveTo(18.443f, 20.264f)
lineTo(17.943f, 19.397f)
curveTo(16.968f, 19.961f, 16.328f, 20.881f, 16.089f,
21.894f)
lineTo(17.062f, 22.123f)
lineTo(18.036f, 22.352f)
curveTo(18.153f, 21.852f, 18.467f, 21.405f, 18.943f,
21.129f)
lineTo(18.443f, 20.264f)
close()
moveTo(16.445f, 22.833f)
lineTo(16.201f, 21.863f)
curveTo(15.517f, 22.035f, 14.801f, 22.128f, 14.062f,
22.128f)
lineTo(14.062f, 23.128f)
lineTo(14.063f, 24.128f)
curveTo(14.968f, 24.128f, 15.848f, 24.014f, 16.689f,
23.802f)
lineTo(16.445f, 22.833f)
close()
moveTo(14.062f, 23.128f)
verticalLineTo(22.128f)
curveTo(13.323f, 22.128f, 12.607f, 22.036f, 11.923f,
21.864f)
lineTo(11.679f, 22.834f)
lineTo(11.435f, 23.803f)
curveTo(12.277f, 24.015f, 13.157f, 24.128f, 14.062f,
24.128f)
verticalLineTo(23.128f)
close()
moveTo(11.062f, 22.124f)
lineTo(12.035f, 21.895f)
curveTo(11.798f, 20.882f, 11.159f, 19.961f, 10.183f,
19.397f)
lineTo(9.683f, 20.264f)
lineTo(9.182f, 21.129f)
curveTo(9.659f, 21.404f, 9.971f, 21.852f, 10.088f, 22.352f)
lineTo(11.062f, 22.124f)
close()
moveTo(9.683f, 20.264f)
lineTo(10.183f, 19.397f)
curveTo(9.287f, 18.881f, 8.272f, 18.758f, 7.34f, 18.973f)
lineTo(7.565f, 19.948f)
lineTo(7.791f, 20.922f)
curveTo(8.249f, 20.816f, 8.744f, 20.876f, 9.183f, 21.129f)

```



```

lineTo(9.683f, 20.264f)
close()
moveTo(6.656f, 19.709f)
lineTo(7.417f, 19.06f)
curveTo(6.723f, 18.248f, 6.177f, 17.308f, 5.817f, 16.282f)
lineTo(4.873f, 16.612f)
lineTo(3.93f, 16.943f)
curveTo(4.373f, 18.207f, 5.045f, 19.362f, 5.896f, 20.358f)
lineTo(6.656f, 19.709f)
close()
moveTo(5.222f, 15.631f)
lineTo(5.862f, 16.4f)
curveTo(6.722f, 15.683f, 7.274f, 14.602f, 7.274f, 13.389f)
horizontalLineTo(6.274f)
horizontalLineTo(5.274f)
curveTo(5.274f, 13.981f, 5.007f, 14.509f, 4.582f, 14.863f)
lineTo(5.222f, 15.631f)
close()
moveTo(6.274f, 13.389f)
lineTo(7.274f, 13.389f)
curveTo(7.274f, 12.177f, 6.723f, 11.095f, 5.862f, 10.378f)
lineTo(5.222f, 11.147f)
lineTo(4.582f, 11.915f)
curveTo(5.007f, 12.269f, 5.274f, 12.798f, 5.274f, 13.39f)
lineTo(6.274f, 13.389f)
close()
moveTo(4.874f, 10.165f)
lineTo(5.817f, 10.496f)
curveTo(6.178f, 9.468f, 6.726f, 8.528f, 7.421f, 7.716f)
lineTo(6.661f, 7.066f)
lineTo(5.901f, 6.416f)
curveTo(5.048f, 7.413f, 4.374f, 8.569f, 3.93f, 9.834f)
lineTo(4.874f, 10.165f)
close()
moveTo(7.568f, 6.827f)
lineTo(7.343f, 7.802f)
curveTo(8.273f, 8.016f, 9.287f, 7.895f, 10.182f, 7.379f)
lineTo(9.682f, 6.513f)
lineTo(9.182f, 5.646f)
curveTo(8.745f, 5.899f, 8.252f, 5.959f, 7.793f, 5.853f)
lineTo(7.568f, 6.827f)
close()
moveTo(9.682f, 6.513f)
lineTo(10.182f, 7.378f)
curveTo(11.156f, 6.815f, 11.795f, 5.896f, 12.034f, 4.885f)
lineTo(11.06f, 4.655f)
lineTo(10.087f, 4.426f)
curveTo(9.969f, 4.925f, 9.657f, 5.372f, 9.181f, 5.647f)
lineTo(9.682f, 6.513f)
close()
moveTo(11.677f, 3.946f)
lineTo(11.922f, 4.916f)
curveTo(12.606f, 4.744f, 13.323f, 4.652f, 14.062f, 4.652f)
verticalLineTo(3.652f)
verticalLineTo(2.652f)
curveTo(13.157f, 2.652f, 12.276f, 2.765f, 11.433f, 2.977f)
lineTo(11.677f, 3.946f)
close()
moveTo(11.06f, 4.655f)
lineTo(12.034f, 4.885f)
curveTo(12.038f, 4.869f, 12.042f, 4.856f, 12.046f, 4.848f)
curveTo(12.05f, 4.839f, 12.052f, 4.839f, 12.047f, 4.844f)
curveTo(12.042f, 4.85f, 12.03f, 4.863f, 12.008f, 4.878f)
curveTo(11.985f, 4.894f, 11.955f, 4.908f, 11.922f, 4.916f)

```

```

lineTo(11.677f, 3.946f)
lineTo(11.433f, 2.977f)
curveTo(10.619f, 3.182f, 10.216f, 3.879f, 10.087f, 4.426f)
lineTo(11.06f, 4.655f)
close()
moveTo(5.222f, 11.147f)
lineTo(5.862f, 10.378f)
curveTo(5.848f, 10.366f, 5.836f, 10.354f, 5.829f, 10.344f)
curveTo(5.821f, 10.334f, 5.821f, 10.33f, 5.823f, 10.336f)
curveTo(5.825f, 10.342f, 5.832f, 10.361f, 5.833f, 10.392f)
curveTo(5.835f, 10.424f, 5.83f, 10.46f, 5.817f, 10.496f)
lineTo(4.874f, 10.165f)
lineTo(3.93f, 9.834f)
curveTo(3.615f, 10.731f, 4.106f, 11.519f, 4.582f, 11.915f)
lineTo(5.222f, 11.147f)
close()
moveTo(7.565f, 19.948f)
lineTo(7.34f, 18.973f)
curveTo(7.324f, 18.977f, 7.31f, 18.979f, 7.301f, 18.979f)
curveTo(7.291f, 18.979f, 7.29f, 18.978f, 7.297f, 18.979f)
curveTo(7.304f, 18.981f, 7.321f, 18.986f, 7.345f, 18.999f)
curveTo(7.368f, 19.013f, 7.394f, 19.033f, 7.417f, 19.06f)
lineTo(6.656f, 19.709f)
lineTo(5.896f, 20.358f)
curveTo(6.44f, 20.996f, 7.243f, 21.049f, 7.791f, 20.922f)
lineTo(7.565f, 19.948f)
close()
moveTo(11.679f, 22.834f)
lineTo(11.923f, 21.864f)
curveTo(11.957f, 21.872f, 11.987f, 21.886f, 12.01f, 21.902f)
curveTo(12.032f, 21.917f, 12.044f, 21.93f, 12.049f, 21.936f)
curveTo(12.054f, 21.941f, 12.052f, 21.941f, 12.048f,
21.932f)
curveTo(12.044f, 21.924f, 12.039f, 21.911f, 12.035f,
21.895f)
lineTo(11.062f, 22.124f)
lineTo(10.088f, 22.352f)
curveTo(10.217f, 22.9f, 10.62f, 23.598f, 11.435f, 23.803f)
lineTo(11.679f, 22.834f)
close()
moveTo(17.062f, 22.123f)
lineTo(16.089f, 21.894f)
curveTo(16.085f, 21.91f, 16.08f, 21.923f, 16.076f, 21.931f)
curveTo(16.072f, 21.94f, 16.071f, 21.94f, 16.075f, 21.935f)
curveTo(16.08f, 21.929f, 16.092f, 21.916f, 16.114f, 21.901f)
curveTo(16.137f, 21.886f, 16.167f, 21.872f, 16.201f,
21.863f)
lineTo(16.445f, 22.833f)
lineTo(16.689f, 23.802f)
curveTo(17.503f, 23.597f, 17.907f, 22.899f, 18.036f,
22.352f)
lineTo(17.062f, 22.123f)
close()
moveTo(21.466f, 19.711f)
lineTo(20.706f, 19.061f)
curveTo(20.729f, 19.035f, 20.754f, 19.014f, 20.778f,
19.001f)
curveTo(20.801f, 18.987f, 20.819f, 18.982f, 20.826f,
18.981f)
curveTo(20.833f, 18.979f, 20.832f, 18.98f, 20.822f, 18.98f)
curveTo(20.813f, 18.98f, 20.8f, 18.979f, 20.784f, 18.975f)
lineTo(20.558f, 19.949f)
lineTo(20.333f, 20.923f)
curveTo(20.88f, 21.05f, 21.683f, 20.997f, 22.227f, 20.36f)

```

```

        lineTo(21.466f, 19.711f)
        close()
        moveTo(23.25f, 10.165f)
        lineTo(22.307f, 10.497f)
        curveTo(22.294f, 10.461f, 22.289f, 10.424f, 22.291f,
10.392f)
        curveTo(22.292f, 10.361f, 22.299f, 10.343f, 22.301f,
10.336f)
        curveTo(22.303f, 10.33f, 22.303f, 10.334f, 22.295f, 10.344f)
        curveTo(22.288f, 10.354f, 22.276f, 10.367f, 22.262f,
10.379f)
        lineTo(22.902f, 11.147f)
        lineTo(23.542f, 11.915f)
        curveTo(24.018f, 11.518f, 24.508f, 10.731f, 24.194f, 9.834f)
        lineTo(23.25f, 10.165f)
        close()
        moveTo(4.873f, 16.612f)
        lineTo(5.817f, 16.282f)
        curveTo(5.83f, 16.318f, 5.834f, 16.354f, 5.833f, 16.386f)
        curveTo(5.832f, 16.417f, 5.825f, 16.436f, 5.823f, 16.442f)
        curveTo(5.821f, 16.448f, 5.821f, 16.444f, 5.829f, 16.434f)
        curveTo(5.836f, 16.424f, 5.847f, 16.412f, 5.862f, 16.4f)
        lineTo(5.222f, 15.631f)
        lineTo(4.582f, 14.863f)
        curveTo(4.106f, 15.259f, 3.615f, 16.046f, 3.93f, 16.943f)
        lineTo(4.873f, 16.612f)
        close()
        moveTo(20.556f, 6.827f)
        lineTo(20.78f, 7.801f)
        curveTo(20.796f, 7.798f, 20.81f, 7.796f, 20.819f, 7.796f)
        curveTo(20.828f, 7.796f, 20.829f, 7.797f, 20.822f, 7.796f)
        curveTo(20.815f, 7.794f, 20.798f, 7.789f, 20.775f, 7.776f)
        curveTo(20.751f, 7.762f, 20.726f, 7.742f, 20.703f, 7.715f)
        lineTo(21.463f, 7.065f)
        lineTo(22.223f, 6.415f)
        curveTo(21.679f, 5.78f, 20.878f, 5.726f, 20.332f, 5.852f)
        lineTo(20.556f, 6.827f)
        close()
        moveTo(6.661f, 7.066f)
        lineTo(7.421f, 7.716f)
        curveTo(7.398f, 7.742f, 7.372f, 7.762f, 7.349f, 7.776f)
        curveTo(7.326f, 7.789f, 7.308f, 7.794f, 7.301f, 7.796f)
        curveTo(7.294f, 7.797f, 7.295f, 7.796f, 7.305f, 7.797f)
        curveTo(7.314f, 7.797f, 7.327f, 7.798f, 7.343f, 7.802f)
        lineTo(7.568f, 6.827f)
        lineTo(7.793f, 5.853f)
        curveTo(7.246f, 5.727f, 6.445f, 5.78f, 5.901f, 6.416f)
        lineTo(6.661f, 7.066f)
        close()
        moveTo(22.903f, 15.631f)
        lineTo(22.263f, 16.4f)
        curveTo(22.278f, 16.412f, 22.289f, 16.424f, 22.296f,
16.434f)
        curveTo(22.304f, 16.444f, 22.304f, 16.448f, 22.302f,
16.442f)
        curveTo(22.3f, 16.436f, 22.293f, 16.417f, 22.292f, 16.386f)
        curveTo(22.291f, 16.354f, 22.295f, 16.317f, 22.308f,
16.281f)
        lineTo(23.251f, 16.612f)
        lineTo(24.195f, 16.943f)
        curveTo(24.51f, 16.046f, 24.019f, 15.259f, 23.543f, 14.863f)
        lineTo(22.903f, 15.631f)
        close()
        moveTo(16.448f, 3.946f)

```

```

        lineTo(16.204f, 4.916f)
        curveTo(16.17f, 4.908f, 16.14f, 4.894f, 16.117f, 4.878f)
        curveTo(16.095f, 4.863f, 16.083f, 4.85f, 16.078f, 4.845f)
        curveTo(16.073f, 4.839f, 16.075f, 4.84f, 16.079f, 4.848f)
        curveTo(16.083f, 4.856f, 16.088f, 4.869f, 16.092f, 4.885f)
        lineTo(17.065f, 4.655f)
        lineTo(18.038f, 4.425f)
        curveTo(17.909f, 3.879f, 17.506f, 3.182f, 16.692f, 2.977f)
        lineTo(16.448f, 3.946f)
        close()
    }
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFFF4F4F)),
        strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeL-
ineJoin = Miter,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
        moveTo(14.0f, 14.0f)
        moveToRelative(3.043f, 0.0f)
        arcToRelative(3.043f, 3.043f, 0.0f, true, false, -6.087f,
0.0f)
        arcToRelative(3.043f, 3.043f, 0.0f, true, false, 6.087f,
0.0f)
    }
    }
    .build()
    return _settings!!
}

private var _settings: ImageVector? = null

```

Листинг 31 – *Statistic.kt* – Иконка "Статистика"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Butt
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Статистика" – стилизованная гистограмма с графиком.
 * Используется для навигации на главный экран с обзором статистики.
 *
 * Изображение представляет собой:
 * - Три столбца разной высоты, отображающие данные
 * - Линию графика, соединяющую вершины столбцов
 * - Горизонтальную ось внизу
 *
 * Размер: 28x28 dp
 * Цвет: светлый (#F4F4F4)
 * Стил: комбинированный (заливка для столбцов, контур для линии графика)
 */
public val FinlyticsIconPack.Statistic: ImageVector
    get() {
        if (_statistic != null) {
            return _statistic!!
        }
    }

```

```

        _statistic = Builder(name = "Statistic", defaultWidth = 28.0.dp, defaultHeight = 28.0.dp,
            viewportWidth = 28.0f, viewportHeight = 28.0f).apply {
                // Левый столбец гистограммы (заливка)
                path(fill = SolidColor(Color(0xFFFF4F4F4)), stroke = null,
                    strokeLineWidth = 0.0f,
                    strokeLineCap = Butt, strokeLineJoin = Miter, strokeLineMiter = 4.0f,
                    pathFillType = NonZero) {
                    moveTo(16.227f, 5.9f)
                    curveTo(16.227f, 5.072f, 15.556f, 4.4f, 14.727f, 4.4f)
                    curveTo(13.899f, 4.4f, 13.227f, 5.072f, 13.227f, 5.9f)
                    horizontalLineTo(14.727f)
                    horizontalLineTo(16.227f)
                    close()
                    moveTo(14.727f, 19.9f)
                    horizontalLineTo(16.227f)
                    verticalLineTo(5.9f)
                    horizontalLineTo(14.727f)
                    horizontalLineTo(13.227f)
                    verticalLineTo(19.9f)
                    horizontalLineTo(14.727f)
                    close()
                }
                // Правый столбец гистограммы (заливка)
                path(fill = SolidColor(Color(0xFFFF4F4F4)), stroke = null,
                    strokeLineWidth = 0.0f,
                    strokeLineCap = Butt, strokeLineJoin = Miter, strokeLineMiter = 4.0f,
                    pathFillType = NonZero) {
                    moveTo(22.242f, 10.45f)
                    curveTo(22.242f, 9.622f, 21.57f, 8.95f, 20.742f, 8.95f)
                    curveTo(19.913f, 8.95f, 19.242f, 9.622f, 19.242f, 10.45f)
                    horizontalLineTo(20.742f)
                    horizontalLineTo(22.242f)
                    close()
                    moveTo(20.742f, 19.9f)
                    horizontalLineTo(22.242f)
                    verticalLineTo(10.45f)
                    horizontalLineTo(20.742f)
                    horizontalLineTo(19.242f)
                    verticalLineTo(19.9f)
                    horizontalLineTo(20.742f)
                    close()
                }
                // Линия графика и горизонтальная ось (контур)
                path(fill = SolidColor(Color(0x00000000)), stroke = SolidColor(Color(0xFFFF4F4F4)),
                    strokeLineWidth = 3.0f, strokeLineCap = Round, strokeLineJoin = Miter,
                    strokeLineMiter = 4.0f, pathFillType = NonZero) {
                    moveTo(3.5f, 4.5f)
                    verticalLineTo(20.05f)
                    curveTo(3.5f, 21.707f, 4.843f, 23.05f, 6.5f, 23.05f)
                    horizontalLineTo(24.751f)
                }
                // Средний столбец гистограммы (заливка)
                path(fill = SolidColor(Color(0xFFFF4F4F4)), stroke = null,
                    strokeLineWidth = 0.0f,
                    strokeLineCap = Butt, strokeLineJoin = Miter, strokeLineMiter = 4.0f,
                    pathFillType = NonZero) {
                    moveTo(10.614f, 15.35f)
                    curveTo(10.614f, 14.521f, 9.942f, 13.85f, 9.114f, 13.85f)

```

```

        curveTo(8.285f, 13.85f, 7.614f, 14.521f, 7.614f, 15.35f)
        horizontalLineTo(9.114f)
        horizontalLineTo(10.614f)
        close()
        moveTo(9.114f, 19.9f)
        horizontalLineTo(10.614f)
        verticalLineTo(15.35f)
        horizontalLineTo(9.114f)
        horizontalLineTo(7.614f)
        verticalLineTo(19.9f)
        horizontalLineTo(9.114f)
        close()
    }
}

    .build()
    return _statistic!!
}

private var _statistic: ImageVector? = null

```

Листинг 32 – *Wallet.kt* – Иконка "Кошелек"

```

package ui.theme.icons.finlyticsiconpack

import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.PathFillType.Companion.NonZero
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap.Companion.Butt
import androidx.compose.ui.graphics.StrokeCap.Companion.Round
import androidx.compose.ui.graphics.StrokeJoin.Companion.Miter
import androidx.compose.ui.graphics.StrokeJoin.Companion.Round as
StrokeJoinRound
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.ImageVector.Builder
import androidx.compose.ui.graphics.vector.path
import androidx.compose.ui.unit.dp
import ui.theme.icons.FinlyticsIconPack

/**
 * Иконка "Кошелек" – стилизованное изображение кошелька.
 * Используется для отображения баланса на главном экране.
 *
 * Изображение представляет собой:
 * - Основной контур кошелька с закругленными углами
 * - Декоративный элемент (клапан) сверху
 * - Слот для карты сбоку
 * - Индикатор считывания чипа
 *
 * Размер: 22x22 dp
 * Цвет: желтый (#FFECB324) – соответствует цвету баланса
 * Стил: контурная иконка
 */
public val FinlyticsIconPack.Wallet: ImageVector
    get() {
        if (_wallet != null) {
            return _wallet!!
        }
        _wallet = Builder(name = "Wallet", defaultWidth = 22.0.dp,
            defaultHeight = 22.0.dp,
            viewportWidth = 22.0f, viewportHeight = 22.0f).apply {
            // Основной контур кошелька
            path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
                Color(Color(0xFFECB324)),
                strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeLineJoin

```

```

= Miter,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(6.0f, 4.17f)
    lineTo(16.0f, 4.17f)
    arcTo(5.0f, 5.0f, 0.0f, false, true, 21.0f, 9.17f)
    lineTo(21.0f, 15.17f)
    arcTo(5.0f, 5.0f, 0.0f, false, true, 16.0f, 20.17f)
    lineTo(6.0f, 20.17f)
    arcTo(5.0f, 5.0f, 0.0f, false, true, 1.0f, 15.17f)
    lineTo(1.0f, 9.17f)
    arcTo(5.0f, 5.0f, 0.0f, false, true, 6.0f, 4.17f)
    close()
    }
    // Декоративный клапан кошелька
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFECB324)),
        strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeLineJoin
= Miter,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(18.0f, 4.67f)
    curveTo(18.0f, 2.346f, 15.868f, 0.609f, 13.592f, 1.077f)
    lineTo(4.992f, 2.848f)
    curveTo(2.668f, 3.326f, 1.0f, 5.372f, 1.0f, 7.745f)
    lineTo(1.0f, 11.17f)
    }
    // Декоративная полоска на кошельке
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFECB324)),
        strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(5.0f, 15.67f)
    horizontalLineTo(11.0f)
    }
    // Слот для карты
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFECB324)),
        strokeLineWidth = 2.0f, strokeLineCap = Butt, strokeLineJoin
= Miter,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(14.0f, 12.17f)
    curveTo(14.0f, 10.789f, 15.119f, 9.67f, 16.5f, 9.67f)
    horizontalLineTo(21.0f)
    verticalLineTo(14.67f)
    horizontalLineTo(16.5f)
    curveTo(15.119f, 14.67f, 14.0f, 13.55f, 14.0f, 12.17f)
    close()
    }
    // Индикатор считывания чипа
    path(fill = SolidColor(Color(0x00000000)), stroke = Solid-
Color(Color(0xFFECB324)),
        strokeLineWidth = 2.0f, strokeLineCap = Round, strokeL-
ineJoin = StrokeJoinRound,
        strokeLineMiter = 4.0f, pathFillType = NonZero) {
    moveTo(16.5f, 12.17f)
    horizontalLineTo(16.7f)
    }
    }
    .build()
    return _wallet!!
}

private var _wallet: ImageVector? = null

```

3.5.2. Backend

3.5.2.1. Структура слоя

В контексте офлайн-приложения "Finlytics" под backend понимается слой данных и бизнес-логики, работающий локально на устройстве пользователя.

Архитектурный паттерн: MVVM (Model-View-ViewModel)

Структура backend-слоя:

1. Модель данных:
 - Operation.kt – data-класс, представляющий модель финансовой операции с полями: id, тип, сумма, категория, дата и описание.
2. Репозиторий и вспомогательные утилиты:
 - FinanceRepository.kt – центральный репозиторий, абстрагирующий доступ к данным и обеспечивающий единую точку взаимодействия между ViewModel и базой данных;
 - LoggingConfig.kt – утилита для настройки корректного логирования с поддержкой UTF-8.
3. Работа с базой данных SQLite:
 - Конфигурация и инициализация БД:
 - DatabaseConfig.kt – управляет подключением к SQLite базе данных, автоматически находит или создает файл БД в доступных локациях;
 - DatabaseInitializer.kt – отвечает за создание всех необходимых таблиц при первом запуске приложения и обеспечение целостности схемы БД;
 - DefaultCategories.kt – предоставляет стандартные категории доходов и расходов, инициализирует их при первом запуске приложения.

- Операции с категориями (CRUD):
 - `GetIncomeCategories.kt` – получение всех категорий доходов из базы данных;
 - `GetExpensesCategories.kt` – получение всех категорий расходов из базы данных;
 - `AddCategory.kt` – добавление новых категорий доходов и расходов;
 - `DeleteCategory.kt` – удаление категорий доходов и расходов по их идентификаторам.
- Операции с транзакциями доходов (CRUD):
 - `AddIncomeTransaction.kt` – добавление новых транзакций доходов в базу данных;
 - `GetIncomeTransactions.kt` – получение всех транзакций доходов с информацией о категориях;
 - `UpdateIncomeTransaction.kt` – обновление существующих транзакций доходов;
 - `DeleteIncomeTransaction.kt` – удаление транзакций доходов по их идентификаторам.
- Операции с транзакциями расходов (CRUD):
 - `AddExpensesTransaction.kt` – добавление новых транзакций расходов в базу данных;
 - `GetExpensesTransactions.kt` – получение всех транзакций расходов с информацией о категориях;
 - `UpdateExpensesTransaction.kt` – обновление существующих транзакций расходов;
 - `DeleteExpensesTransaction.kt` – удаление транзакций расходов по их идентификаторам.

3.5.2.2. Листинги компонентов интерфейса:

3.5.2.2.1. Модель данных:

Листинг 33 – *Operation.kt* – Модель данных операции

```
package models

import java.time.LocalDate

/**
 * Модель данных, представляющая финансовую операцию (доход или расход).
 *
 * @property id Уникальный идентификатор операции в базе данных
 * @property type Тип операции: "Доход" или "Расход"
 * @property amount Сумма операции (в рублях)
 * @property category Категория операции (например, "Еда", "Зарплата",
 "Транспорт")
 * @property date Дата совершения операции
 * @property name Описание операции (необязательное поле)
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
data class Operation(
    val id: Int,
    val type: String,
    val amount: Double,
    val category: String,
    val date: LocalDate,
    val name: String?
)
```

3.5.2.2.2. Репозиторий и вспомогательные утилиты:

Листинг 34 – *FinanceRepository.kt* – Репозиторий данных

```
package repository

import db.database_request.*
import models.Operation
import java.time.LocalDate

/**
 * Репозиторий для работы с финансовыми данными.
 * Служит прослойкой между ViewModel и базой данных.
 * Инкапсулирует все операции по доступу к данным: получение, добавление,
 обновление, удаление.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
class FinanceRepository {
    /**
     * Инициализирует базу данных при создании репозитория.
     * Создает необходимые таблицы, если они еще не существуют.
     */
    init {
        DatabaseInitializer.createTablesIfNotExist()
    }
}
```

```

/**
 * Получает все операции (доходы и расходы) из базы данных.
 *
 * @return Отсортированный по дате (от новых к старым) список всех операций
 */
fun getAllOperations(): List<Operation> {
    println("\n=== REPOSITORY: ПОЛУЧЕНИЕ ВСЕХ ОПЕРАЦИЙ ===")

    // Получаем операции доходов и преобразуем в модель Operation
    val income = GetIncomeTransactions.getAll().map { row ->
        Operation(
            id = row.id,
            type = "Доход",
            amount = row.amount,
            category = row.category,
            date = LocalDate.parse(row.date),
            name = row.name
        )
    }

    // Получаем операции расходов и преобразуем в модель Operation
    val expenses = GetExpensesTransactions.getAll().map { row ->
        Operation(
            id = row.id,
            type = "Расход",
            amount = row.amount,
            category = row.category,
            date = LocalDate.parse(row.date),
            name = row.name
        )
    }

    // Подробная отладка
    println("Доходы получены: ${income.size} операций")
    println("Расходы получены: ${expenses.size} операций")

    if (income.isNotEmpty()) {
        println("Последняя операция дохода: ${income.first()}")
    }
    if (expenses.isNotEmpty()) {
        println("Последняя операция расхода: ${expenses.first()}")
    }

    // Соединяем и сортируем по дате (сначала новые)
    val allOps = (income + expenses).sortedByDescending { it.date }
    println("Загружено операций: доходов=${income.size}, расходов=${expenses.size}, всего=${allOps.size}")
    println("=====\n")
    return allOps
}

/**
 * Получает операции за указанный временной период.
 *
 * @param from Начальная дата периода (включительно)
 * @param to Конечная дата периода (включительно)
 * @return Отфильтрованный список операций в указанном диапазоне дат
 */
fun getOperations(from: LocalDate, to: LocalDate): List<Operation> {
    println("Получение операций с $from по $to")
    return getAllOperations().filter { it.date in from..to }
}

```

```

/**
 * Добавляет новую операцию в базу данных.
 *
 * @param op Операция для добавления
 * @return Добавленная операция с присвоенным ID
 * @throws IllegalArgumentException Если категория не найдена
 * @throws RuntimeException Если не удалось добавить операцию
 */
fun addOperation(op: Operation): Operation {
    println("\n=== ДОБАВЛЕНИЕ ОПЕРАЦИИ ===")
    println("Операция: $op")
    println("Тип операции: ${op.type}")
    println("Категория: ${op.category}")
    println("Сумма: ${op.amount}")
    println("Дата: ${op.date}")
    println("Описание: ${op.name}")
    println("=====\n")

    return if (op.type == "Доход") {
        // Получаем ID категории доходов
        val catId = GetIncomeCategories.getIdByName(op.category)
        ?: throw IllegalArgumentException("Категория доходов '$${op.category}' не найдена")

        // Добавляем транзакцию в базу данных
        val success = AddIncomeTransaction.addIncomeTransaction(
            name = op.name,
            sum = op.amount,
            categoryId = catId,
            date = op.date.toString(),
        )

        if (!success) {
            throw RuntimeException("Не удалось добавить операцию до-
хода")
        }

        // Возвращаем операцию с ID, полученным из базы
        op.copy(id = GetIncomeTransactions.getLastId())
    } else {
        // Получаем ID категории расходов
        val catId = GetExpensesCategories.getIdByName(op.category)
        ?: throw IllegalArgumentException("Категория расходов '$${op.category}' не найдена")

        // Добавляем транзакцию в базу данных
        val success = AddExpensesTransaction.addExpensesTransaction(
            name = op.name,
            sum = op.amount,
            categoryId = catId,
            date = op.date.toString()
        )

        if (!success) {
            throw RuntimeException("Не удалось добавить операцию рас-
хода")
        }

        // Возвращаем операцию с ID, полученным из базы
        op.copy(id = GetExpensesTransactions.getLastId())
    }
}

```

```

/**
 * Обновляет существующую операцию в базе данных.
 *
 * @param op Операция с обновленными данными (должен быть задан коррект-
ный id)
 * @throws RuntimeException Если операция не может быть обновлена
 */
fun updateOperation(op: Operation) {
    println("Обновление операции: $op")
    val success = if (op.type == "Доход") {
        UpdateIncomeTransaction.update(
            transactionId = op.id,
            name = op.name,
            sum = op.amount,
            categoryName = op.category,
            date = op.date.toString()
        )
    } else {
        UpdateExpensesTransaction.update(
            transactionId = op.id,
            name = op.name,
            sum = op.amount,
            categoryName = op.category,
            date = op.date.toString()
        )
    }

    if (!success) {
        throw RuntimeException("Не удалось обновить операцию")
    }
}

/**
 * Удаляет операцию из базы данных по идентификатору и типу.
 *
 * @param id Уникальный идентификатор операции
 * @param type Тип операции ("Доход" или "Расход")
 * @throws RuntimeException Если операция не может быть удалена
 */
fun deleteOperation(id: Int, type: String) {
    println("Удаление операции: id=$id, type=$type")
    val success = if (type == "Доход") {
        DeleteIncomeTransaction.deleteIncomeTransaction(id)
    } else {
        DeleteExpensesTransaction.deleteExpensesTransaction(id)
    }

    if (!success) {
        throw RuntimeException("Не удалось удалить операцию")
    }
}

/**
 * Получает список всех категорий доходов.
 *
 * @return Список названий категорий доходов
 */
fun getIncomeCategories(): List<String> {
    println("Получение категорий доходов...")
    val categories = GetIncomeCategories.getAllNames()
    println("Категории доходов: $categories")
    return categories
}

```

```

/**
 * Получает список всех категорий расходов.
 *
 * @return Список названий категорий расходов
 */
fun getExpenseCategories(): List<String> {
    println("Получение категорий расходов...")
    val categories = GetExpensesCategories.getAllNames()
    println("Категории расходов: $categories")
    return categories
}

/**
 * Добавляет новую категорию в базу данных.
 *
 * @param name Название новой категории
 * @param isIncome Тип категории (true - доходы, false - расходы)
 * @return true если категория успешно добавлена, false в случае ошибки
 */
fun addCategory(name: String, isIncome: Boolean): Boolean {
    println("Добавление категории: name='$name', isIncome=$isIncome")
    return if (isIncome) {
        AddCategory.addIncomeCategory(name)
    } else {
        AddCategory.addExpensesCategory(name)
    }
}

/**
 * Удаляет категорию из базы данных.
 *
 * @param name Название категории для удаления
 * @param isIncome Тип категории (true - доходы, false - расходы)
 * @throws IllegalArgumentException Если категория не найдена
 * @throws RuntimeException Если категория не может быть удалена
 */
fun deleteCategory(name: String, isIncome: Boolean) {
    println("Удаление категории: name='$name', isIncome=$isIncome")
    val id = if (isIncome) GetIncomeCategories.getIdByName(name)
    else GetExpensesCategories.getIdByName(name)

    if (id == null) {
        throw IllegalArgumentException("Категория '$name' не найдена")
    }

    val success = if (isIncome) {
        DeleteCategory.deleteIncomeCategory(id)
    } else {
        DeleteCategory.deleteExpensesCategory(id)
    }

    if (!success) {
        throw RuntimeException("Не удалось удалить категорию '$name'")
    }
}
}

```

Листинг 35 – *LoggingConfig.kt* – Конфигурация логирования

```

package utils

import java.io.PrintStream
import java.nio.charset.StandardCharsets

```

```

/**
 * Утилита для настройки корректного логирования с поддержкой UTF-8.
 * Решает проблемы с отображением русских символов в консоли.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object LoggingConfig {
    /**
     * Настраивает System.out и System.err для корректной работы с UTF-8.
     * Устанавливает системные свойства кодировки для вывода в консоль.
     *
     * Основные действия:
     * - Перенаправляет System.out и System.err в UTF-8 PrintStream
     * - Устанавливает системные свойства file.encoding,
     *   sun.stdout.encoding, sun.stderr.encoding
     *
     * @see PrintStream
     * @see StandardCharsets.UTF_8
     */
    fun setupLogging() {
        // Устанавливаем UTF-8 кодировку для System.out и System.err
        System.setOut(PrintStream(System.out, true, StandardCharsets.UTF_8))
        System.setErr(PrintStream(System.err, true, StandardCharsets.UTF_8))

        // Устанавливаем системную кодировку
        System.setProperty("file.encoding", "UTF-8")
        System.setProperty("sun.stdout.encoding", "UTF-8")
        System.setProperty("sun.stderr.encoding", "UTF-8")

        println("Логирование настроено. Кодировка:
        ${System.getProperty("file.encoding")}")
    }
}

```

3.5.2.2.3. Конфигурация и инициализация БД:

Листинг 36 – DatabaseConfig.kt – Конфигурация подключения к SQLite

```

package db.database_request

import java.io.File

/**
 * Конфигурация подключения к базе данных SQLite.
 * Автоматически находит или создает файл базы данных в доступных локациях.
 *
 * Основные функции:
 * - Определение пути к файлу БД с поиском в нескольких возможных локациях
 * - Автоматическое создание файла БД при его отсутствии
 * - Формирование JDBC URL для подключения к SQLite
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object DatabaseConfig {
    private const val DB_NAME = "Finlytics.db"

    /**
     * URL подключения к базе данных SQLite.
     * Ищет существующий файл БД или создает новый в текущей директории.
     */
}

```

```

    * Поиск выполняется в следующих путях:
    * 1. Текущая директория приложения (user.dir)
    * 2. Путь для разработки (src/main/kotlin/db/database)
    * 3. Альтернативный путь (db/database)
    * 4. Просто имя файла
    *
    * @return JDBC URL для подключения к SQLite
    */
    val DB_URL: String by lazy {
        // Пробуем несколько путей для поиска базы данных
        val possiblePaths = listOf(
            File(System.getProperty("user.dir"), DB_NAME).absolutePath, //
            Текущая директория
            File("src/main/kotlin/db/database", DB_NAME).absolutePath, //
            Путь при разработке
            File("db/database", DB_NAME).absolutePath, //
            Альтернативный путь
            DB_NAME
        )
        // Просто имя файла

        var dbPath = possiblePaths.firstOrNull { File(it).exists() } ?:
possiblePaths[0]

        // Если файл не существует, создадим его в текущей директории
        val dbFile = File(dbPath)
        if (!dbFile.exists()) {
            dbFile.parentFile?.mkdirs()
            try {
                dbFile.createNewFile()
                println("Создана новая база данных: ${dbFile.absolutePath}")
            } catch (e: Exception) {
                println("Ошибка при создании базы данных: ${e.message}")
                // Если не удалось создать по этому пути, пробуем другой
                dbPath = possiblePaths[1]
                val altFile = File(dbPath)
                altFile.parentFile?.mkdirs()
                altFile.createNewFile()
                println("Создана альтернативная база данных:
${altFile.absolutePath}")
            }
        }

        println("Используется база данных: ${dbFile.absolutePath}")
        "jdbc:sqlite:$dbPath"
    }
}

```

Листинг 37 – DatabaseInitializer.kt – Инициализация БД и создание таблиц

```

package db.database_request

import java.sql.DriverManager
import java.io.File

/**
 * Инициализатор базы данных. Создает таблицы при первом запуске приложения.
 * Добавляет категории по умолчанию.
 *
 * Основные функции:
 * - Создание всех необходимых таблиц (если они не существуют)
 * - Настройка внешних ключей
 * - Инициализация стандартных категорий доходов и расходов
 */

```



```

* @author Finlytics Team
* @since 1.0.0
*/
object DatabaseInitializer {
    // Используем конфигурацию для получения пути к БД
    private val DB_URL = DatabaseConfig.DB_URL

    init {
        // Создаем директорию для базы данных, если ее нет
        val dbFile = File("src/main/kotlin/db/database/Finlytics.db")
        dbFile.parentFile.mkdirs()
        println("Путь к БД: ${dbFile.absolutePath}")
        println("БД существует: ${dbFile.exists()}")
    }

    /**
     * Создает все необходимые таблицы в базе данных, если они не суще-
     ствуют.
     * Выполняется при каждом запуске приложения для обеспечения целостности
     схемы БД.
     *
     * Создаваемые таблицы:
     * - Income_categories: категории доходов
     * - expenses_categories: категории расходов
     * - Income_transactions: транзакции доходов
     * - expenses_transactions: транзакции расходов
     *
     * Настроены внешние ключи с ограничением ON DELETE RESTRICT,
     * что предотвращает удаление категорий, связанных с операциями.
     */
    fun createTablesIfNotExist() {
        // SQL-запросы для создания таблиц
        val sql = """
            CREATE TABLE IF NOT EXISTS Income_categories (
                id_income_category INTEGER PRIMARY KEY AUTOINCREMENT,
                income_category_name TEXT NOT NULL UNIQUE
            );
            CREATE TABLE IF NOT EXISTS expenses_categories (
                id_expenses_category INTEGER PRIMARY KEY AUTOINCREMENT,
                expenses_category_name TEXT NOT NULL UNIQUE
            );
            CREATE TABLE IF NOT EXISTS Income_transactions (
                id_income_transaction INTEGER PRIMARY KEY AUTOINCREMENT,
                income_transaction_name TEXT,
                income_transaction_sum REAL NOT NULL,
                id_income_category INTEGER,
                income_transaction_date TEXT NOT NULL,
                FOREIGN KEY (id_income_category) REFERENCES Income_catego-
ries(id_income_category) ON DELETE RESTRICT
            );
            CREATE TABLE IF NOT EXISTS expenses_transactions (
                id_expenses_transaction INTEGER PRIMARY KEY AUTOINCREMENT,
                expenses_transaction_name TEXT,
                expenses_transaction_sum REAL NOT NULL,
                id_expenses_category INTEGER,
                expenses_transaction_date TEXT NOT NULL,
                FOREIGN KEY (id_expenses_category) REFERENCES expenses_cate-
gories(id_expenses_category) ON DELETE RESTRICT
            );
            """.trimIndent()

        try {
            DriverManager.getConnection(DB_URL).use { conn ->
                println("\n=== DATABASE INITIALIZATION ===")
            }
        }
    }
}

```

```

println("Подключение к БД успешно: $DB_URL")

// Проверяем существующие таблицы
val metaData = conn.metaData
val tables = metaData.getTables(null, null, "%", null)
println("Существующие таблицы в БД:")
while (tables.next()) {
    println("    - ${tables.getString("TABLE_NAME")}")
}
tables.close()

conn.createStatement().use { stmt ->
    // Выполняем каждый запрос отдельно (разделены точкой с
запятой)
    sql.split(";").forEach { query ->
        if (query.trim().isEmpty()) {
            stmt.execute(query.trim())
            println("Выполнен запрос:
${query.trim().take(50)}...")
        }
    }
}
println("Таблицы созданы/проверены успешно")
println("=====\n")

// Добавляем категории по умолчанию
DefaultCategories.initializeDefaultCategories()
}
} catch (e: Exception) {
    println("\n!!! ОШИБКА ИНИЦИАЛИЗАЦИИ БД !!!")
    println("Сообщение: ${e.message}")
    println("Трассировка:")
    e.printStackTrace()
    println("=====\n")
}
}
}

```

Листинг 38 – *DefaultCategories.kt* – Заполнение категорий по умолчанию

```

package db.database_request

/**
 * Предоставляет стандартные категории доходов и расходов.
 * Инициализирует их при первом запуске приложения.
 *
 * Стандартные категории:
 * - Доходы: Зарплата, Стипендия, Фриланс, Инвестиции, Подарок
 * - Расходы: Еда, Транспорт, Жилье, Развлечения, Образование, Здоровье,
Одежда, Коммунальные услуги
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object DefaultCategories {
    // Стандартные категории доходов
    private val defaultIncomeCategories = listOf(
        "Зарплата",
        "Стипендия",
        "Фриланс",
        "Инвестиции",
        "Подарок"
    )
}

```

```

// Стандартные категории расходов
private val defaultExpenseCategories = listOf(
    "Еда",
    "Транспорт",
    "Жилье",
    "Развлечения",
    "Образование",
    "Здоровье",
    "Одежда",
    "Коммунальные услуги"
)

/**
 * Добавляет стандартные категории в базу данных, если их еще нет.
 * Вызывается при инициализации базы данных.
 * Каждая категория добавляется только если она еще не существует в
базе.
 */
fun initializeDefaultCategories() {
    println("Инициализация категорий по умолчанию...")

    // Добавляем категории доходов
    var incomeCount = 0
    defaultIncomeCategories.forEach { category ->
        if (!incomeCategoryExists(category)) {
            val success = AddCategory.addIncomeCategory(category)
            if (success) incomeCount++
        }
    }
    println("Добавлено категорий доходов: $incomeCount")

    // Добавляем категории расходов
    var expenseCount = 0
    defaultExpenseCategories.forEach { category ->
        if (!expenseCategoryExists(category)) {
            val success = AddCategory.addExpensesCategory(category)
            if (success) expenseCount++
        }
    }
    println("Добавлено категорий расходов: $expenseCount")
}

/**
 * Проверяет существование категории доходов.
 *
 * @param name Название категории для проверки
 * @return true если категория существует, false в противном случае
 */
private fun incomeCategoryExists(name: String): Boolean {
    return GetIncomeCategories.getAllNames().any { it.equals(name,
ignoreCase = true) }
}

/**
 * Проверяет существование категории расходов.
 *
 * @param name Название категории для проверки
 * @return true если категория существует, false в противном случае
 */
private fun expenseCategoryExists(name: String): Boolean {
    return GetExpensesCategories.getAllNames().any { it.equals(name,
ignoreCase = true) }
}
}

```

3.5.2.2.4. Операции с категориями (CRUD):

Листинг 39 – *GetIncomeCategories.kt* – Получение категорий доходов

```
package db.database_request

import java.sql.DriverManager

/**
 * Объект для получения категорий доходов из базы данных.
 * Предоставляет методы для получения списка категорий и поиска по имени.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object GetIncomeCategories {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Получает все категории доходов из базы данных.
     * Категории возвращаются отсортированными по названию.
     *
     * @return Список пар (id категории, название категории)
     */
    fun getAll(): List<Pair<Int, String>> {
        val list = mutableListOf<Pair<Int, String>>()
        val sql = "SELECT id_income_category, income_category_name FROM Income_categories ORDER BY income_category_name"
        try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.createStatement().use { stmt ->
                    val rs = stmt.executeQuery(sql)
                    while (rs.next()) {
                        list.add(rs.getInt(1) to rs.getString(2))
                    }
                    println("Загружено категорий доходов: ${list.size}")
                }
            }
        } catch (e: Exception) {
            println("Ошибка при получении категорий доходов: ${e.message}")
        }
        return list
    }

    /**
     * Получает список названий всех категорий доходов.
     *
     * @return Список названий категорий
     */
    fun getAllNames() = getAll().map { it.second }

    /**
     * Получает идентификатор категории доходов по её названию.
     *
     * @param name Название категории
     * @return Идентификатор категории или null, если категория не найдена
     */
    fun getIdByName(name: String) = getAll().find { it.second == name }?.first
}
```

Листинг 40 – *GetExpensesCategories.kt* – Получение категорий расходов

```
package db.database_request

import java.sql.DriverManager

/**
 * Объект для получения категорий расходов из базы данных.
 * Предоставляет методы для получения списка категорий и поиска по имени.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object GetExpensesCategories {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Получает все категории расходов из базы данных.
     * Категории возвращаются отсортированными по названию.
     *
     * @return Список пар (id категории, название категории)
     */
    fun getAll(): List<Pair<Int, String>> {
        val list = mutableListOf<Pair<Int, String>>()
        val sql = "SELECT id_expenses_category, expenses_category_name FROM expenses_categories ORDER BY expenses_category_name"
        try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.createStatement().use { stmt ->
                    val rs = stmt.executeQuery(sql)
                    while (rs.next()) {
                        list.add(rs.getInt(1) to rs.getString(2))
                    }
                    println("Загружено категорий расходов: ${list.size}")
                }
            }
        } catch (e: Exception) {
            println("Ошибка при получении категорий расходов: ${e.message}")
        }
        return list
    }

    /**
     * Получает список названий всех категорий расходов.
     *
     * @return Список названий категорий
     */
    fun getAllNames() = getAll().map { it.second }

    /**
     * Получает идентификатор категории расходов по её названию.
     *
     * @param name Название категории
     * @return Идентификатор категории или null, если категория не найдена
     */
    fun getIdByName(name: String) = getAll().find { it.second == name }?.first
}
```

Листинг 41 – *AddCategory.kt* – Добавление новых категорий

```
package db.database_request
```

```

import java.sql.DriverManager

/**
 * Объект для добавления новых категорий в базу данных.
 * Поддерживает добавление как категорий доходов, так и категорий расходов.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object AddCategory {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Добавляет новую категорию доходов в таблицу Income_categories.
     *
     * @param name Название категории (не должно быть пустым)
     * @return true если категория успешно добавлена, false в случае ошибки
     или пустого имени
     */
    fun addIncomeCategory(name: String): Boolean {
        println("Добавление категории доходов: '$name'")
        if (name.isBlank()) {
            println("Имя категории пустое")
            return false
        }

        val sql = "INSERT INTO Income_categories (income_category_name)
VALUES (?)"
        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setString(1, name.trim())
                    val rowsAffected = pstmt.executeUpdate()
                    val result = rowsAffected > 0
                    println("Категория доходов '$name' добавлена. Затронуто
строк: $rowsAffected, успех: $result")
                    result
                }
            }
        } catch (e: Exception) {
            println("Ошибка при добавлении категории доходов '$name':
${e.message}")
            e.printStackTrace()
            false
        }
    }

    /**
     * Добавляет новую категорию расходов в таблицу expenses_categories.
     *
     * @param name Название категории (не должно быть пустым)
     * @return true если категория успешно добавлена, false в случае ошибки
     или пустого имени
     */
    fun addExpensesCategory(name: String): Boolean {
        println("Добавление категории расходов: '$name'")
        if (name.isBlank()) {
            println("Имя категории пустое")
            return false
        }

        val sql = "INSERT INTO expenses_categories (expenses_category_name)
VALUES (?)"
        return try {

```

```

        DriverManager.getConnection(DB_URL).use { conn ->
            conn.prepareStatement(sql).use { pstmt ->
                pstmt.setString(1, name.trim())
                val rowsAffected = pstmt.executeUpdate()
                val result = rowsAffected > 0
                println("Категория расходов '$name' добавлена. Затронуто
строк: $rowsAffected, успех: $result")
                result
            }
        }
    } catch (e: Exception) {
        println("Ошибка при добавлении категории расходов '$name':
${e.message}")
        e.printStackTrace()
        false
    }
}
}

```

Листинг 42 – DeleteCategory.kt – Удаление категорий

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для удаления категорий из базы данных.
 * Поддерживает удаление как категорий доходов, так и категорий расходов.
 *
 * Важно: удаление категории возможно только если с ней не связано ни одной
операции.
 * Ограничение ON DELETE RESTRICT в схеме БД предотвращает удаление катего-
рий,
 * связанных с существующими транзакциями.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object DeleteCategory {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Удаляет категорию доходов по её идентификатору.
     *
     * @param categoryId Идентификатор удаляемой категории (должен быть > 0)
     * @return true если категория успешно удалена, false в случае ошибки
или некорректного ID
     */
    fun deleteIncomeCategory(categoryId: Int): Boolean {
        if (categoryId <= 0) return false
        val sql = "DELETE FROM Income_categories WHERE id_income_category =
?"

        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setInt(1, categoryId)
                    val rowsAffected = pstmt.executeUpdate()
                    println("Удалена категория доходов с ID $categoryId. За-
тронуто строк: $rowsAffected")
                    rowsAffected > 0
                }
            }
        } catch (e: Exception) {
            println("Ошибка при удалении категории доходов: ${e.message}")
        }
    }
}

```

```

        e.printStackTrace()
        false
    }
}

/**
 * Удаляет категорию расходов по её идентификатору.
 *
 * @param categoryId Идентификатор удаляемой категории (должен быть > 0)
 * @return true если категория успешно удалена, false в случае ошибки
или некорректного ID
 */
fun deleteExpensesCategory(categoryId: Int): Boolean {
    if (categoryId <= 0) return false
    val sql = "DELETE FROM expenses_categories WHERE id_expenses_cate-
gory = ?"
    return try {
        DriverManager.getConnection(DB_URL).use { conn ->
            conn.prepareStatement(sql).use { pstmt ->
                pstmt.setInt(1, categoryId)
                val rowsAffected = pstmt.executeUpdate()
                println("Удалена категория расходов с ID $categoryId.
Затронуто строк: $rowsAffected")
                rowsAffected > 0
            }
        }
    } catch (e: Exception) {
        println("Ошибка при удалении категории расходов: ${e.message}")
        e.printStackTrace()
        false
    }
}
}

```

3.5.2.2.5. Операции с транзакциями доходов (CRUD):

Листинг 43 – *AddIncomeTransaction.kt* – Добавление транзакций доходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для добавления транзакций доходов в базу данных.
 * Реализует операцию INSERT для таблицы Income_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object AddIncomeTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Добавляет новую транзакцию дохода в базу данных.
     *
     * @param name Название транзакции (может быть null)
     * @param sum Сумма транзакции (должна быть положительной)
     * @param categoryId Идентификатор категории дохода (должен быть > 0)
     * @param date Дата транзакции в формате строки (ГГГГ-ММ-ДД)
     * @return true если транзакция успешно добавлена, false в случае ошибки
или некорректных данных
     */
}

```



```

*/
fun addIncomeTransaction(name: String?, sum: Double, categoryId: Int,
date: String): Boolean {
    if (sum <= 0 || categoryId <= 0) return false
    val sql = """
        INSERT INTO Income_transactions
        (income_transaction_name, income_transaction_sum,
id_income_category, income_transaction_date)
        VALUES (?, ?, ?, ?)
    """.trimIndent()
    return try {
        DriverManager.getConnection(DB_URL).use { conn ->
            conn.prepareStatement(sql).use { pstmt ->
                pstmt.setString(1, name)
                pstmt.setDouble(2, sum)
                pstmt.setInt(3, categoryId)
                pstmt.setString(4, date)
                pstmt.executeUpdate() > 0
            }
        }
    } catch (e: Exception) {
        false
    }
}
}

```

Листинг 44 – *GetIncomeTransactions.kt* – Получение транзакций доходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для получения транзакций доходов из базы данных.
 * Выполняет JOIN с таблицей категорий для получения полной информации.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object GetIncomeTransactions {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Модель строки результата запроса транзакций доходов.
     *
     * @property id Уникальный идентификатор транзакции
     * @property name Название транзакции (может быть null)
     * @property amount Сумма транзакции
     * @property date Дата транзакции в формате строки
     * @property category Название категории транзакции
     */
    data class Row(
        val id: Int,
        val name: String?,
        val amount: Double,
        val date: String,
        val category: String
    )

    /**
     * Получает все транзакции доходов из базы данных.
     * Транзакции возвращаются в порядке убывания даты (от новых к старым).
     *
     * @return Список всех транзакций доходов с информацией о категориях
     */
}

```

```

    */
    fun getAll(): List<Row> {
        val list = mutableListOf<Row>()
        val sql = """
            SELECT it.id_income_transaction, it.income_transaction_name,
            it.income_transaction_sum,
            it.income_transaction_date, ic.income_category_name
            FROM Income_transactions it
            JOIN Income_categories ic ON it.id_income_category = ic.id_in-
            come_category
            ORDER BY it.income_transaction_date DESC
        """.trimIndent()
        try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.createStatement().use { stmt ->
                    val rs = stmt.executeQuery(sql)
                    while (rs.next()) {
                        list.add(Row(rs.getInt(1), rs.getString(2), rs.get-
                        Double(3), rs.getString(4), rs.getString(5)))
                    }
                }
            }
        } catch (e: Exception) { }
        return list
    }

    /**
     * Получает максимальный идентификатор транзакции дохода.
     * Используется для получения ID только что добавленной транзакции.
     *
     * @return Максимальный ID или 0, если транзакций нет
     */
    fun getLastId() = getAll().maxOrNull { it.id } ?: 0
}

```

Листинг 45 – UpdateIncomeTransaction.kt – Обновление транзакций доходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для обновления транзакций доходов в базе данных.
 * Выполняет операцию UPDATE для таблицы Income_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object UpdateIncomeTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Обновляет существующую транзакцию дохода.
     *
     * @param transactionId Идентификатор обновляемой транзакции (должен
     * быть > 0)
     * @param name Новое название транзакции (может быть null)
     * @param sum Новая сумма транзакции (должна быть положительной)
     * @param categoryName Новое название категории (должна существовать в
     * базе)
     * @param date Новая дата транзакции в формате строки (ГГГГ-ММ-ДД)
     * @return true если транзакция успешно обновлена, false в случае ошибки
     * или некорректных данных
     */
}

```

```

fun update(
    transactionId: Int,
    name: String? = null,
    sum: Double,
    categoryName: String,
    date: String
): Boolean {
    val categoryId = GetIncomeCategories.getIdByName(categoryName) ?:
return false
    if (transactionId <= 0 || sum <= 0 || categoryId <= 0) return false

    val sql = """
        UPDATE Income_transactions
        SET income_transaction_name = ?,
            income_transaction_sum = ?,
            id_income_category = ?,
            income_transaction_date = ?
        WHERE id_income_transaction = ?
    """.trimIndent()

    return try {
        DriverManager.getConnection(DB_URL).use { conn ->
            conn.prepareStatement(sql).use { pstmt ->
                pstmt.setString(1, name)
                pstmt.setDouble(2, sum)
                pstmt.setInt(3, categoryId)
                pstmt.setString(4, date)
                pstmt.setInt(5, transactionId)
                pstmt.executeUpdate() > 0
            }
        }
    } catch (e: Exception) {
        false
    }
}
}

```

Листинг 46 – DeleteIncomeTransaction.kt – Удаление транзакций доходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для удаления транзакций доходов из базы данных.
 * Выполняет операцию DELETE для таблицы Income_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object DeleteIncomeTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Удаляет транзакцию дохода по её идентификатору.
     *
     * @param transactionId Идентификатор удаляемой транзакции (должен быть
     > 0)
     * @return true если транзакция успешно удалена, false в случае ошибки
     или некорректного ID
     */
    fun deleteIncomeTransaction(transactionId: Int): Boolean {
        if (transactionId <= 0) return false
    }
}

```

```

        val sql = "DELETE FROM Income_transactions WHERE
id_income_transaction = ?"

        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setInt(1, transactionId)
                    pstmt.executeUpdate() > 0
                }
            }
        } catch (e: Exception) {
            false
        }
    }
}

```

3.5.2.2.6. Операции с транзакциями расходов (CRUD):

Листинг 47 – *AddExpensesTransaction.kt* – Добавление транзакций расходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для добавления транзакций расходов в базу данных.
 * Реализует операцию INSERT для таблицы expenses_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object AddExpensesTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Добавляет новую транзакцию расхода в базу данных.
     *
     * @param name Название транзакции (может быть null)
     * @param sum Сумма транзакции (должна быть положительной)
     * @param categoryId Идентификатор категории расхода (должен быть > 0)
     * @param date Дата транзакции в формате строки (ГГГГ-ММ-ДД)
     * @return true если транзакция успешно добавлена, false в случае ошибки
     * или некорректных данных
     */
    fun addExpensesTransaction(name: String?, sum: Double, categoryId: Int,
date: String): Boolean {
        if (sum <= 0 || categoryId <= 0) return false
        val sql = """
            INSERT INTO expenses_transactions
            (expenses_transaction_name, expenses_transaction_sum,
id_expenses_category, expenses_transaction_date)
            VALUES (?, ?, ?, ?)
        """.trimIndent()
        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setString(1, name)
                    pstmt.setDouble(2, sum)
                    pstmt.setInt(3, categoryId)
                    pstmt.setString(4, date)
                    pstmt.executeUpdate() > 0
                }
            }
        }
    }
}

```

```

    }
    } catch (e: Exception) {
        false
    }
}
}

```

Листинг 48 – *GetExpensesTransactions.kt* – Получение транзакций расходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для получения транзакций расходов из базы данных.
 * Выполняет JOIN с таблицей категорий для получения полной информации.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object GetExpensesTransactions {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Модель строки результата запроса транзакций расходов.
     *
     * @property id Уникальный идентификатор транзакции
     * @property name Название транзакции (может быть null)
     * @property amount Сумма транзакции
     * @property date Дата транзакции в формате строки
     * @property category Название категории транзакции
     */
    data class Row(
        val id: Int,
        val name: String?,
        val amount: Double,
        val date: String,
        val category: String
    )

    /**
     * Получает все транзакции расходов из базы данных.
     * Транзакции возвращаются в порядке убывания даты (от новых к старым).
     *
     * @return Список всех транзакций расходов с информацией о категориях
     */
    fun getAll(): List<Row> {
        val list = mutableListOf<Row>()
        val sql = """
            SELECT et.id_expenses_transaction, et.expenses_transaction_name,
            et.expenses_transaction_sum,
            et.expenses_transaction_date, ec.expenses_category_name
            FROM expenses_transactions et
            JOIN expenses_categories ec ON et.id_expenses_category =
            ec.id_expenses_category
            ORDER BY et.expenses_transaction_date DESC
        """.trimIndent()
        try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.createStatement().use { stmt ->
                    val rs = stmt.executeQuery(sql)
                    while (rs.next()) {
                        list.add(Row(rs.getInt(1), rs.getString(2), rs.getDouble(3), rs.getString(4), rs.getString(5)))
                    }
                }
            }
        }
    }
}

```

```

    }
    }
    } catch (e: Exception) { }
    return list
}

/**
 * Получает максимальный идентификатор транзакции расхода.
 * Используется для получения ID только что добавленной транзакции.
 *
 * @return Максимальный ID или 0, если транзакций нет
 */
fun getLastId() = getAll().maxOrNull { it.id } ?: 0
}

```

Листинг 49 – UpdateExpensesTransaction.kt – Обновление транзакций расходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для обновления транзакций расходов в базе данных.
 * Выполняет операцию UPDATE для таблицы expenses_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object UpdateExpensesTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Обновляет существующую транзакцию расхода.
     *
     * @param transactionId Идентификатор обновляемой транзакции (должен
     * быть > 0)
     * @param name Новое название транзакции (может быть null)
     * @param sum Новая сумма транзакции (должна быть положительной)
     * @param categoryName Новое название категории (должна существовать в
     * базе)
     * @param date Новая дата транзакции в формате строки (ГГГГ-ММ-ДД)
     * @return true если транзакция успешно обновлена, false в случае ошибки
     * или некорректных данных
     */
    fun update(
        transactionId: Int,
        name: String? = null,
        sum: Double,
        categoryName: String,
        date: String
    ): Boolean {
        val categoryId = GetExpensesCategories.getIdByName(categoryName) ?:
        return false
        if (transactionId <= 0 || sum <= 0 || categoryId <= 0) return false

        val sql = """
            UPDATE expenses_transactions
            SET expenses_transaction_name = ?,
                expenses_transaction_sum = ?,
                id_expenses_category = ?,
                expenses_transaction_date = ?
            WHERE id_expenses_transaction = ?
        """.trimIndent()
    }
}

```

```

        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setString(1, name)
                    pstmt.setDouble(2, sum)
                    pstmt.setInt(3, categoryId)
                    pstmt.setString(4, date)
                    pstmt.setInt(5, transactionId)
                    pstmt.executeUpdate() > 0
                }
            }
        } catch (e: Exception) {
            false
        }
    }
}

```

Листинг 501 – DeleteExpensesTransaction.kt – Удаление транзакций расходов

```

package db.database_request

import java.sql.DriverManager

/**
 * Объект для удаления транзакций расходов из базы данных.
 * Выполняет операцию DELETE для таблицы expenses_transactions.
 *
 * @author Finlytics Team
 * @since 1.0.0
 */
object DeleteExpensesTransaction {
    private val DB_URL = DatabaseConfig.DB_URL

    /**
     * Удаляет транзакцию расхода по её идентификатору.
     *
     * @param transactionId Идентификатор удаляемой транзакции (должен быть
     * > 0)
     * @return true если транзакция успешно удалена, false в случае ошибки
     * или некорректного ID
     */
    fun deleteExpensesTransaction(transactionId: Int): Boolean {
        if (transactionId <= 0) return false

        val sql = "DELETE FROM expenses_transactions WHERE
id_expenses_transaction = ?"

        return try {
            DriverManager.getConnection(DB_URL).use { conn ->
                conn.prepareStatement(sql).use { pstmt ->
                    pstmt.setInt(1, transactionId)
                    pstmt.executeUpdate() > 0
                }
            }
        } catch (e: Exception) {
            false
        }
    }
}

```

3.6. Документирование кода

3.6.1. Документация разработчика

Документирование кода выполнено с использованием KDoc - стандартной системы документации для Kotlin. Все публичные классы, функции и свойства имеют подробные комментарии с описанием назначения, параметров, возвращаемых значений и возможных исключений.

Для обобщения всей документации разработчика и представления в едином HTML-файле использовался инструмент генерации документации Dokka. Скриншот с одной из страниц сгенерированной документации разработчика представлен на Рисунок 3.2.

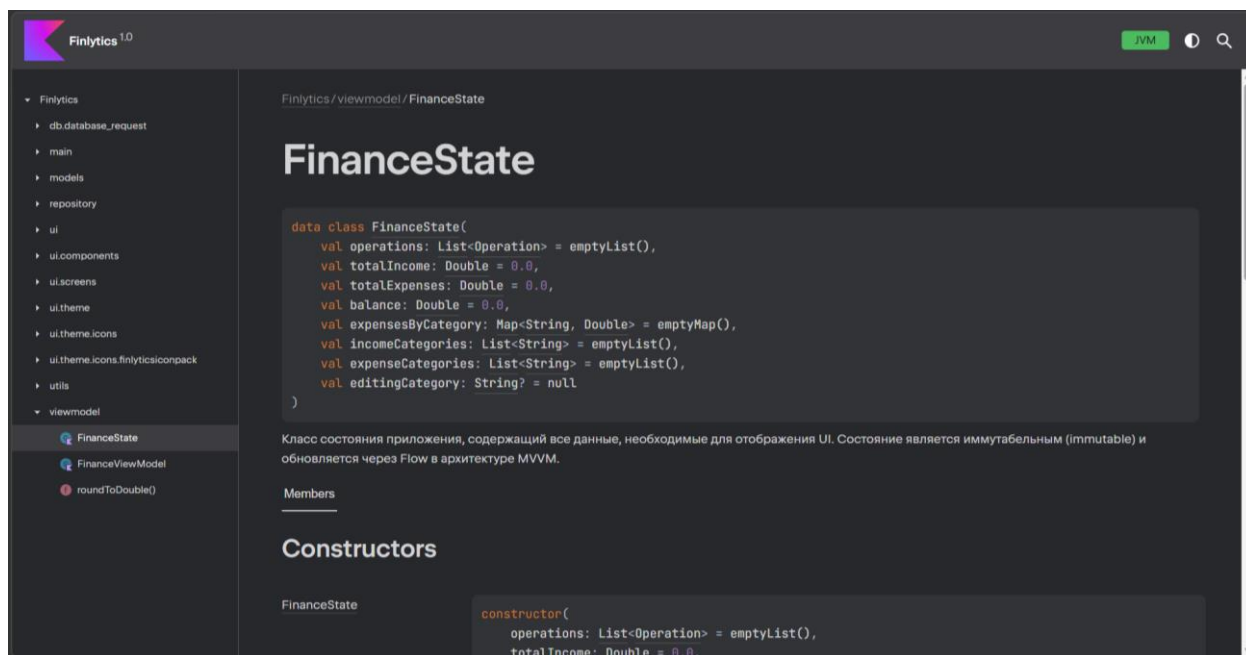


Рисунок 3.2 — Страница документации разработчика

3.6.2. Документация пользователя

Для обеспечения удобства работы с приложением «Finlytics» и предоставления пользователю полной информации о его функциональных возможностях, была разработана многоуровневая система документации, включающая как встроенные подсказки в интерфейсе, так и развёрнутую справочную систему в виде вики на платформе GitHub.

Основные элементы документации:

1. Встроенная помощь в интерфейсе (UI)

Все ключевые элементы управления приложения (кнопки, поля ввода, фильтры) снабжены всплывающими подсказками (ToolTips), поясняющими их назначение. При вводе данных система предоставляет понятные сообщения об ошибках (например, при указании некорректной суммы или даты), что позволяет пользователю интуитивно освоить работу с приложением.

2. Вики GitHub (GitHub Wiki)

Основным источником полной и структурированной информации о приложении является вики-репозиторий, доступный по адресу: <https://github.com/Hohrandrey/Finlytics/wiki>. Скриншот созданной документации представлен на Рисунок 3.3.

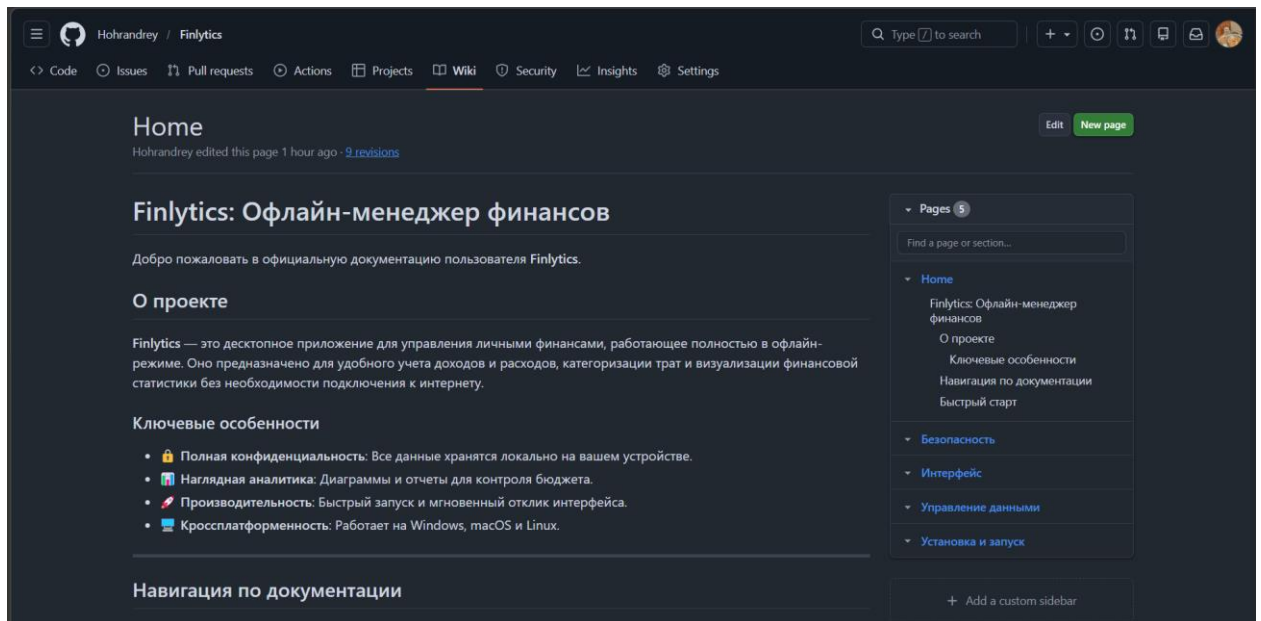


Рисунок 3.3 — Первая страница Wiki на GitHub

4. Тестирование приложения

4.1. Тестовые сценарии (Test Cases)

Таблица 4.1 — Тестовый сценарий 1

Название:	ТС-01 Добавление новой операции (позитивный)	
Функция:	Создание записи о расходе	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
Запуск и инициализация БД	Приложение запущено. База данных инициализирована (есть хотя бы одна категория расходов, например "Еда").	Пройден
Шаги теста		
На нижней панели навигации нажать кнопку "Добавить".	Открывается диалоговое окно "Новая операция".	Пройден
В диалоге выбрать тип "Расход" (если не выбран).	Кнопка "Расход" подсвечивается красным цветом.	Пройден
В поле "Сумма (₽)" ввести "500".	В поле отображается "500".	Пройден
В поле "Дата" оставить текущую дату (или выбрать "Сегодня").	Поле заполнено датой в формате ГГГГ-ММ-ДД.	Пройден
В списке категорий выбрать "Еда".	Строка "Еда" подсвечивается, появляется иконка галочки.	Пройден
В поле "Описание" ввести "Обед".	Поле заполнено текстом "Обед".	Пройден
Нажать кнопку "Создать операцию".	Диалоговое окно закрывается. На главном экране ("Обзор") общая сумма расходов увеличивается на 500 руб., баланс уменьшается на 500 руб. В истории операций появляется новая запись с параметрами: Расход, 500 руб., Еда, Обед.	Пройден

Таблица 4.2 — Тестовый сценарий 2

Название:	ТС-02 Добавление операции с некорректной суммой (негативный)	
Функция:	Валидация вводимых данных	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
Запуск и нажатие «Добавить операцию»	Приложение запущено. Открыт диалог добавления операции ("Новая операция").	Пройден
Шаги теста		

В поле "Сумма (Р)" ввести "-100".	В поле отображается "-100".	Пройден
Заполнить остальные поля корректно (категория, дата).	Все остальные поля заполнены валидными данными.	Пройден
Нажать кнопку "Создать операцию".	Операция не создается. Под полем "Сумма" появляется сообщение об ошибке "Введите сумму > 0". Диалоговое окно не закрывается.	Пройден

Таблица 4.3 — Тестовый сценарий 3

Название:	ТС-03 Редактирование операции	
Функция:	Изменение существующей записи	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
На экране «История» есть операция	В систему добавлена операция "Обед" на 500 руб. Открыт экран "История".	Пройден
Шаги теста		
Найти операцию "Обед" и нажать на желтую кнопку с иконкой карандаша.	Открывается диалоговое окно "Редактирование операции" со всеми полями, заполненными данными этой операции.	Пройден
Изменить сумму на "750".	Поле суммы отображает "750".	Пройден
Изменить категорию на "Транспорт" (выбрать из списка).	В списке категорий выбрана "Транспорт".	Пройден
Нажать кнопку "Сохранить изменения".	Диалоговое окно закрывается. В списке истории у операции "Обед" сумма меняется на 750 руб., категория на "Транспорт". На главном экране расходы и баланс пересчитываются корректно.	Пройден

Таблица 4.4 — Тестовый сценарий 4

Название:	ТС-04 Удаление операции	
Функция:	Удаление существующей записи	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
В системе существует любая операция.	В систему добавлена операция "Обед" на 500 руб. Открыт экран "История".	Пройден
Шаги теста		
Найти операцию и нажать на красную кнопку с иконкой корзины.	Открывается диалоговое окно "Подтверждение удаления" с информацией об операции.	Пройден
В диалоге подтверждения нажать кнопку "Удалить".	Диалог подтверждения закрывается. Операция исчезает из списка истории.	Пройден

	На главном экране общая сумма доходов/расходов и баланс пересчитываются.	
--	--	--

Таблица 4.5 — Тестовый сценарий 5

Название:	ТС-05 Создание новой категории расходов	
Функция:	Добавление пользовательской категории	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
Открыть «Настройки»	Открыт экран "Настройки". Видна панель "Категории расходов"..	Пройден
Шаги теста		
В правой панели "Категории расходов" нажать синюю кнопку "+".	Открывается диалоговое окно "Добавить категорию расходов".	Пройден
В диалоге ввести название "Кафе".	Поле ввода содержит текст "Кафе".	Пройден
Нажать кнопку "Добавить".	Диалог закрывается. В списке категорий расходов появляется новая запись "Кафе". При создании новой операции расхода категория "Кафе" доступна для выбора.	Пройден

Таблица 4.6 — Тестовый сценарий 6

Название:	ТС-06 Удаление используемой категории (негативный)	
Функция:	Защита целостности данных	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
Существует операция с категорией "Еда".	В систему добавлена операция "Обед" на 500 руб. с категорией "Еда". Открыт экран "История".	Пройден
Шаги теста		
В панели "Категории расходов" найти категорию "Еда".	Категория "Еда" отображается в списке.	Пройден
Нажать красную кнопку с иконкой корзины у категории "Еда".	Открывается диалоговое окно "Подтверждение удаления" с предупреждением: "Существуют операции с этой категорией...". Кнопка "Удалить" неактивна.	Пройден
Нажать кнопку "Отмена".	Диалог закрывается. Категория "Еда" остается в списке.	Пройден

Таблица 4.7 — Тестовый сценарий 7

Название:	ТС-07 Фильтрация данных по периоду
Функция:	Аналитика за выбранный промежуток

Действие	Ожидаемый результат	Результаты теста
Предусловие		
В системе есть операции за разные даты	В системе есть операции за разные даты (например, за Март и Апрель 2024). Открыт экран "Обзор".	Пройден
Шаги теста		
В блоке "Период времени" выбрать "Месяц".	Кнопка "Месяц" подсвечивается синим цветом.	Пройден
С помощью стрелок навигации выбрать месяц "Март 2024".	В поле отображается "03.2024".	Пройден
Посмотреть на блоки "Доходы", "Расходы", "Баланс".	Цифры в блоках пересчитаны только за период с 01.03.2024 по 31.03.2024.	Пройден

Таблица 4.8 — Тестовый сценарий 8

Название:	ТС-08 Переключение между доходами и расходами	
Функция:	Переключение режима диаграммы	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
В БД есть записи о доходах и расходах	На главном экране ("Обзор") отображается статистика за выбранный период.	Пройден
Шаги теста		
Нажать кнопку "Доходы" (зеленая).	Кнопка "Доходы" подсвечивается зеленым. Круговая диаграмма перестраивается, показывая структуру доходов по категориям. Список категорий справа меняется на список категорий доходов с суммами и процентами. Цветовое оформление диаграмм меняется на оттенки синего/зеленого.	Пройден
Нажать кнопку "Расходы" (зеленая).	Кнопка "Расходы" подсвечивается красным. Круговая диаграмма перестраивается, показывая структуру расходов по категориям. Список категорий справа меняется на список категорий расходов с суммами и процентами. Цветовое оформление диаграмм меняется на оттенки красного.	Пройден

Таблица 4.9 — Тестовый сценарий 9

Название:	ТС-09 Валидация даты в будущее (негативный)	
Функция:	Ограничение ввода дат	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
Нажать «Добавить операцию».	Открыт диалог добавления операции.	Пройден
Шаги теста		

В поле "Дата" ввести дату, которая на 2 дня больше текущей (например, послезавтра).	Поле содержит дату, например, "2026-04-04" (если сегодня 02.04.2026).	Пройден
Заполнить остальные поля корректно (сумма, категория).	Все остальные поля валидны.	Пройден
Нажать кнопку "Создать операцию".	Операция не создается. Под полем "Дата" появляется сообщение об ошибке: "Дата не может быть более чем на 1 день в будущем".	Пройден

Таблица 4.10 — Тестовый сценарий 10

Название:	ТС-10 Работа прогресс-бара "Гонка с призраком"	
Функция:	Сравнение с предыдущим периодом	
Действие	Ожидаемый результат	Результаты теста
Предусловие		
В БД есть записи о расходах	На главном экране выбран период "Месяц". Есть данные за текущий месяц (расходы 5000 руб.) и за прошлый месяц (расходы 10000 руб.).	Пройден
Шаги теста		
Убедиться, что на экране присутствует компонент "Гонка с призраком".	Компонент отображается над блоком статистики.	Пройден
Проверить заполнение прогресс-бара и цвет.	Прогресс-бар заполнен на 50% (т.к. $5000 / 10000 = 0.5$). Цвет прогресс-бара — зеленый (менее 80%). Отображается текст: "Потрачено: 5000 Р", "Лимит прошлого периода: 10000 Р".	Пройден

4.2. Инструменты для проведения тестирования

Для проекта "Finlytics" были выбраны следующие инструменты тестирования:

1. JUnit (среда выполнения тестов):
 - Назначение: Фреймворк для модульного тестирования кода на Kotlin и Java.
 - Применение: Будет использован для написания и выполнения модульных тестов (Unit Tests) для backend-логики приложения. С его помощью можно тестировать методы FinanceRepository,

классы для работы с БД (AddIncomeTransaction, GetExpensesCategories), валидаторы данных и другие компоненты в изоляции от UI.

- Обоснование: Является стандартом де-факто для тестирования на JVM, обеспечивает простую аннотационную модель для создания тестов, широко поддерживается в среде IntelliJ IDEA.

2. Swing UI Testers / Compose UI Test Framework:

- Назначение: Библиотеки для автоматизированного тестирования пользовательского интерфейса.
- Применение: Для Compose for Desktop существуют встроенные инструменты тестирования UI (похожие на Compose UI Test для Android). Они позволяют эмулировать действия пользователя (клики, ввод текста), проверять состояние UI-компонентов (видимость, текст, цвет) и выполнять интеграционные тесты всего приложения.
- Обоснование: Критически важно для проверки сценариев, описанных в тестовых случаях (ТС-01 - ТС-10), таких как открытие диалогов, нажатие кнопок, отображение ошибок и корректность изменений интерфейса после операций.

3. Apache JMeter:

- Назначение: Инструмент для нагрузочного и стресс-тестирования.
- Применение: Для оценки нефункциональных требований к производительности. JMeter может эмулировать множество одновременных запросов к БД (хотя приложение офлайн-однопользовательское), чтобы проверить время отклика при фильтрации большого количества операций (требование: не более 500 мс).
- Обоснование: Позволяет получить объективные и измеримые результаты времени отклика системы при различных сценариях нагрузки.

4. OWASP ZAP (Zed Attack Proxy):

- Назначение: Инструмент для тестирования безопасности веб- и десктоп-приложений.
- Применение: Несмотря на то, что приложение офлайн, оно использует SQLite. OWASP ZAP можно использовать для проверки уязвимостей, таких как SQL-инъекции, через перехват и модификацию вызовов JDBC (на этапе отладки) или анализ кода на предмет использования небезопасных методов формирования запросов. Требование об использовании параметризованных запросов (PreparedStatement) может быть проверено через статический анализ кода, но ZAP поможет подтвердить это динамически.
- Обоснование: Соответствует требованию безопасности о защите от SQL-инъекций и является стандартом в области автоматизированного поиска уязвимостей.

4.3. Тестирование пользовательского интерфейса

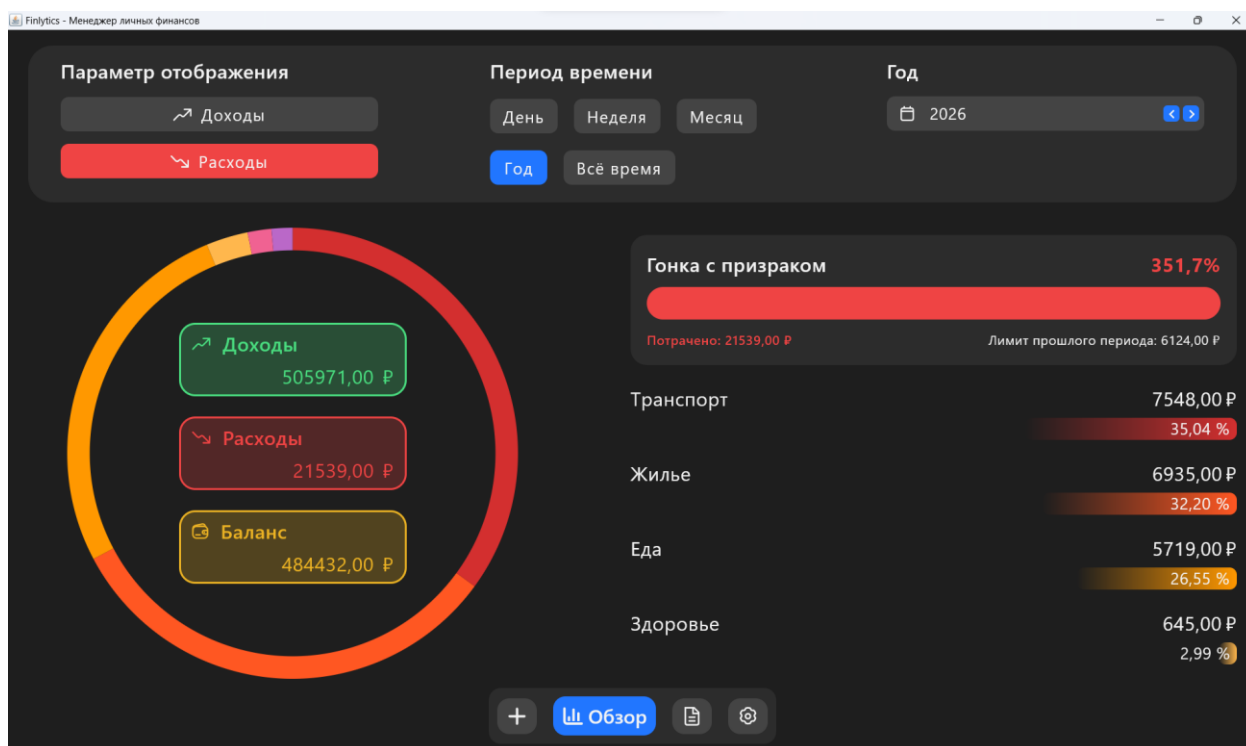


Рисунок 4.1 — Главный экран приложения (расходы)

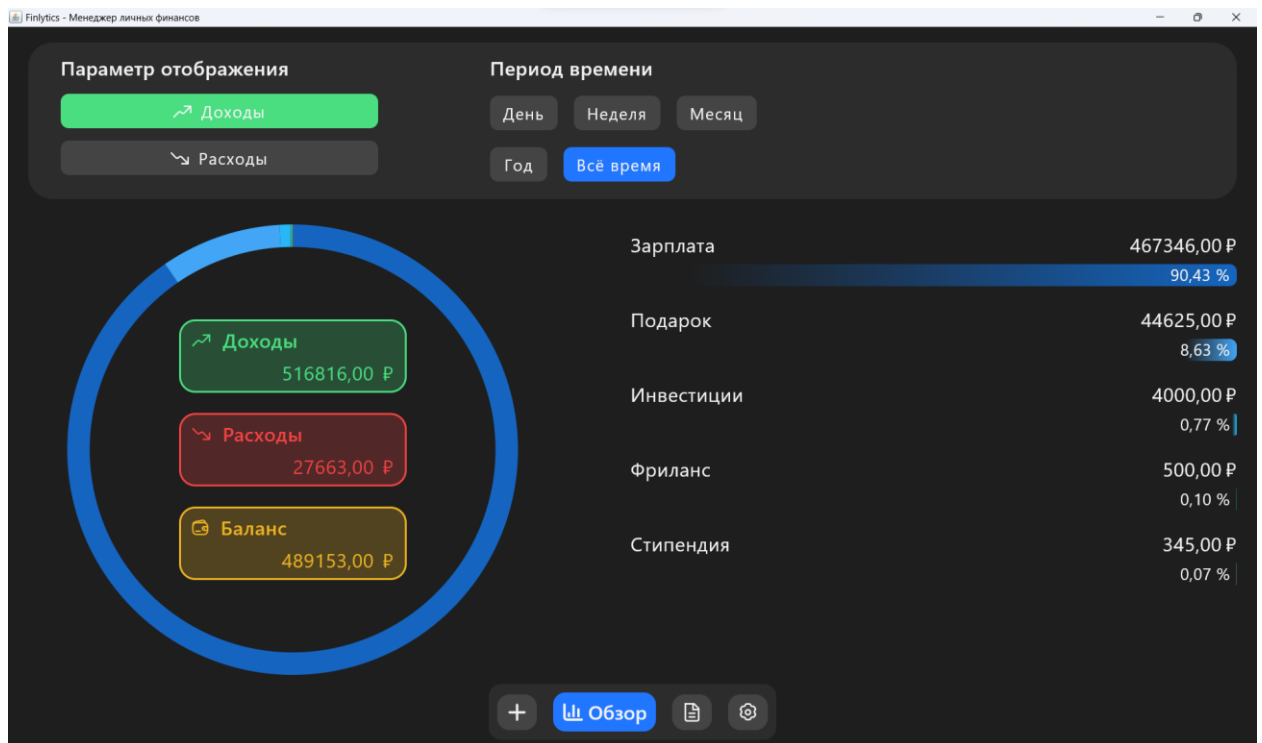


Рисунок 4.2 — Главный экран приложения (доходы)

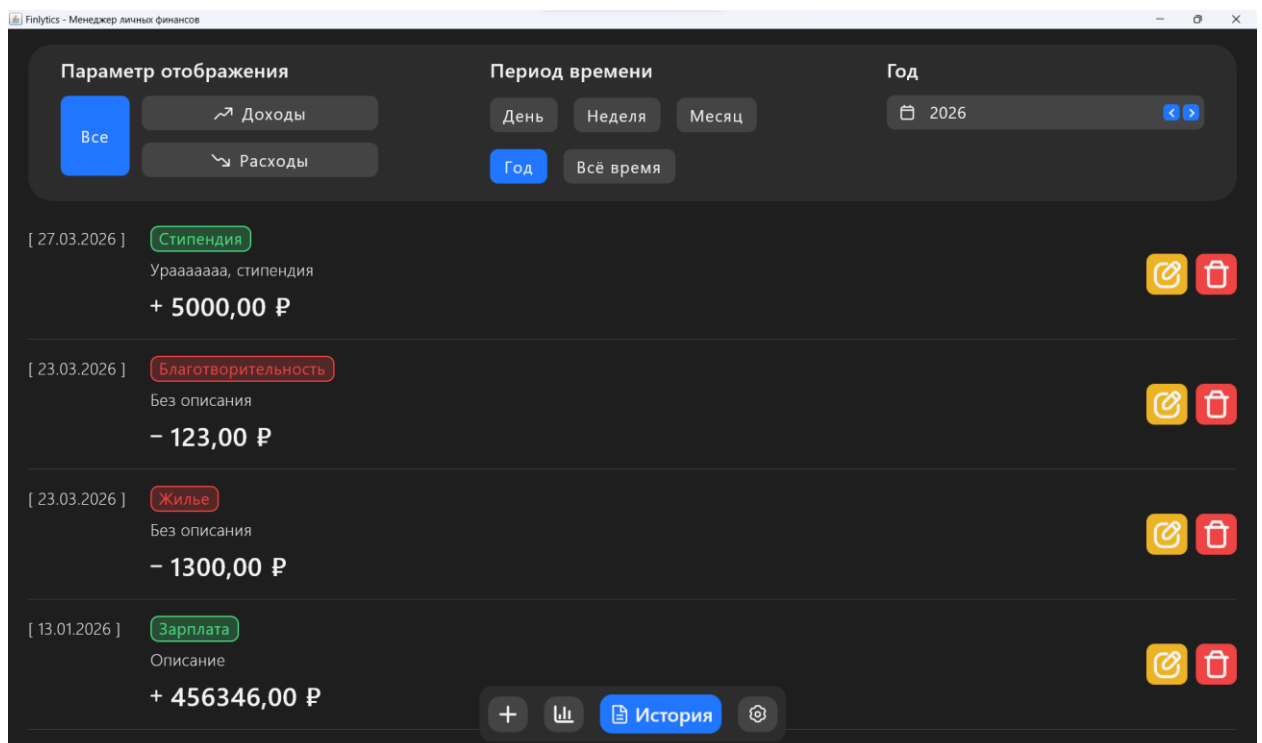


Рисунок 4.3 — История транзакций

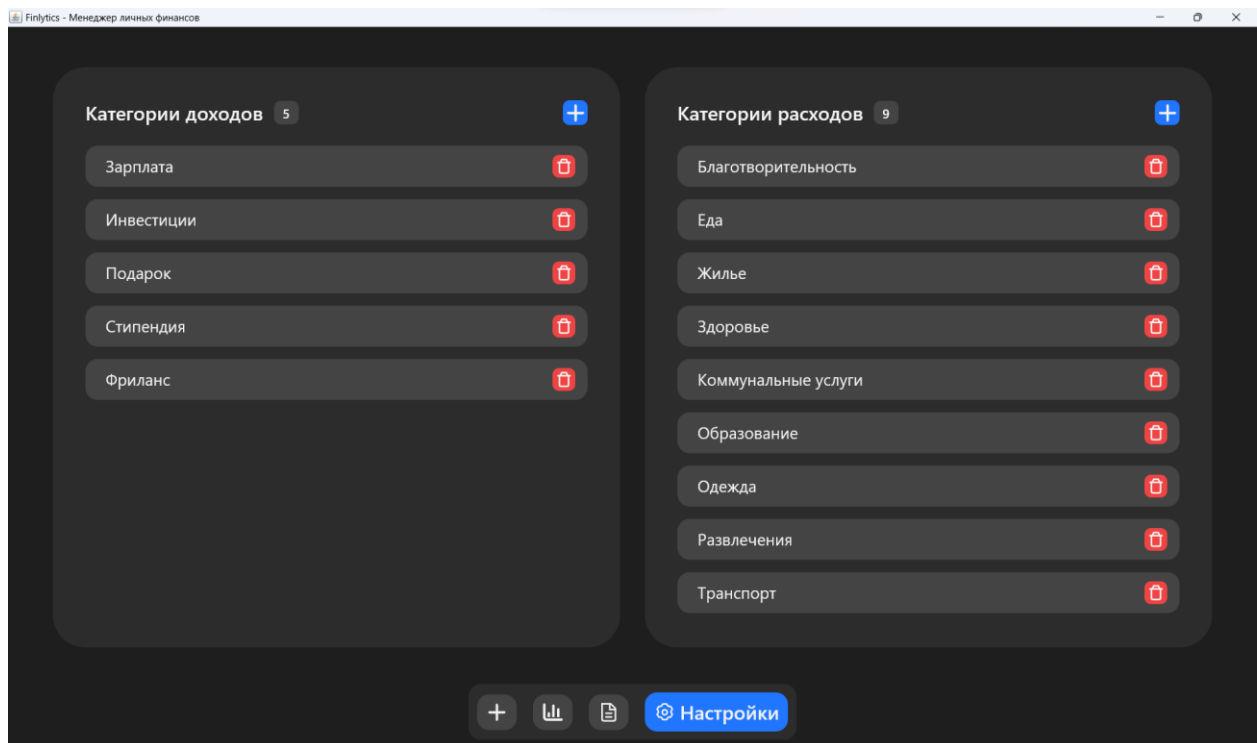


Рисунок 4.4 — Настройки категорий

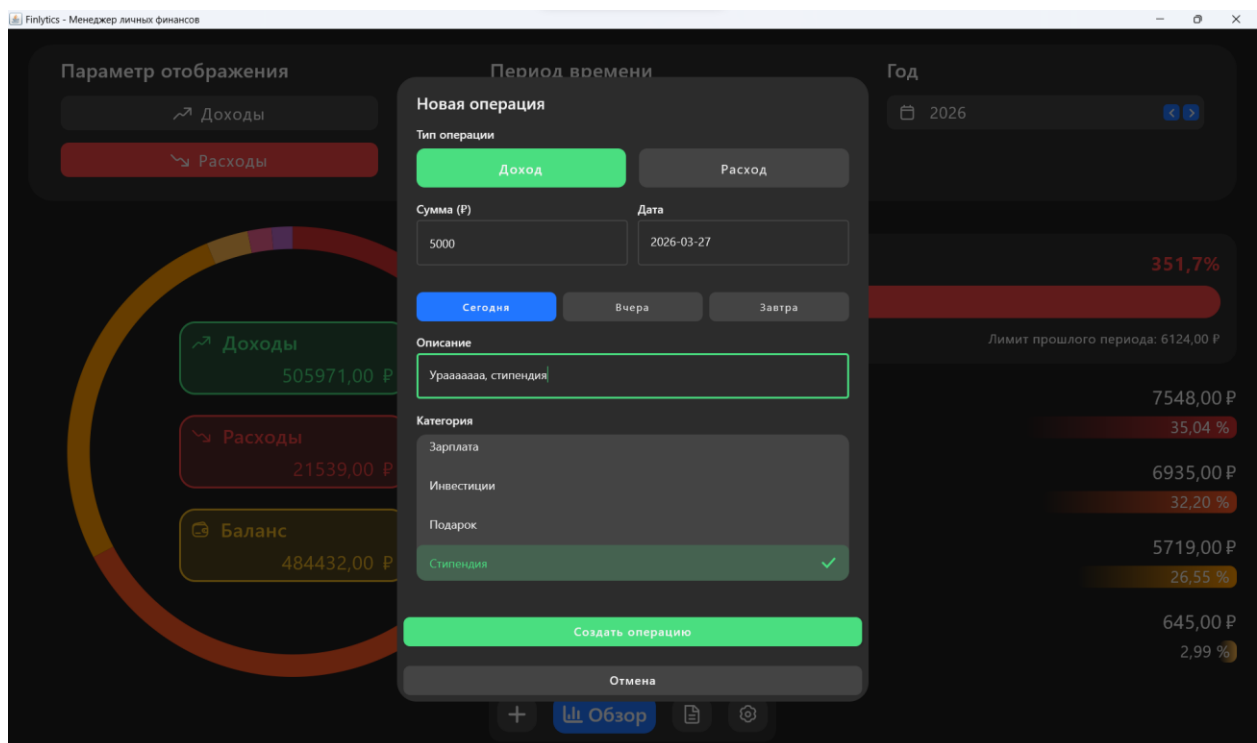


Рисунок 4.5 — Окно добавления/редактирования транзакции

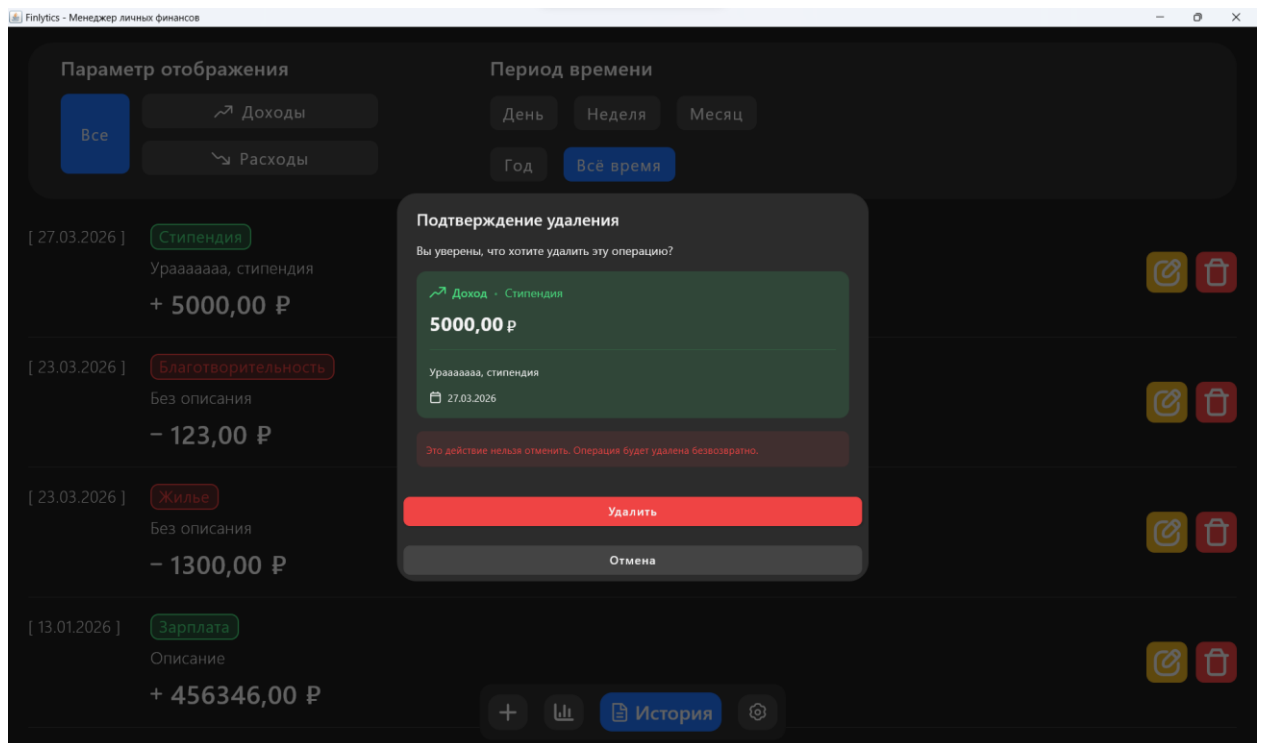


Рисунок 4.6 — Окно подтверждения удаления транзакции

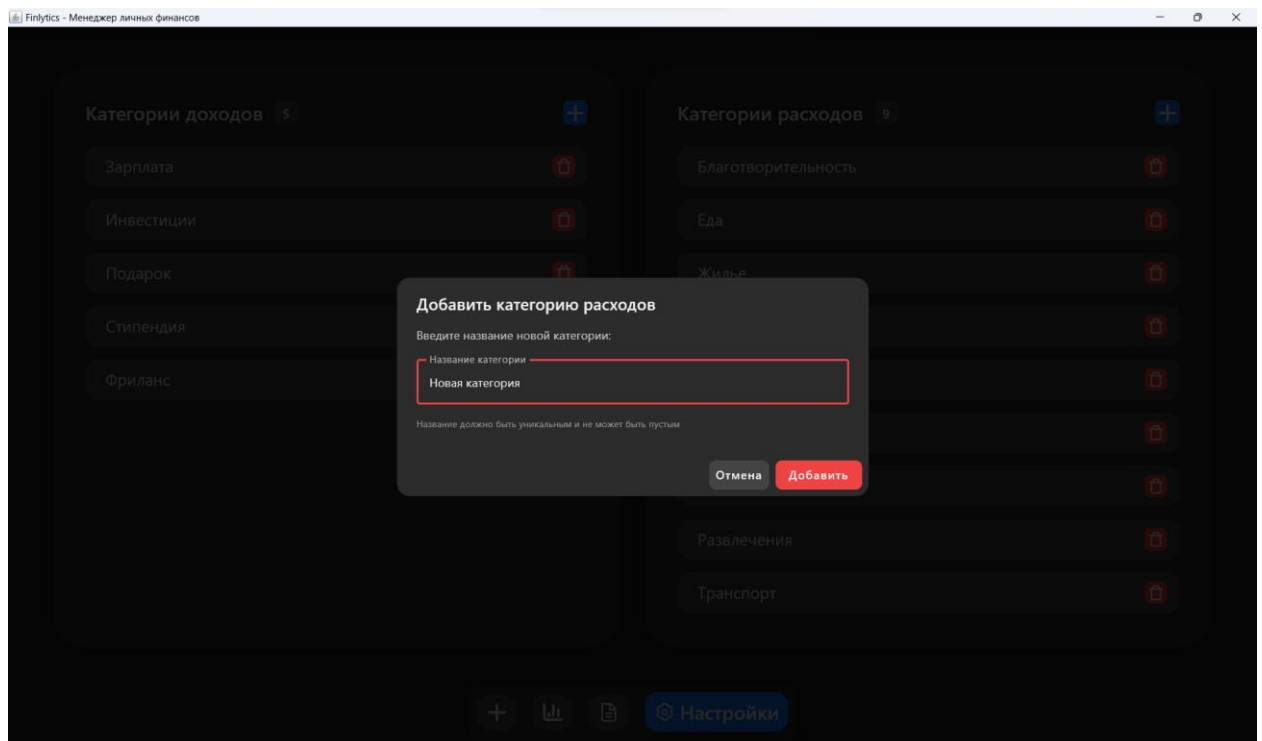


Рисунок 4.7 — Окно добавления категории доходов/расходов

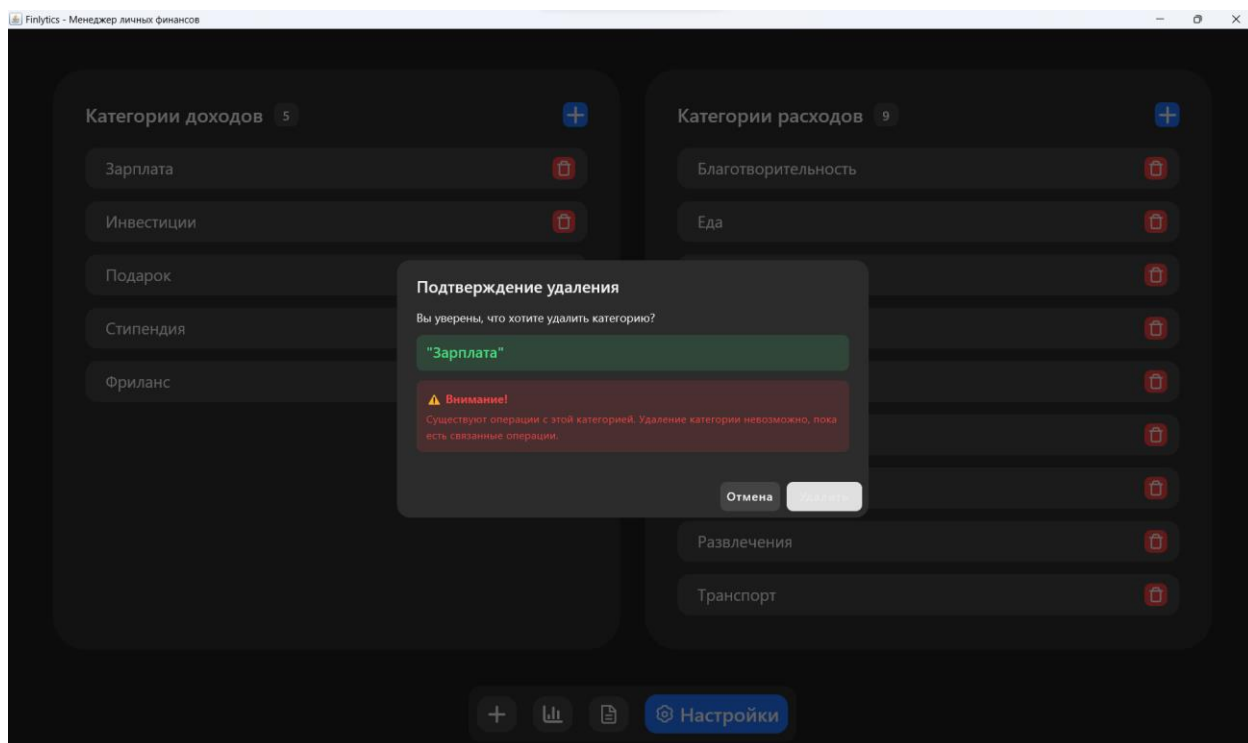


Рисунок 4.8 — Окно подтверждения удаления категории доходов/расходов

4.4. Чек-лист тестирования (Check-list)

№	Наименование тестового случая	Действия или Входные данные	Ожидаемый результат	Фактический результат	Примечание
ТС-01	Добавление новой операции (позитивный)	1. Нажать кнопку "Добавить". 2. Выбрать тип "Расход". 3. Ввести сумму "500". 4. Оставить текущую дату. 5. Выбрать категорию "Еда". 6. Ввести описание "Обед". 7. Нажать "Создать операцию".	1. Диалоговое окно закрывается. 2. На главном экране расходы увеличиваются на 500 руб., баланс уменьшается на 500 руб. 3. В истории появляется запись: Расход, 500 руб., Еда, Обед.	Диалог закрыт, статистика обновлена, запись добавлена	Окружение: Windows 11 Pro (22H2), разрешение 1920x1080, JRE 21. Предусловие: Приложение запущено, БД инициализирована.
ТС-02	Добавление операции с некорректной суммой (негативный)	1. Открыть диалог "Новая операция". 2. В поле "Сумма" ввести "-100". 3. Заполнить остальные поля корректно. 4. Нажать "Создать операцию".	1. Операция не создается. 2. Под полем "Сумма" появляется ошибка "Введите сумму > 0". 3. Диалоговое окно не закрывается.	Ошибка отображена, диалог открыт, операция не создана	Предусловие: Диалог "Новая операция" открыт.
ТС-03	Редактирование операции	1. В истории найти операцию "Обед". 2. Нажать кнопку редактирования (карандаш). 3. Изменить сумму на "750". 4. Изменить категорию на "Транспорт". 5. Нажать "Сохранить изменения".	1. Диалог закрывается. 2. В истории у операции сумма меняется на 750 руб., категория на "Транспорт". 3. На главном экране расходы пересчитаны.	Данные обновлены корректно	Предусловие: В системе существует операция "Обед" с суммой 500 руб.
ТС-04	Удаление операции	1. В истории найти любую операцию. 2. Нажать красную кнопку удаления (корзина).	1. Диалог подтверждения закрывается. 2. Операция исчезает из списка истории.	Операция удалена, статистика корректна	Предусловие: В системе существует хотя бы одна операция. Действия перед тестированием: Создана

		3. В диалоге подтверждения нажать "Удалить".	3. Статистика на главном экране обновлена.		тестовая операция "ТестУдаление" с суммой 100 руб.
ТС-05	Создание новой категории расходов	1. Открыть экран "Настройки". 2. В панели "Категории расходов" нажать кнопку "+". 3. Ввести название "Кафе". 4. Нажать "Добавить".	1. Диалог закрывается. 2. В списке категорий расходов появляется "Кафе". 3. При создании операции категория "Кафе" доступна для выбора.	Категория добавлена, доступна в выборе	Предусловие: Открыт экран "Настройки".
ТС-06	Удаление категории, используемой в операциях (негативный)	1. Создать операцию с категорией "Кафе". 2. Перейти в "Настройки". 3. Нажать удаление у категории "Кафе".	1. Открывается диалог с предупреждением о наличии операций. 2. Кнопка "Удалить" неактивна. 3. Категория остается в списке.	Удаление заблокировано, предупреждение отображено	Предусловие: Создана операция с категорией "Кафе". Ссылка на требование: Проверка внешнего ключа ON DELETE RESTRICT в SQLite.
ТС-07	Фильтрация данных по периоду на главном экране	1. Открыть экран "Обзор". 2. Выбрать период "Месяц". 3. С помощью стрелок выбрать "Март 2024".	1. Кнопка "Месяц" подсвечена синим. 2. В поле отображается "03.2024". 3. Доходы, расходы и баланс пересчитаны за Март 2024.	Фильтрация работает корректно	Предусловие: В системе есть операции за Март 2024 и Апрель 2024.
ТС-08	Переключение между доходами и расходами на диаграмме	1. Открыть экран "Обзор". 2. Нажать кнопку "Доходы".	1. Кнопка "Доходы" подсвечена зеленым. 2. Диаграмма показывает структуру доходов. 3. Список категорий справа показывает категории доходов с суммами и процентами.	Переключение работает корректно	Предусловие: В системе есть операции доходов и расходов за выбранный период.
ТС-09	Валидация даты в будущее (негативный)	1. Открыть диалог "Новая операция". 2. В поле "Дата" ввести дату +2 дня от текущей. 3. Заполнить сумму и категорию.	1. Операция не создается. 2. Под полем "Дата" появляется ошибка "Дата не может быть более чем на 1 день в будущем".	Ошибка отображена, операция не создана	Тестовые данные: Дата = текущая_дата + 2 дня (например, 2026-04-04 при текущей дате 2026-04-02).

		4. Нажать "Создать операцию".			Предусловие: Открыт диалог "Новая операция".
ТС-10	Работа прогресс-бара "Гонка с призраком"	1. На главном экране выбрать период "Месяц". 2. Убедиться, что есть данные за текущий и прошлый месяц. 3. Проверить заполнение прогресс-бара.	1. Компонент "Гонка с призраком" отображается. 2. Прогресс-бар заполнен на (Расходы_тек / Расходы_прошл). 3. Цвет: зеленый (<80%), желтый (80-100%), красный (>100%). 4. Отображаются суммы "Потрачено" и "Лимит прошлого периода".	Прогресс-бар работает корректно	Тестовые данные: Расходы текущего месяца = 5000 руб., расходы прошлого месяца = 10000 руб. (соотношение 50%). Предусловие: Выбран период "Месяц", есть данные за текущий и прошлый месяц. Ожидаемый цвет: Зеленый.

ЗАКЛЮЧЕНИЕ

В рамках проекта "Finlytics: Офлайн-менеджер финансов" было успешно разработано десктопное приложение для управления личными финансами на языке Kotlin с использованием современных технологий и подходов.

Достигнутые результаты:

- реализован полнофункциональный менеджер финансов с возможностью учета доходов и расходов, категоризацией операций и аналитикой;
- создан интуитивно понятный пользовательский интерфейс на основе Compose for Desktop с темной темой и акцентными цветами;
- разработана отказоустойчивая архитектура MVVM, обеспечивающая четкое разделение ответственности компонентов;
- реализована офлайн-работа приложения с использованием SQLite в качестве локального хранилища данных;
- внедрена система визуализации данных с кастомными диаграммами для наглядного представления финансовой статистики;
- обеспечена кросс-платформенность - приложение работает на Windows, macOS и Linux.

Ключевые технические достижения:

- применение реактивного программирования с Kotlin Flows для автоматического обновления UI;
- реализация кастомных Compose-компонентов (диаграммы, диалоговые окна с валидацией);
- создание модульной структуры проекта с четким разделением на слои (UI, ViewModel, Repository, Data);

- интеграция SQLite с автоматической миграцией и инициализацией данных;
- настройка системы логирования с поддержкой UTF-8 для русскоязычного интерфейса.

Практическая значимость:

Разработанное приложение представляет собой готовое к использованию решение для студентов и других пользователей, нуждающихся в простом и эффективном инструменте для контроля личного бюджета без необходимости подключения к интернету. Особенностью приложения является его полная автономность - все данные хранятся локально, что обеспечивает конфиденциальность и доступность в любое время.

Ссылка на GitHub-репозиторий разработанного проекта:
<https://github.com/Hohrandrey/Finlytics>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 34.602—2020 ("Техническое задание на создание автоматизированной системы") является стандартом, который определяет общие требования к разработке автоматизированных систем (АС). Он используется в разных отраслях, включая медицину, и применим для разработки автоматизированных информационных систем (АИС).

2. ГОСТ 34.201—2020. Межгосударственный стандарт. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем: Приказом Федерального агентства по техническому регулированию и метрологии № 1521-ст от 19 ноября 2021 г.: дата введения 2022-01-01. – М.: Российский институт стандартизации, 2021. – 10 с.

3. Блоштин А.В., Лаптев Д.В. Современные подходы к проектированию клиент-серверных приложений // Программные системы: теория и приложения. 2021. №2. URL: <https://elibrary.ru/item.asp?id=46523678> (дата обращения: 26.11.2025)

4. Android Developers — официальный сайт документации по разработке на Kotlin: <https://developer.android.com/kotlin> (дата обращения: 26.11.2025).

5. Kotlin Documentation — полный справочник по языку Kotlin: <https://kotlinlang.org/docs> (дата обращения: 26.11.2025).

6. Жемеров С., Исакова С. Kotlin в действии. 2-е издание. М.: Питер, 2022. URL: <https://www.piter.com/product/kotlin-v-dejstvii> (дата обращения: 26.11.2025).

7. Документация JUnit — для тестирования Kotlin-приложений: <https://junit.org/junit5> (дата обращения: 26.11.2025).